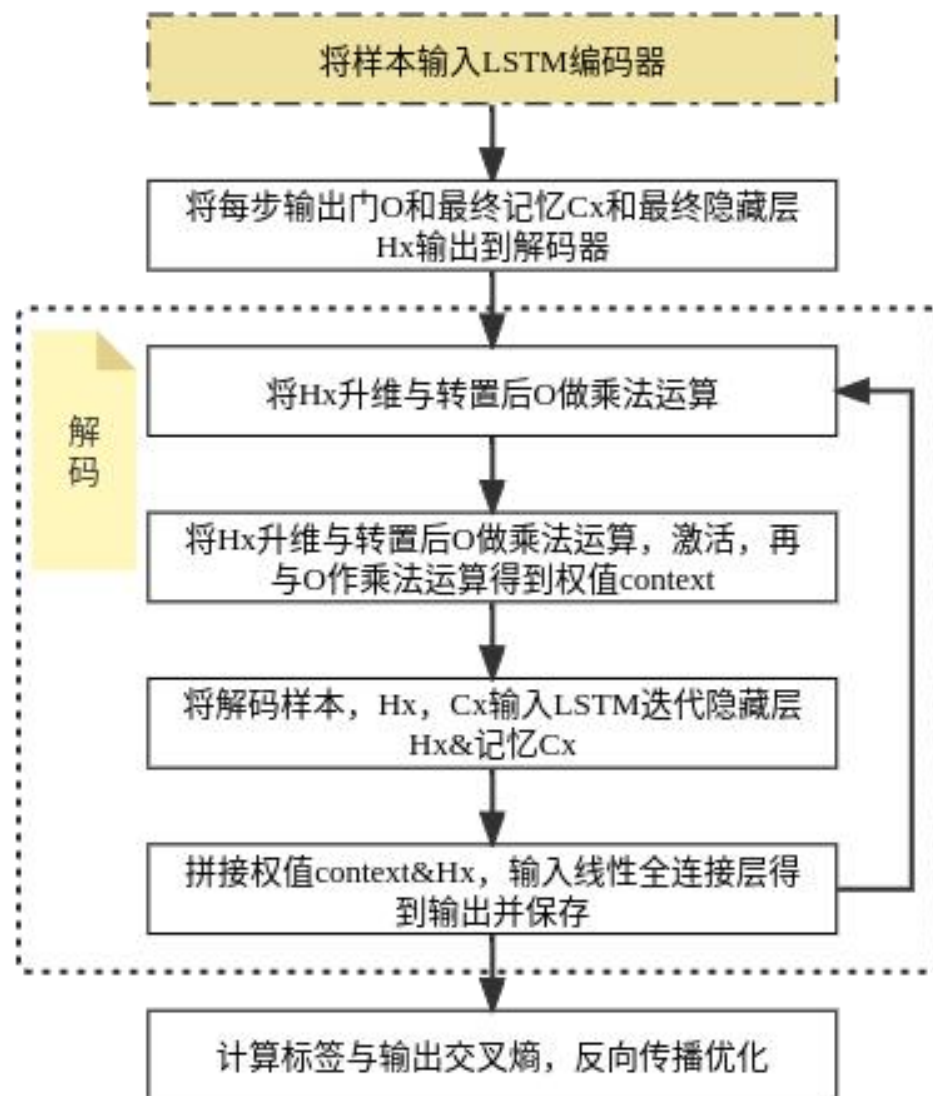
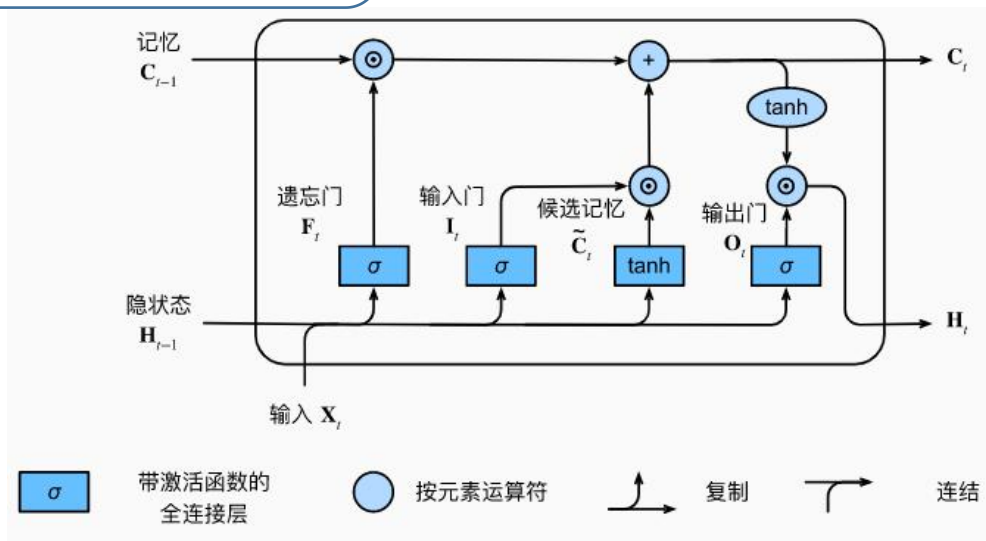


NLP_attention之后

2023.03.20



attention原理



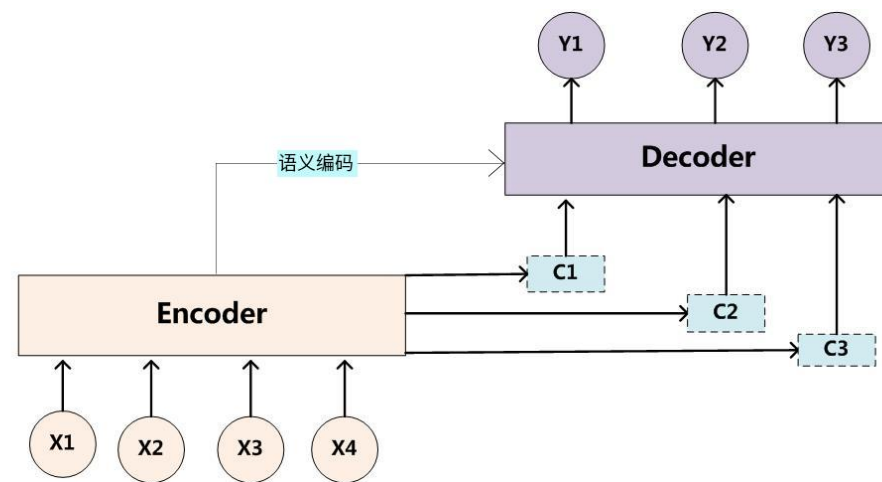
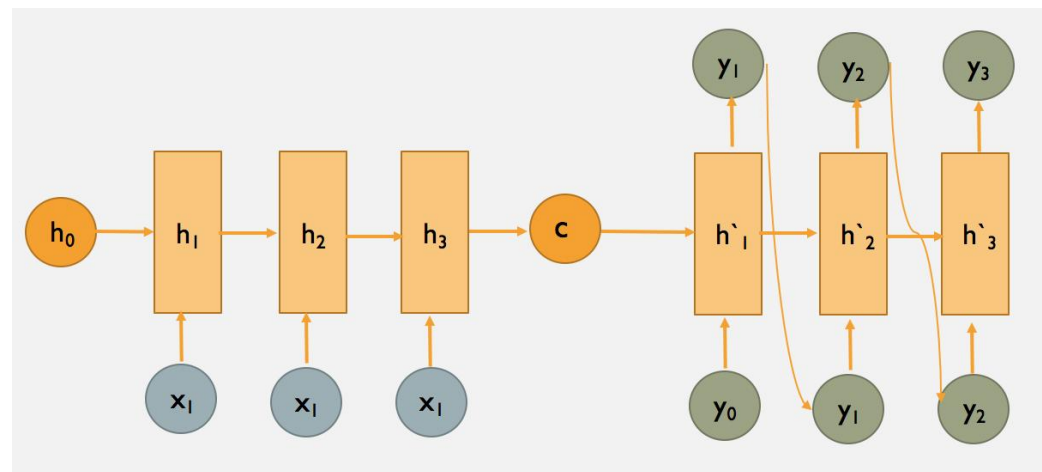
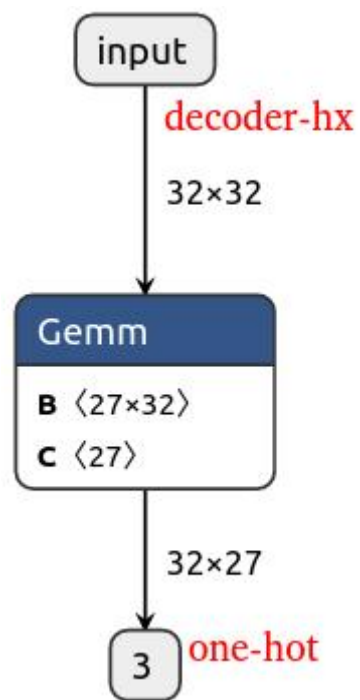
$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i), \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f), \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o), \end{aligned}$$

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t.$$

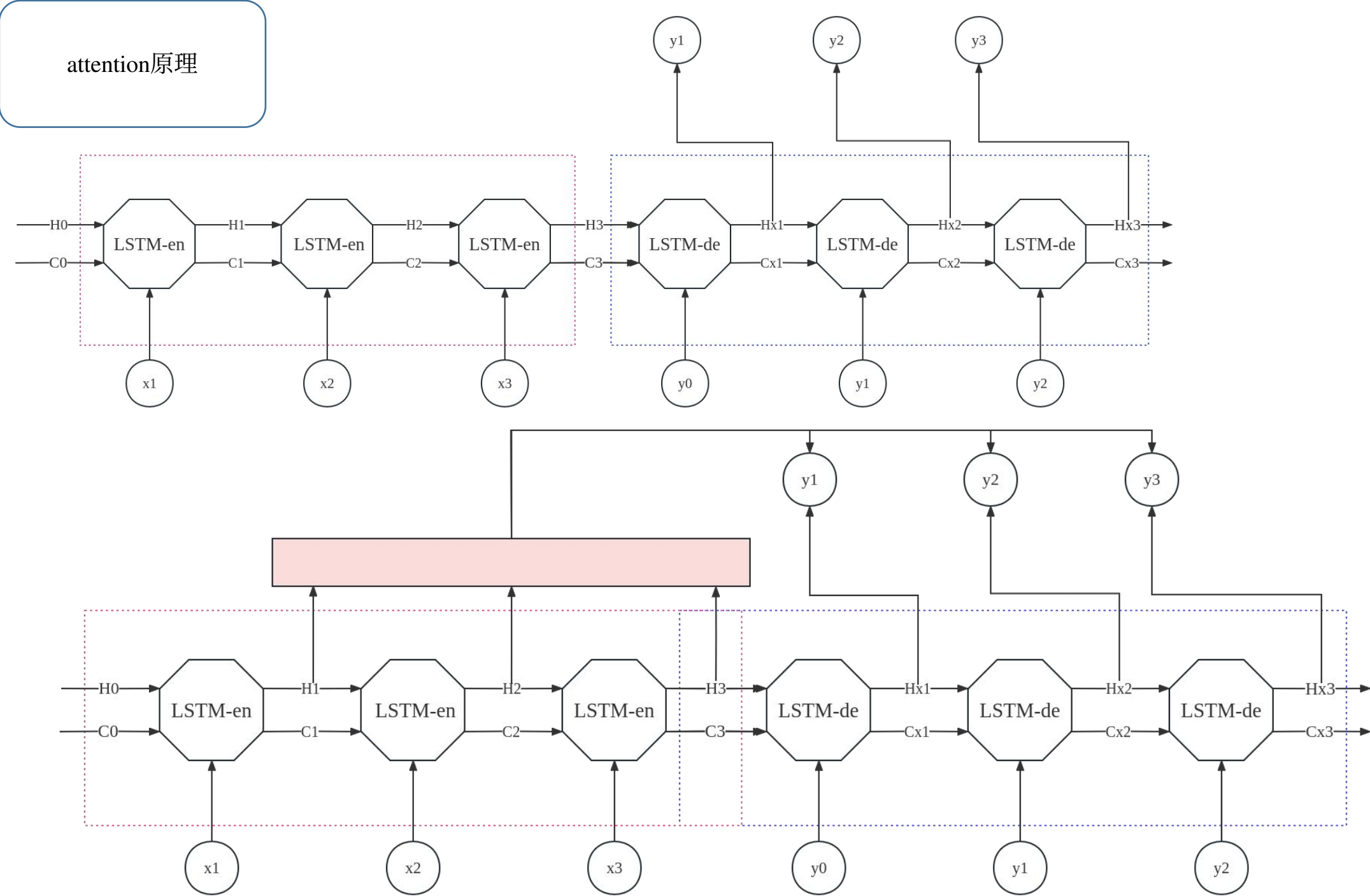
$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

$$H_t = O_t \odot \tanh(C_t)$$

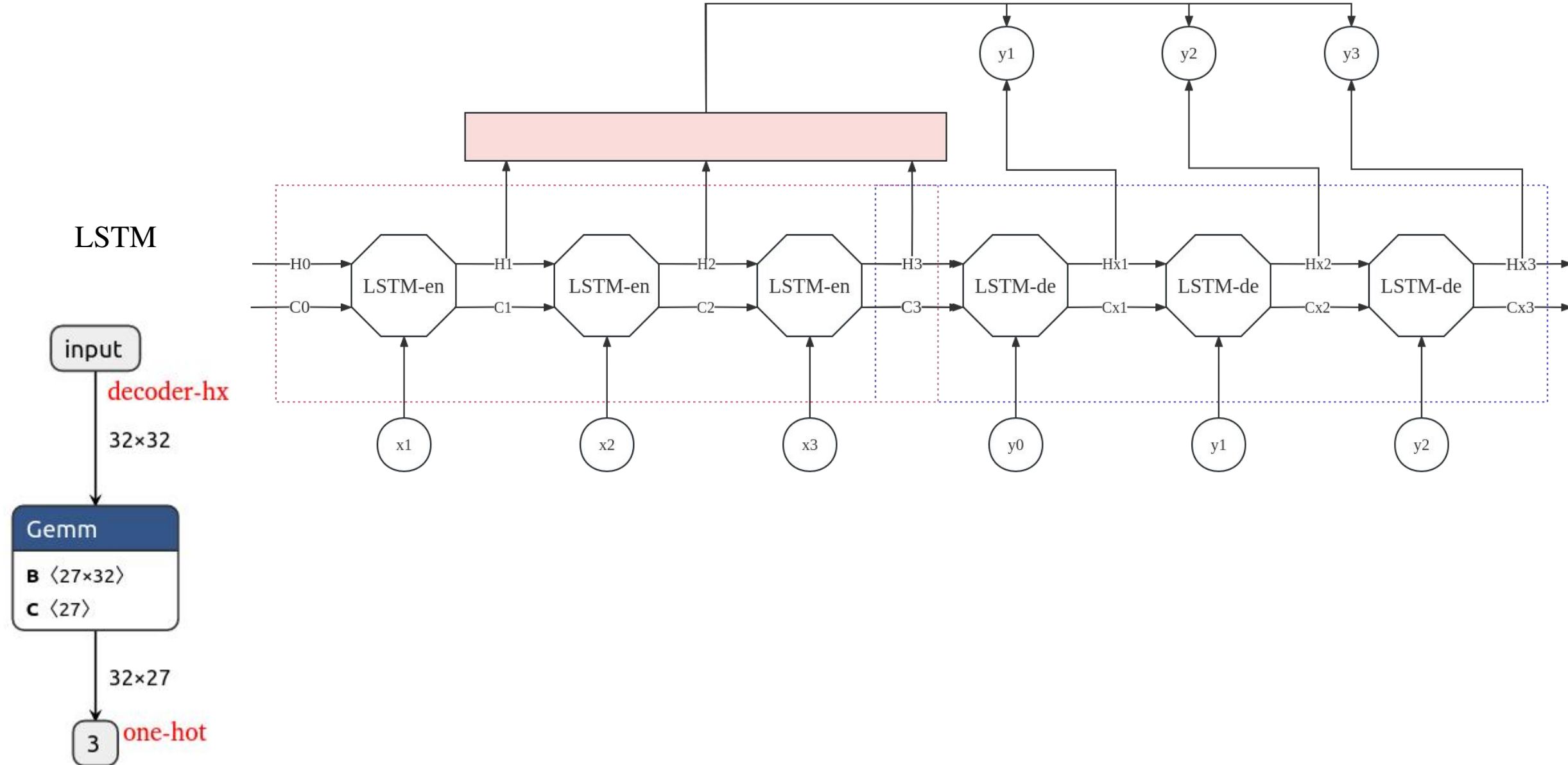
LSTM

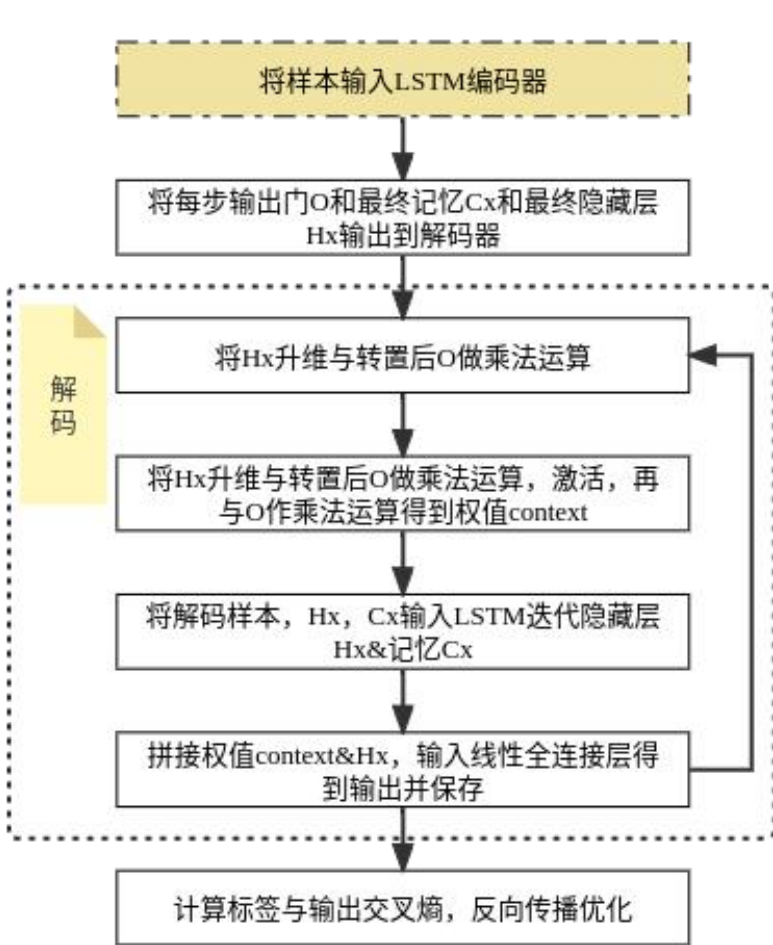


attention原理



LSTM



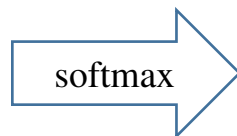


Hx_i							
--------	--	--	--	--	--	--	--

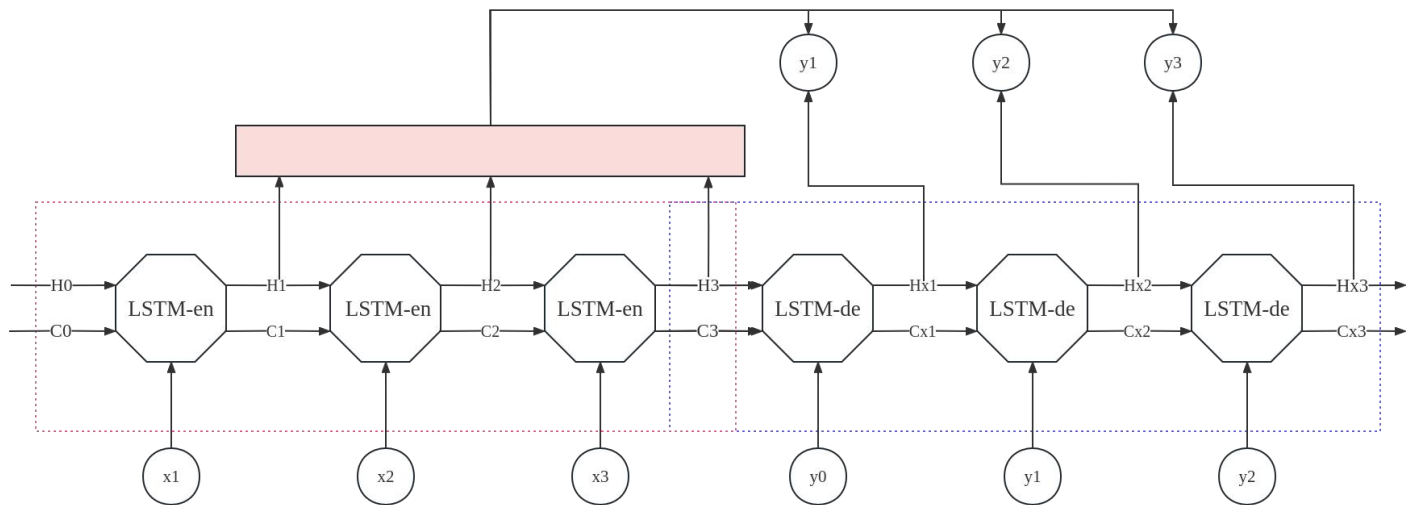
O		
$H1$	$H2$	$H3$



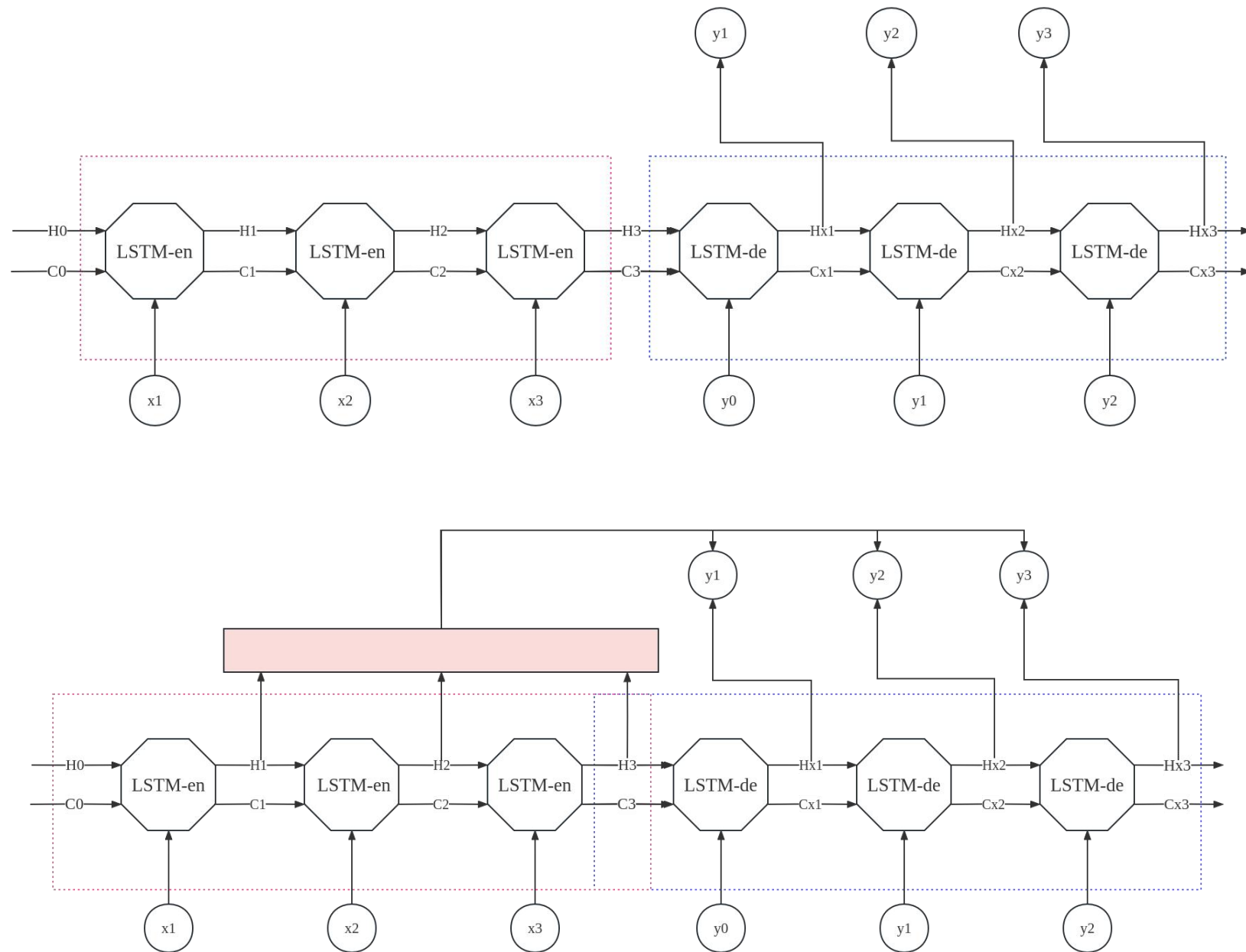
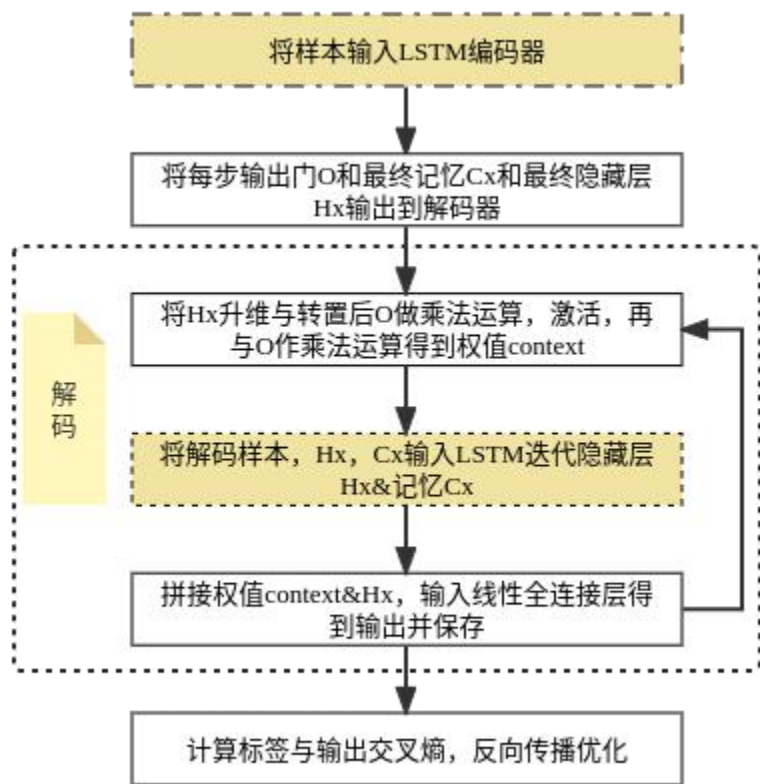
et		
$et1$	$et2$	$et3$

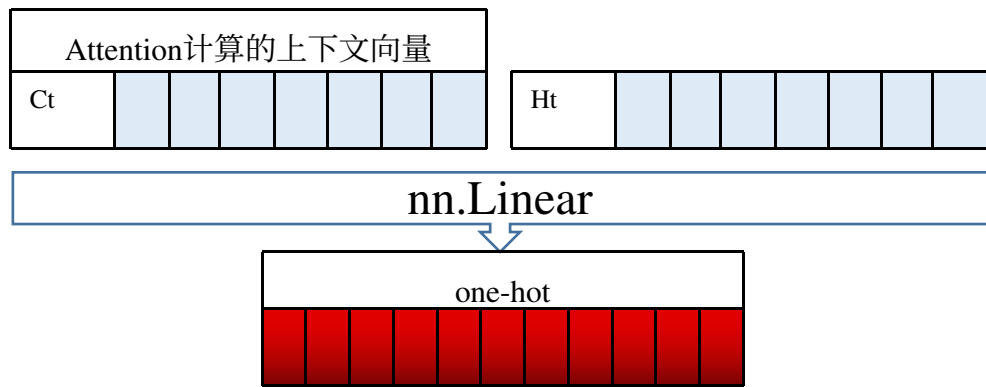


αt		
$\alpha t1$	$\alpha t2$	$\alpha t3$

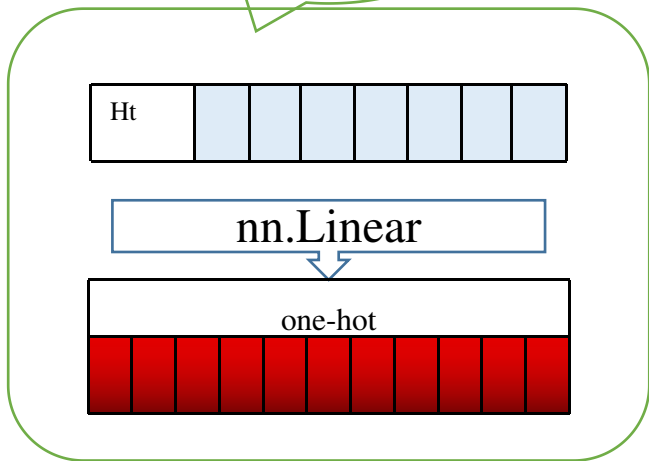


α





LSTM



- 这里的attention本质上讲是对解码器 LSTM隐藏层进行的一个修正

$$e_t = [a(h'_{t-1}, h_1), a(h'_{t-1}, h_2), \dots, a(h'_{t-1}, h_N)]$$

函数 a 用于计算Decoder h'_{t-1} 与Encoder第 i 个神经元的相关性

$$a(h'_{t-1}, h_i) = h_i^T h'_{t-1}$$

$$a(h'_{t-1}, h_i) = h_i^T W h'_{t-1}$$

$$a(h'_{t-1}, h_i) = \tanh(W_1 h_i + W_2 h'_{t-1})$$

Attention 相关性

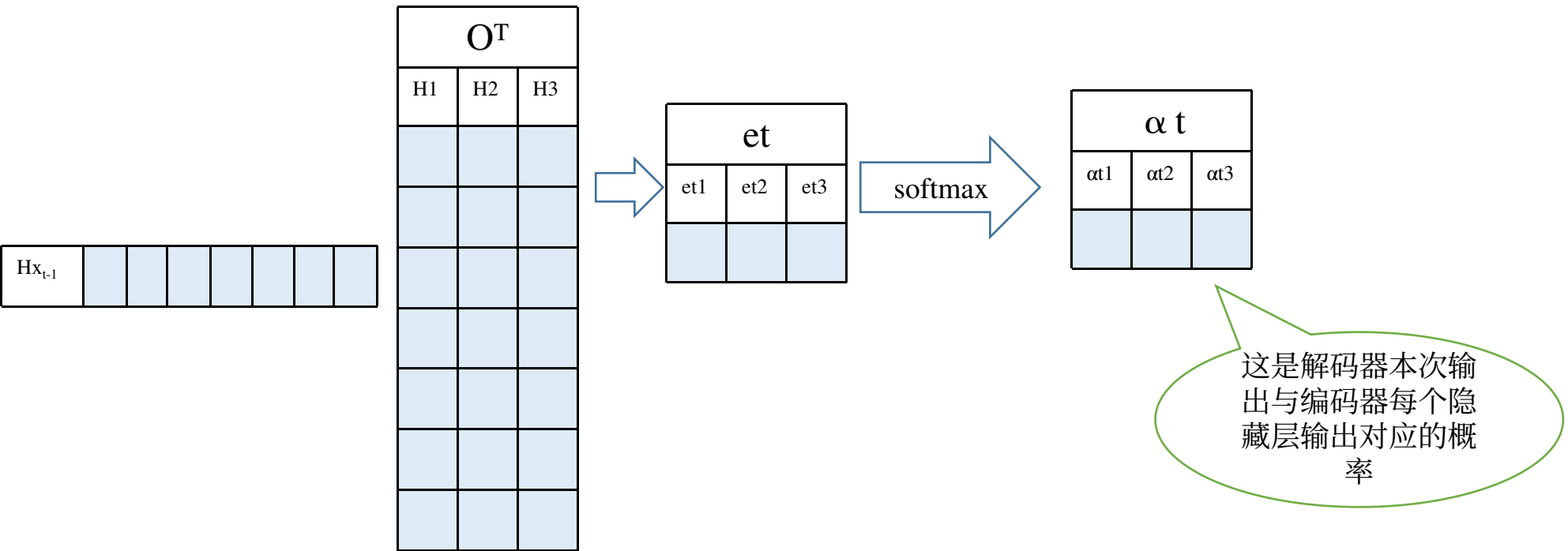
上面得到相关性向量 e_t 后，需要进行归一化，使用 softmax 归一化。然后用归一化后的系数融合 Encoder 的多个隐藏层向量得到 Decoder 当前神经元的上下文向量 c_t ：

$$\alpha_t = \text{softmax}(e_t)$$

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^N \exp(e_{tj})}$$

$$c_t = \sum_{i=1}^N \alpha_{ti} h_i$$

使用 Attention 计算上下文向量 c



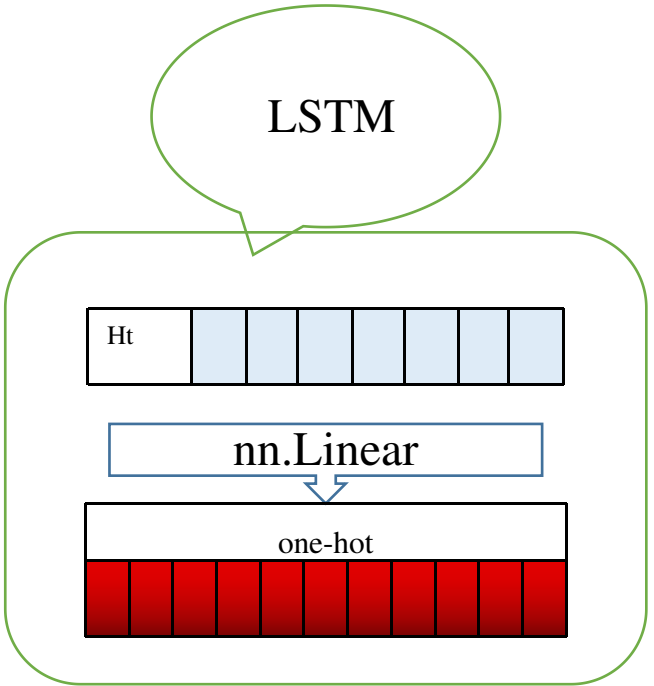
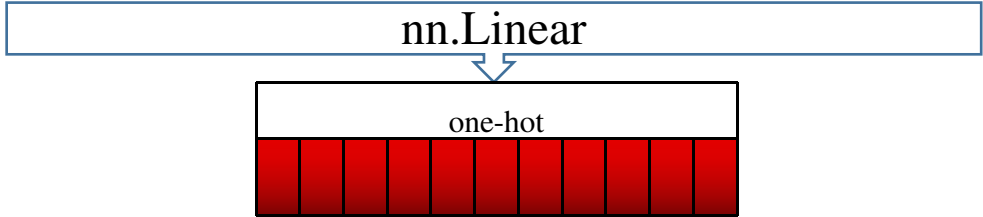
α_t		
α_1	α_2	α_3

O	H1							
	H2							
	H3							

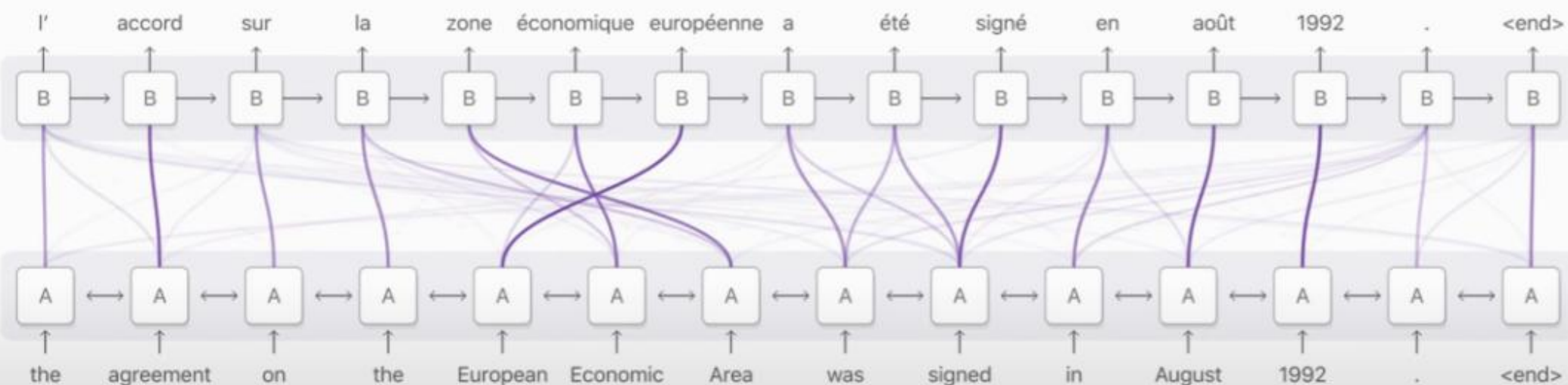
Attention计算的上下文向量							
Ct							

Attention计算的上下文向量							
Ct							

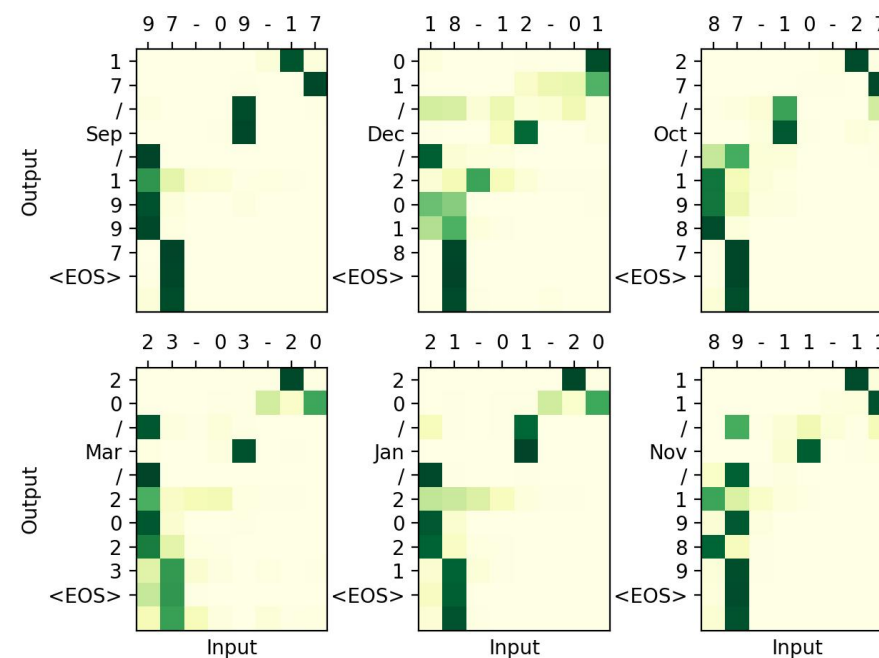
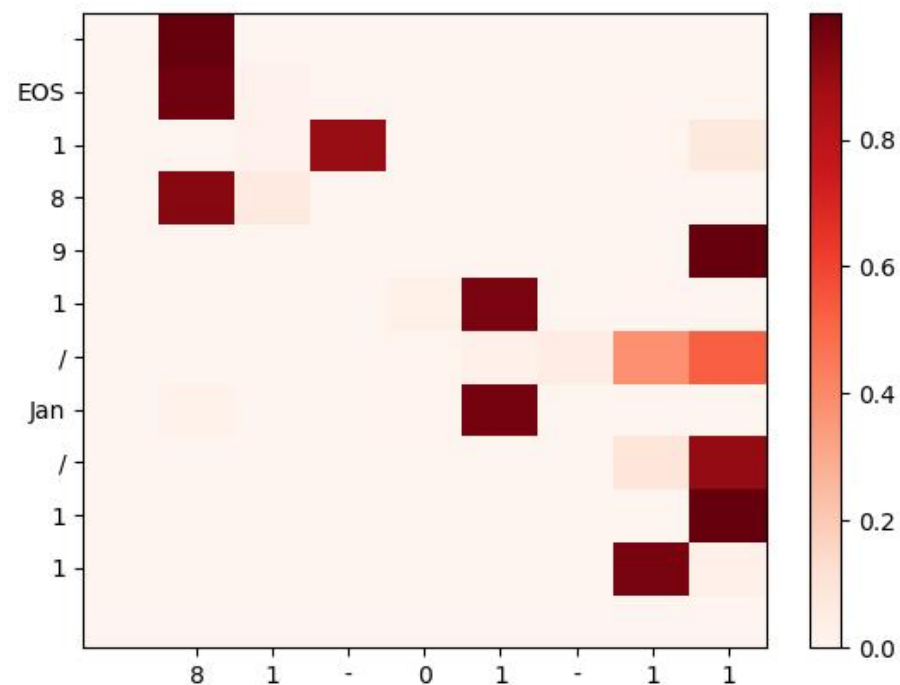
Ht							



Decoder RNN (target language: French)

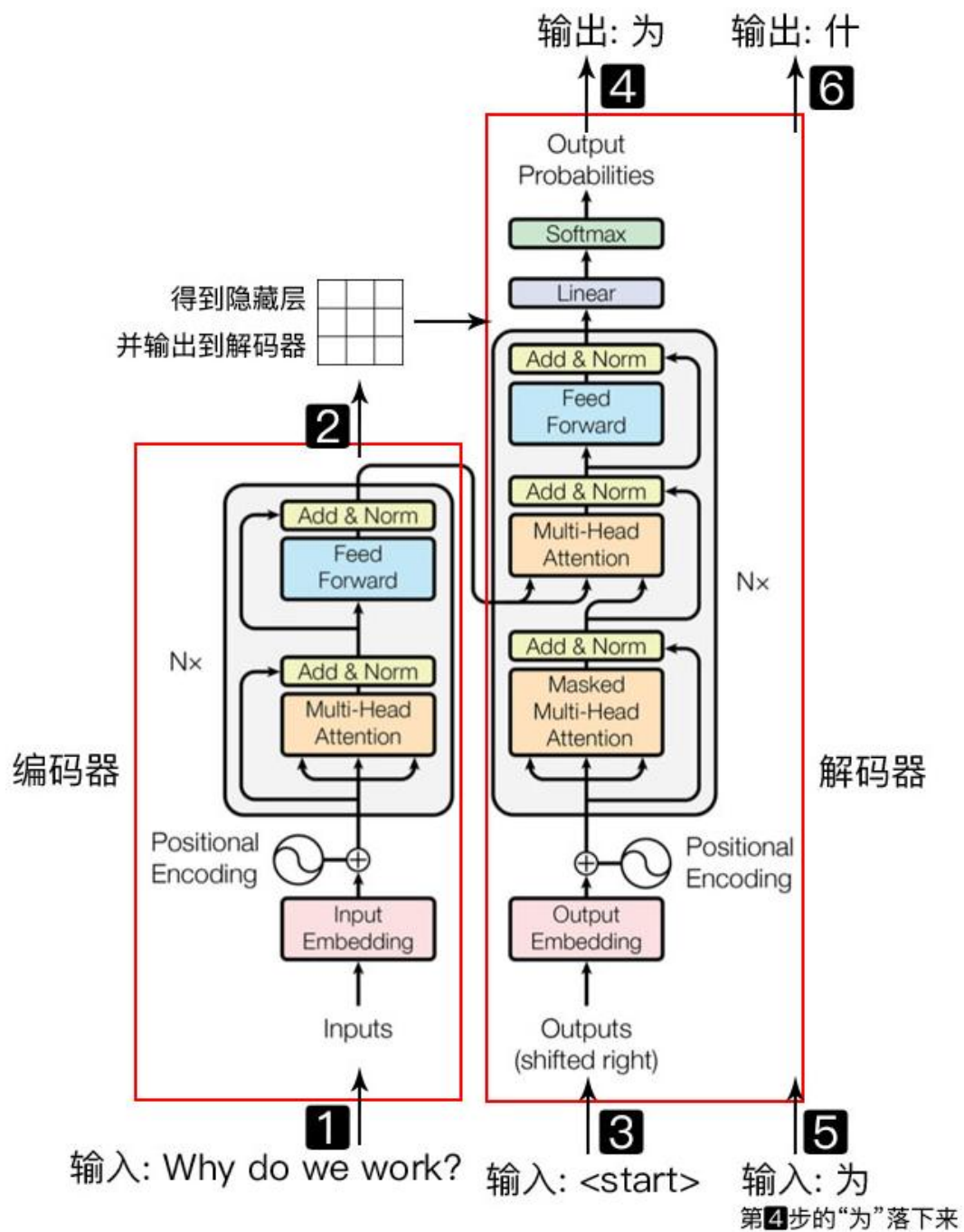


Encoder RNN (source language: English)



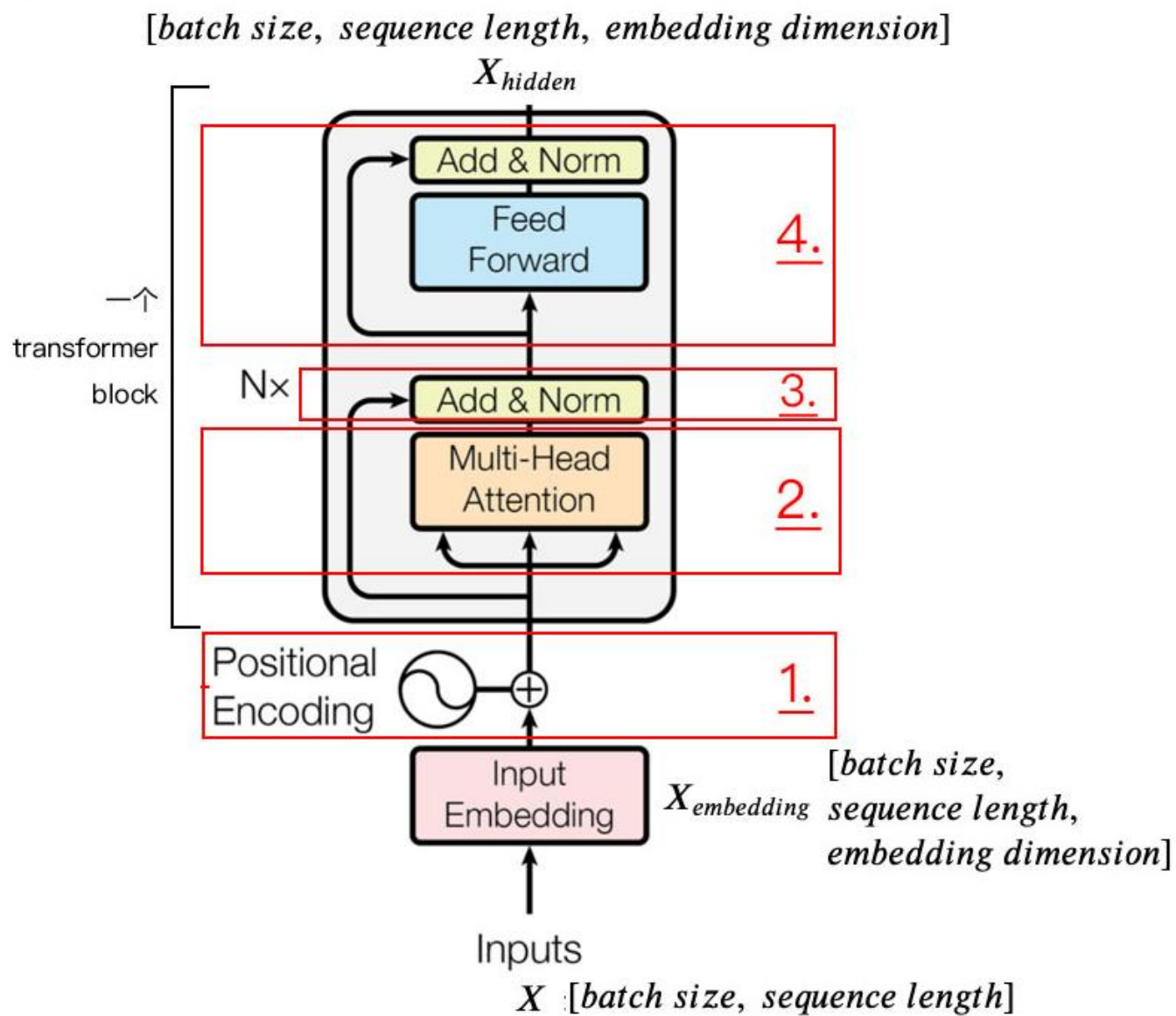
Transformer

transformer整体
=encoder+decoder



encoder

从自然语言序列
经过计算
到隐藏层的过程




```

model Transformer(
  (encoder): Encoder(
    (src_emb): Embedding(6, 512)
    (pos_emb): PositionalEncoding(
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (layers): ModuleList(
      (0): EncoderLayer(
        (enc_self_attn): MultiHeadAttention(
          (W_Q): Linear(in_features=512, out_features=512, bias=False)
          (W_K): Linear(in_features=512, out_features=512, bias=False)
          (W_V): Linear(in_features=512, out_features=512, bias=False)
          (fc): Linear(in_features=512, out_features=512, bias=False)
        )
        (pos_ffn): PoswiseFeedForwardNet(
          (fc): Sequential(
            (0): Linear(in_features=512, out_features=2048, bias=False)
            (1): ReLU()
            (2): Linear(in_features=2048, out_features=512, bias=False)
          )
        )
      )
    )
  )

  (decoder): Decoder(
    (tgt_emb): Embedding(9, 512)
    (pos_emb): PositionalEncoding(
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (layers): ModuleList(
      (0): DecoderLayer(
        (dec_self_attn): MultiHeadAttention(
          (W_Q): Linear(in_features=512, out_features=512, bias=False)
          (W_K): Linear(in_features=512, out_features=512, bias=False)
          (W_V): Linear(in_features=512, out_features=512, bias=False)
          (fc): Linear(in_features=512, out_features=512, bias=False)
        )
        (dec_enc_attn): MultiHeadAttention(
          (W_Q): Linear(in_features=512, out_features=512, bias=False)
          (W_K): Linear(in_features=512, out_features=512, bias=False)
          (W_V): Linear(in_features=512, out_features=512, bias=False)
          (fc): Linear(in_features=512, out_features=512, bias=False)
        )
        (pos_ffn): PoswiseFeedForwardNet(
          (fc): Sequential(
            (0): Linear(in_features=512, out_features=2048, bias=False)
            (1): ReLU()
            (2): Linear(in_features=2048, out_features=512, bias=False)
          )
        )
      )
    )
  )

  (projection): Linear(in_features=512, out_features=9, bias=False)

```

encoder

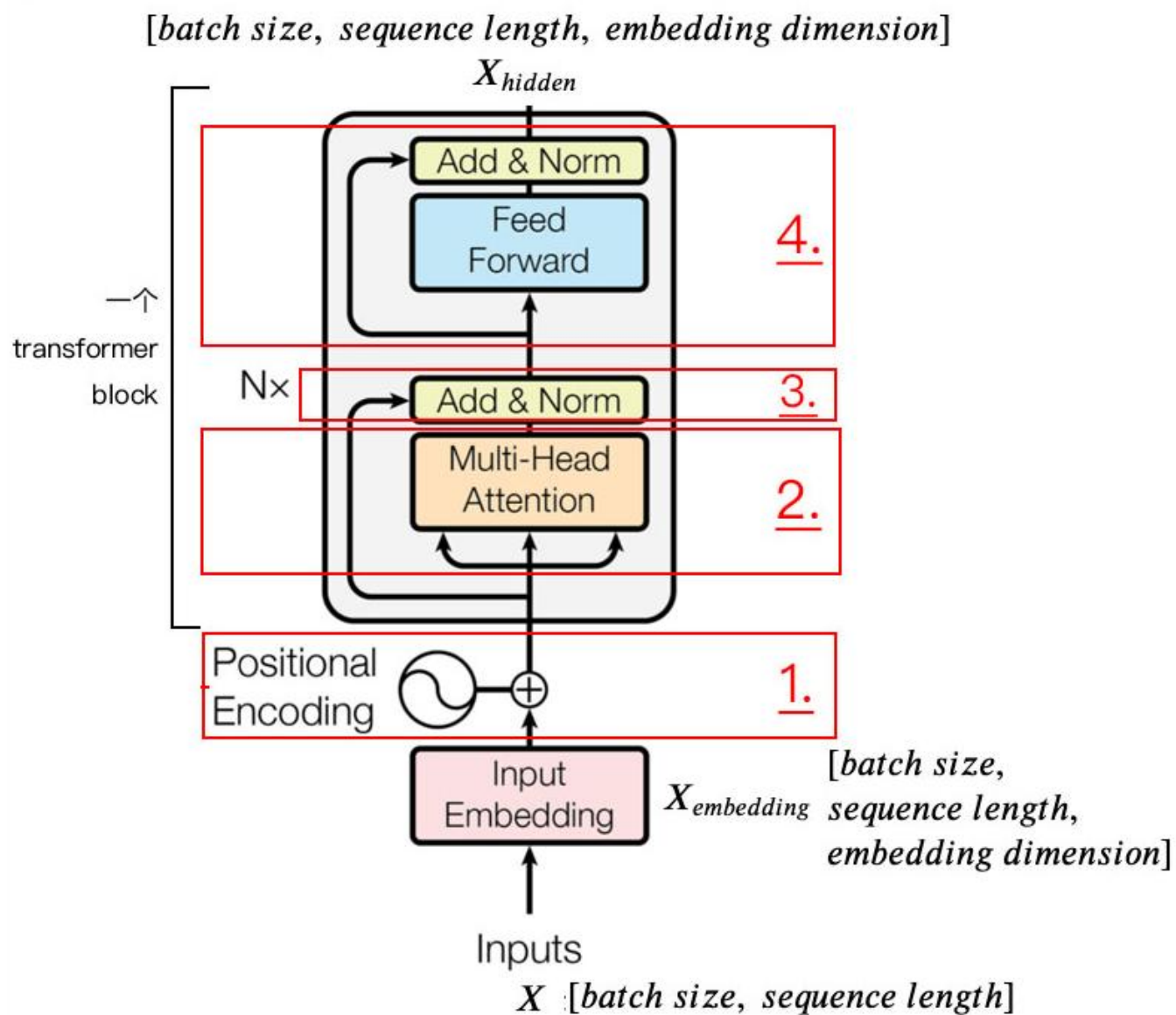
enc_input(编码样本)	dec_input (解码样本)	dec_outputdec_input (标签)
ich mochte ein bier P	S i want a beer .	i want a beer . E
ich mochte ein cola P	S i want a coke .	i want a coke . E
5	6	6

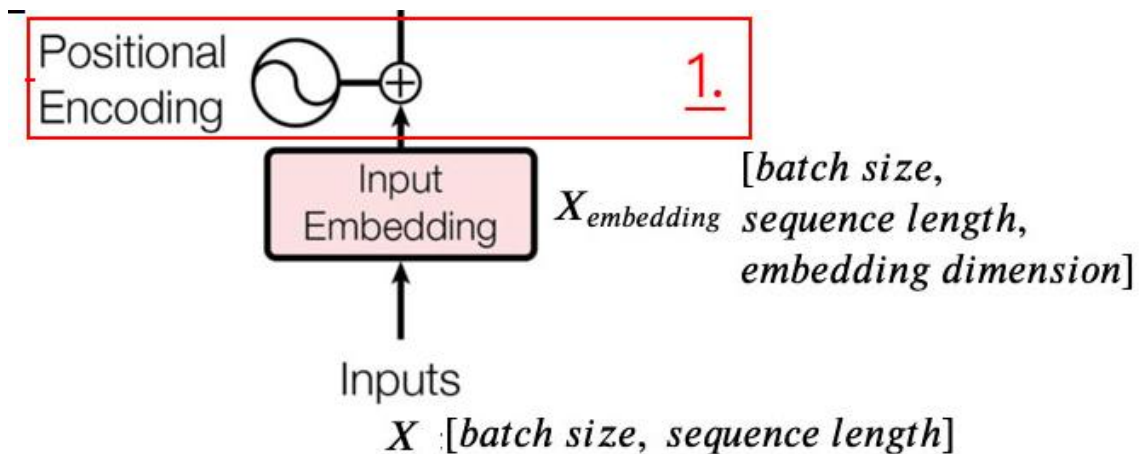
Src_vocab	
0	P
1	ich
2	mochte
3	ein
4	bier
5	cola

tgt_vocab	
0	P
1	i
2	want
3	a
4	beer
5	coke
6	S
7	E
8	.

encoder

从自然语言序列
经过计算
到隐藏层的过程





● 将位置嵌入和字嵌入相加，然后作为输入

$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$

- t : 句中该词序号
- d : 词向量维度 (512)
- i : 嵌入的维度序号
- nn.Dropout: 结果放大 $1/(1-p)$ 即10/9倍

$$\omega_k = \frac{1}{10000^{2k/d}}$$

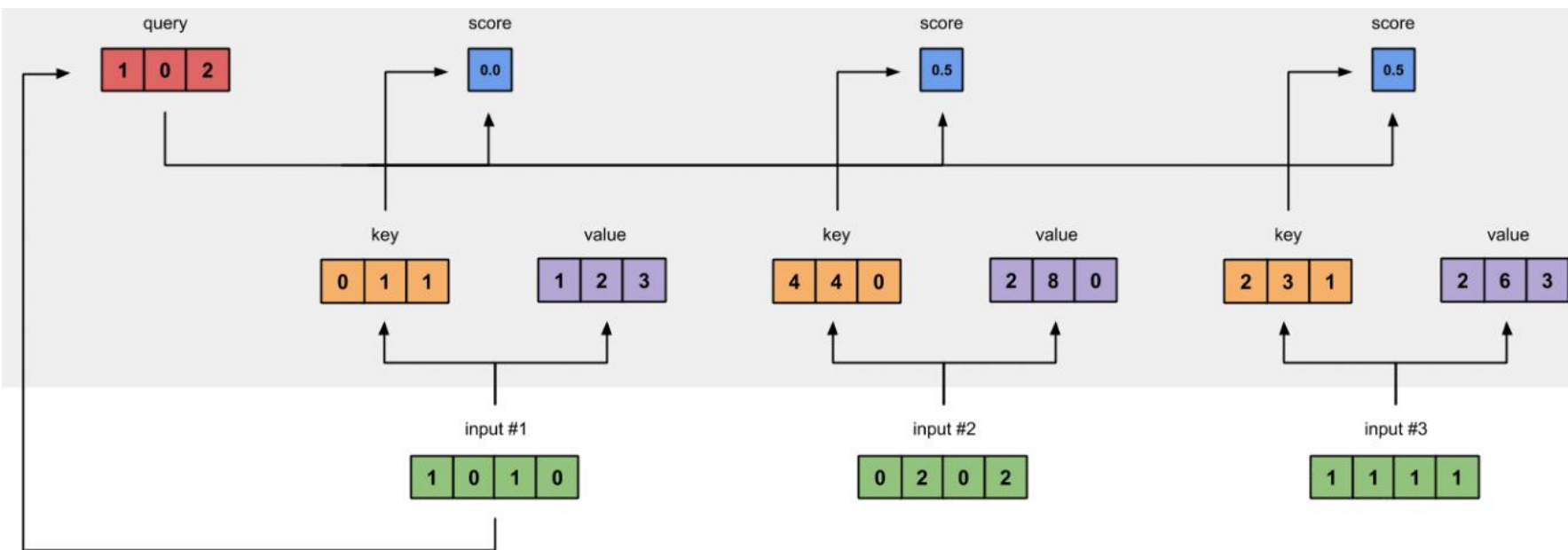
batch1	1	2	3	4	0
batch2	1	2	3	5	0

Word Embedding(1-->512)

batch1	1	2	3	4	0
	...	...	...	...	...
batch2	1	2	3	5	0
	...	...	...	...	...

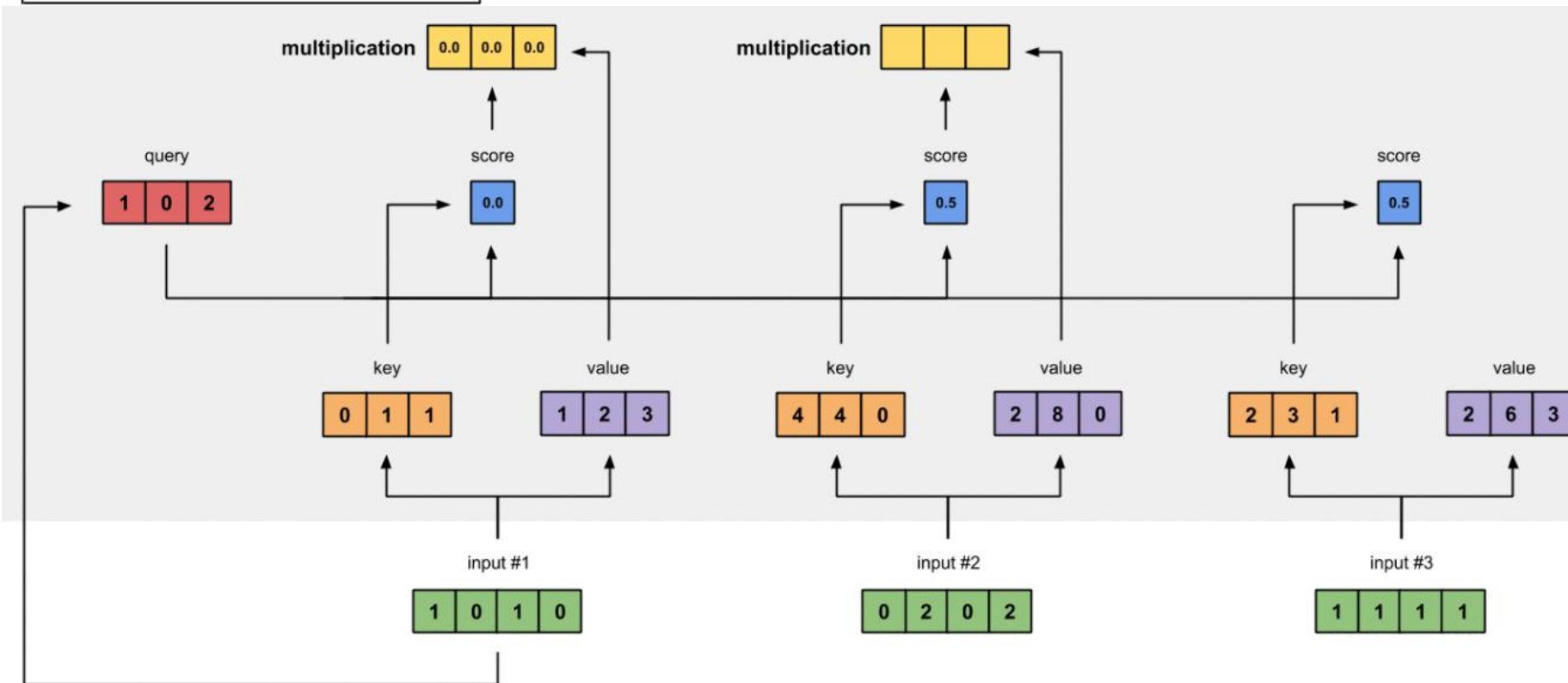
+=Positional Embedded

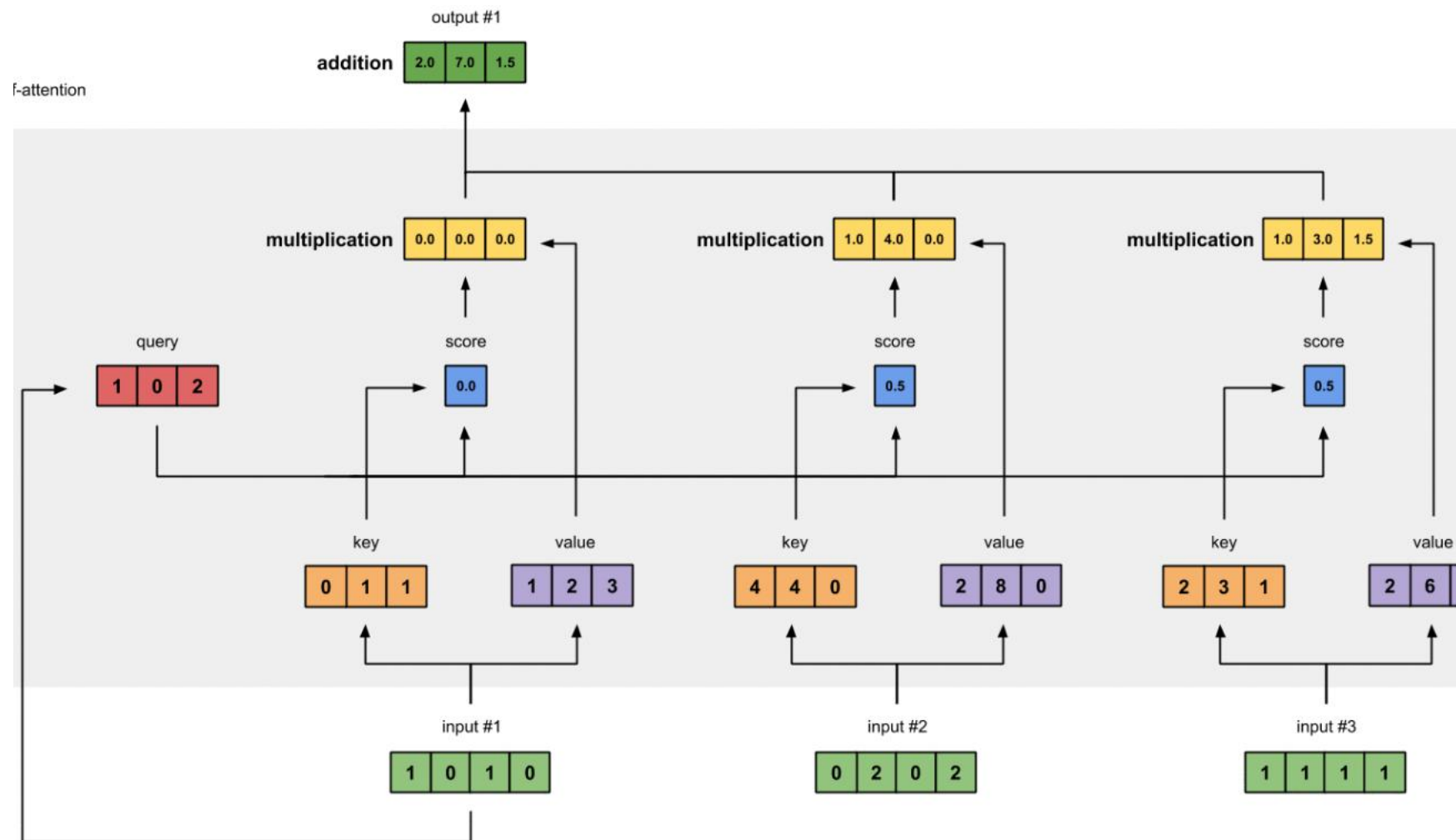
batch1	1	2	3	4	0
	...	...	...	...	...
batch2	1	2	3	5	0
	...	...	...	...	...



$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

=



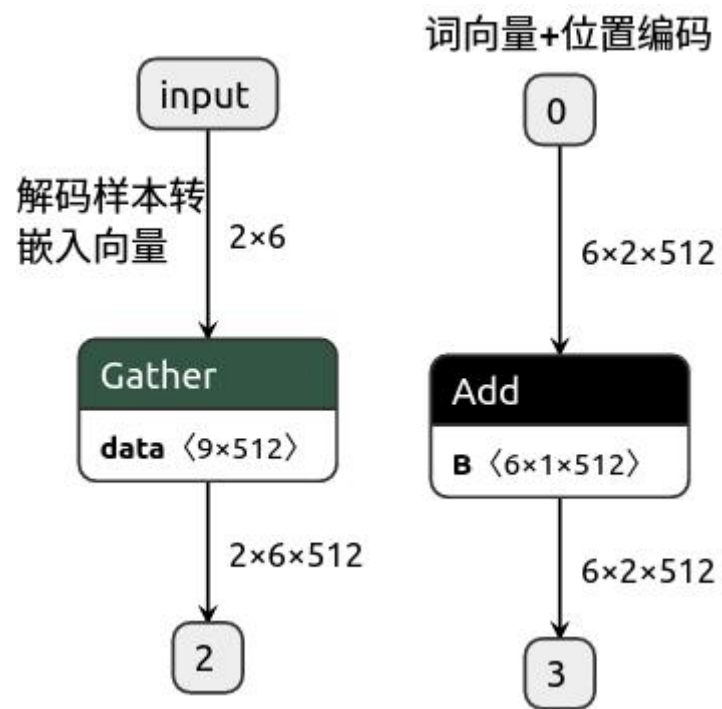


- 其它的输入向量也执行相同的操作，得到通过 self-attention 后的所有输出
- 输出结果：Value向量经过attn加权（multiplication） & 自注意力(score集合--互相之间的注意力)

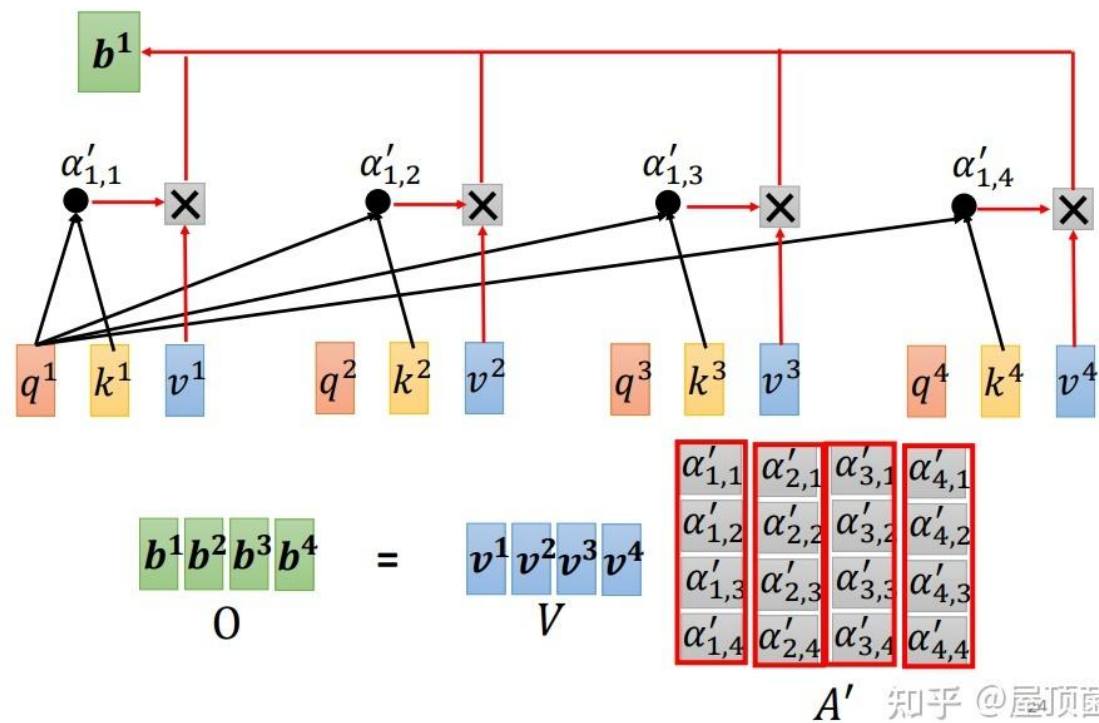
enc_self_attns
enc_outputs

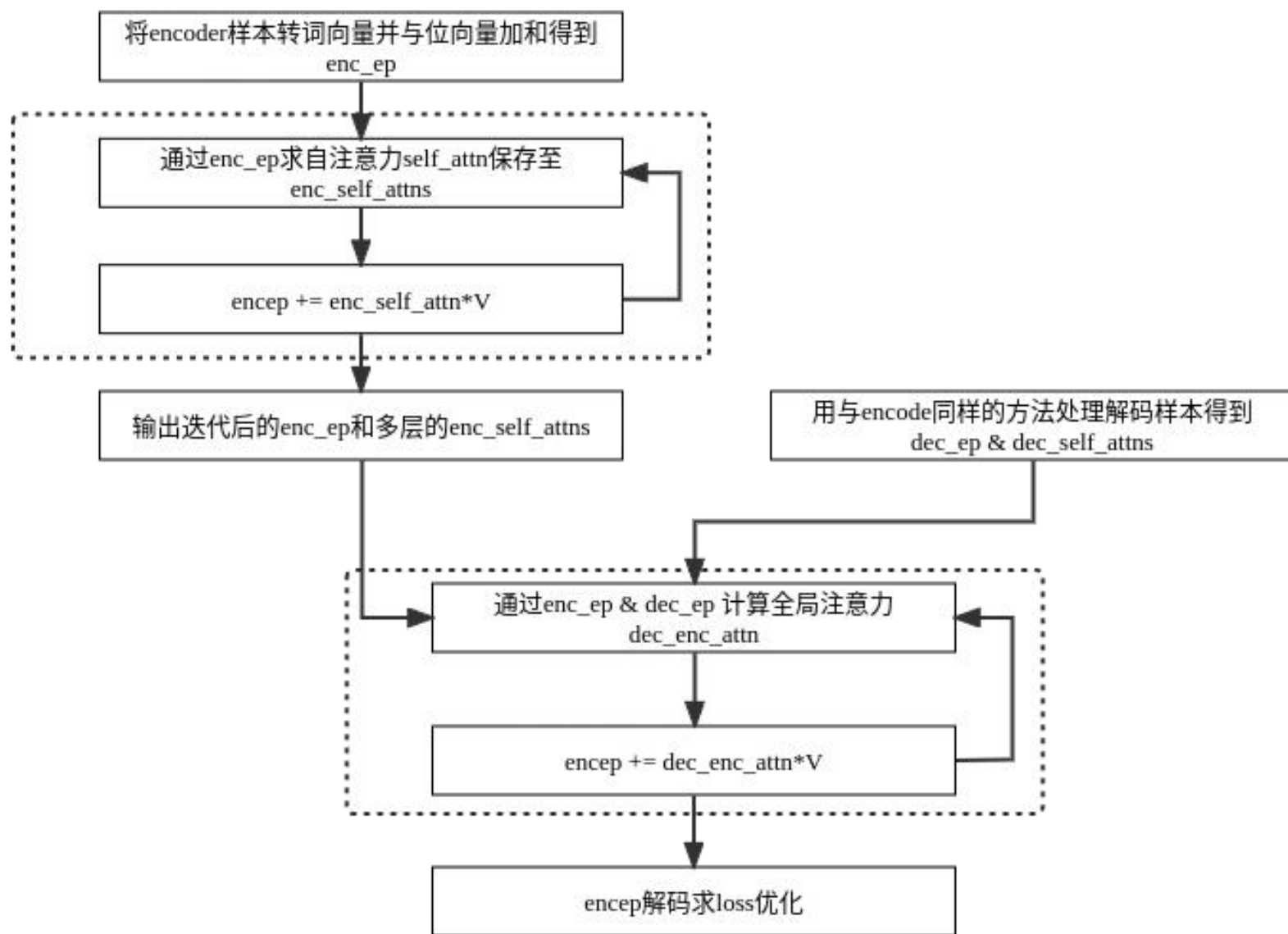
list 6 * tensor(2,8,5,5)
tensor(2,5,512)

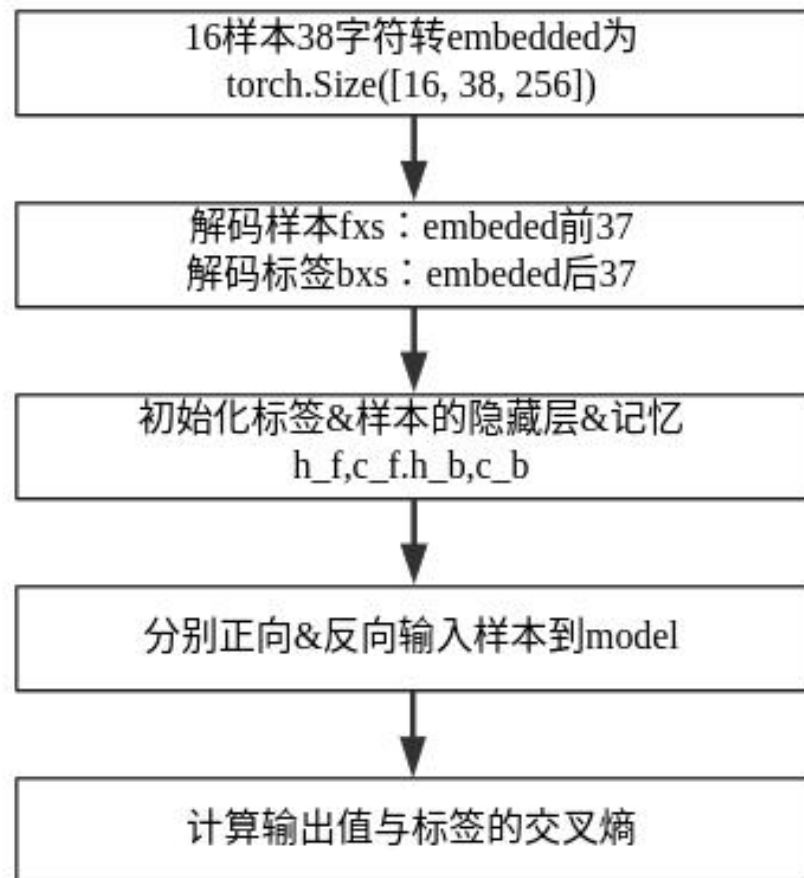
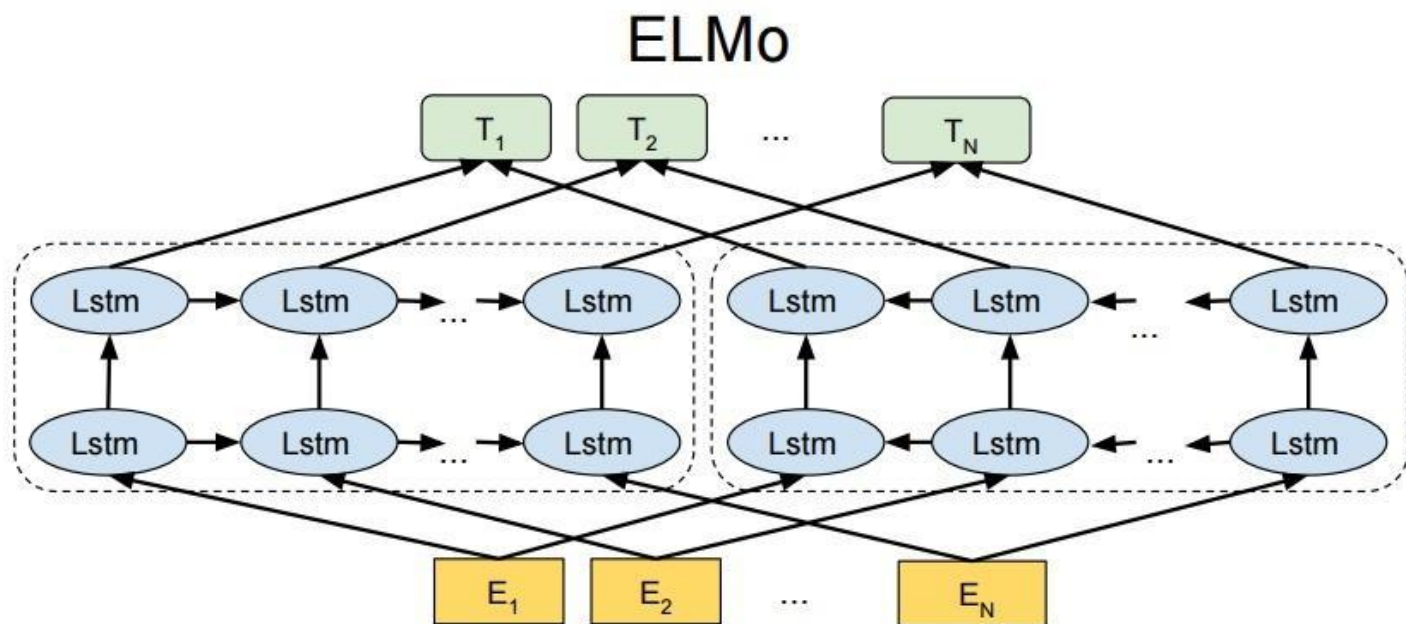
分层(6)注意力
6层注意力加权结果



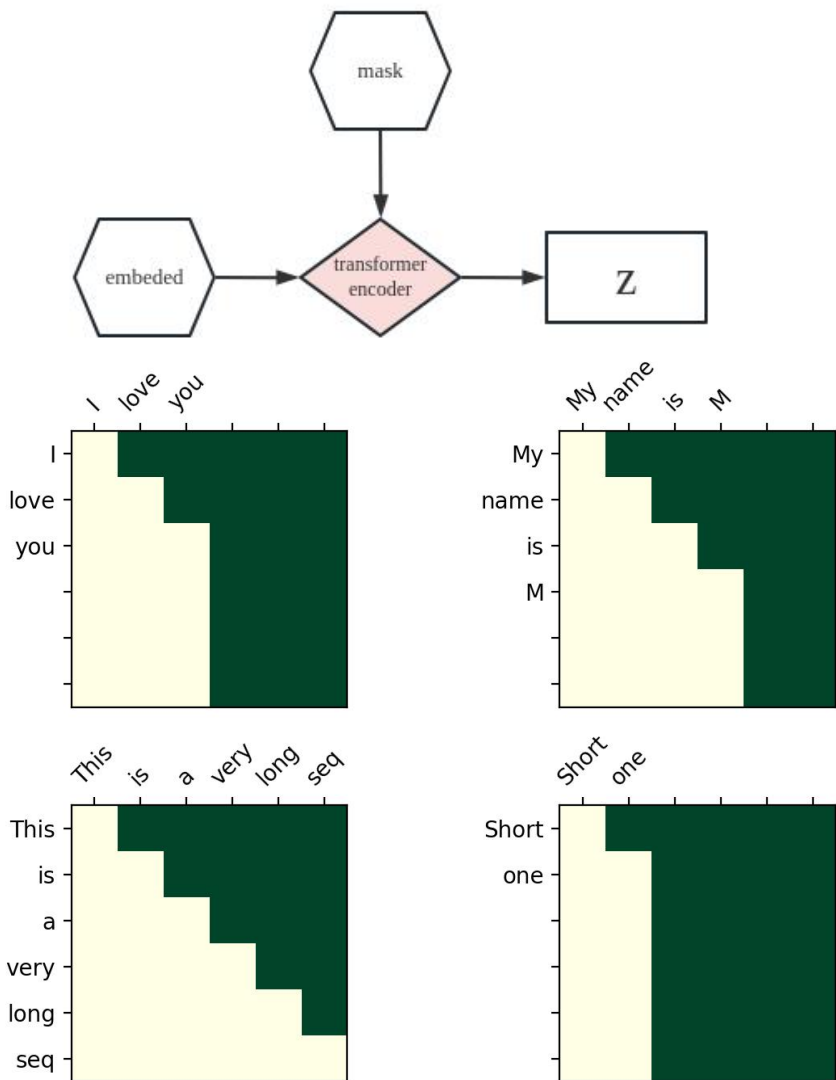
Self-attention



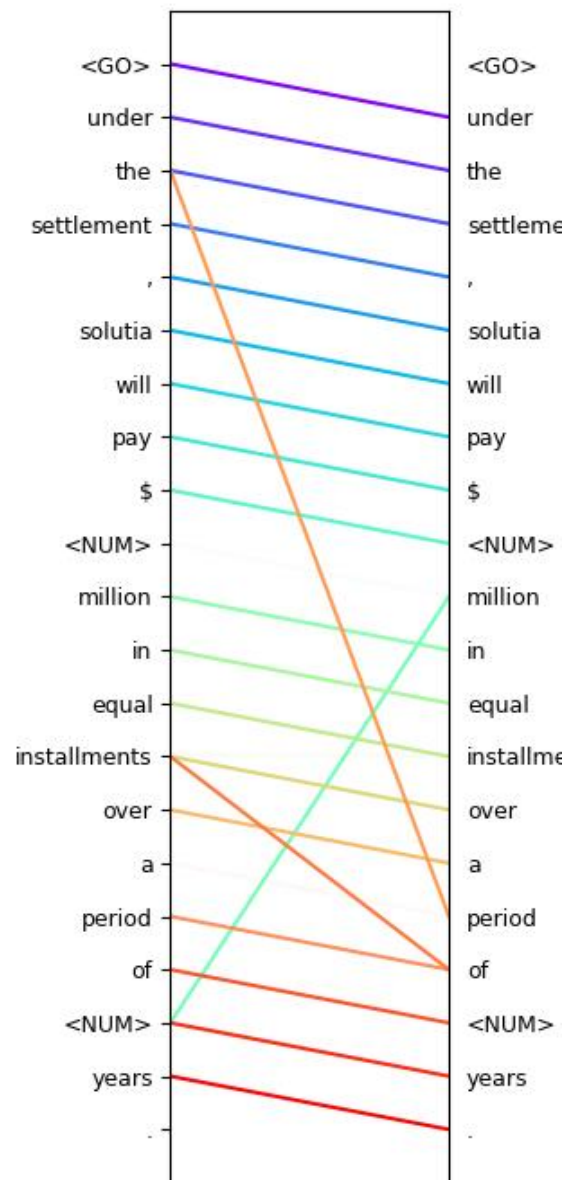
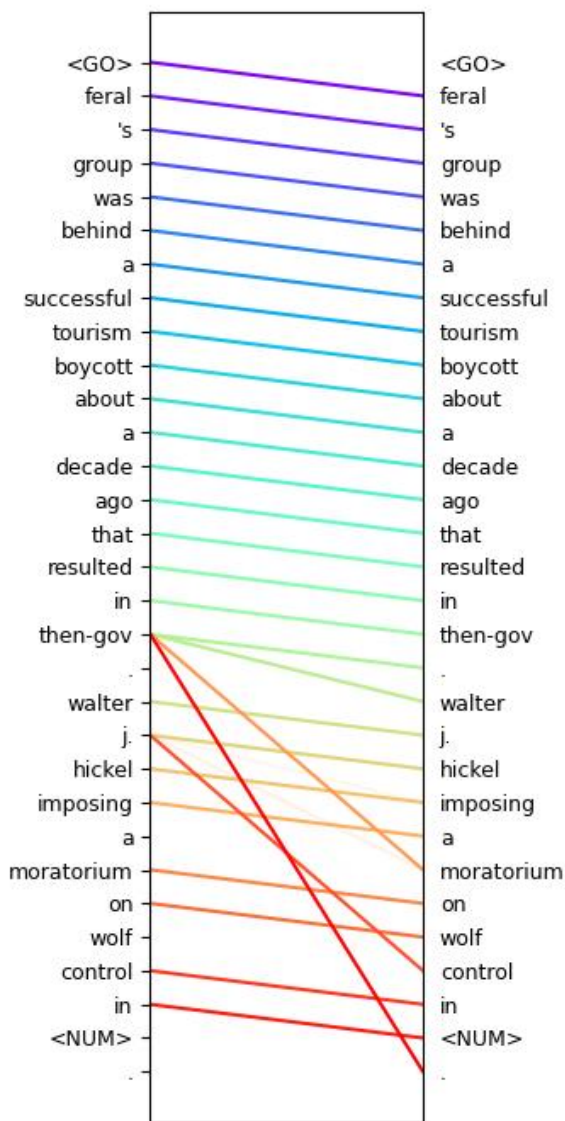




2023.04.23



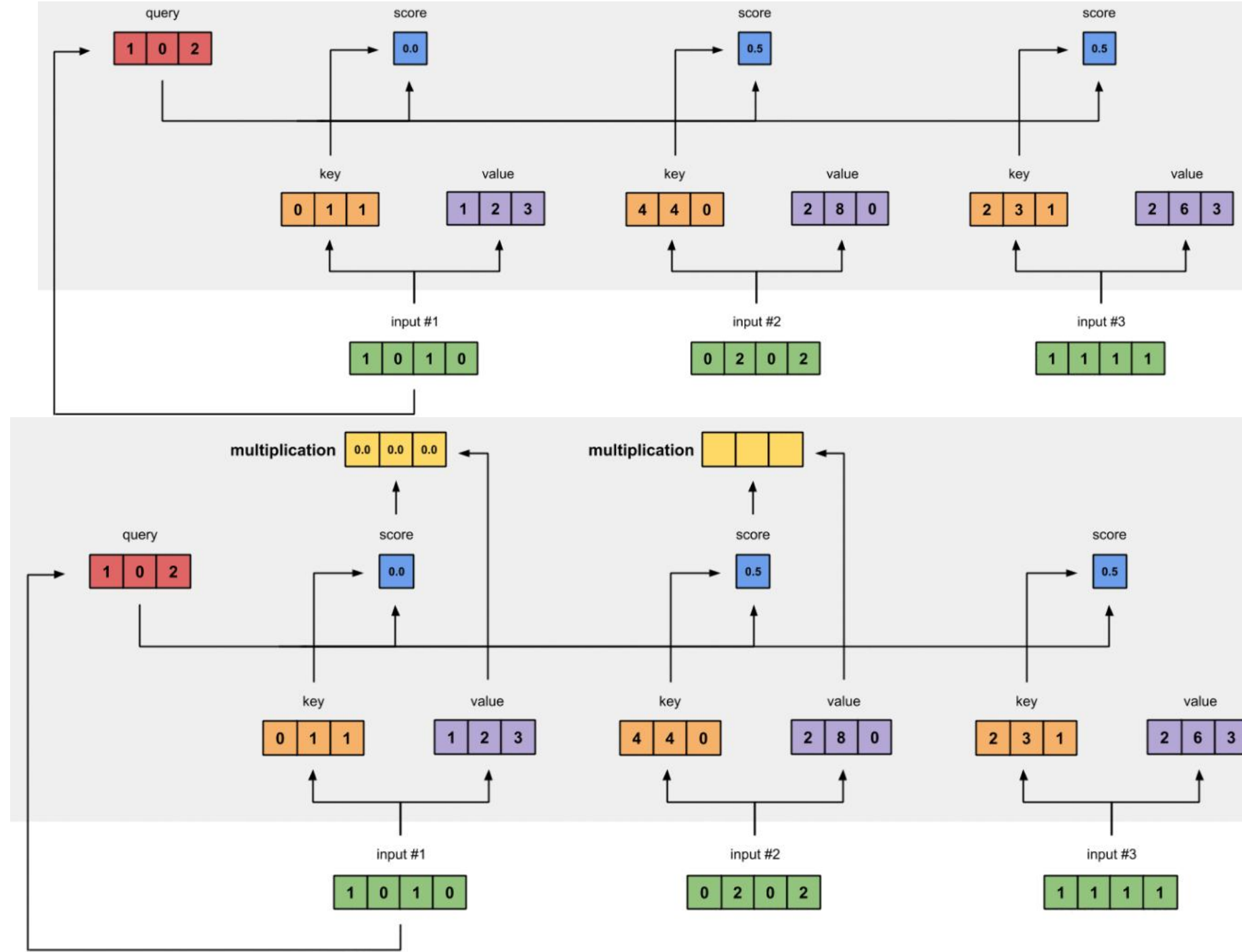
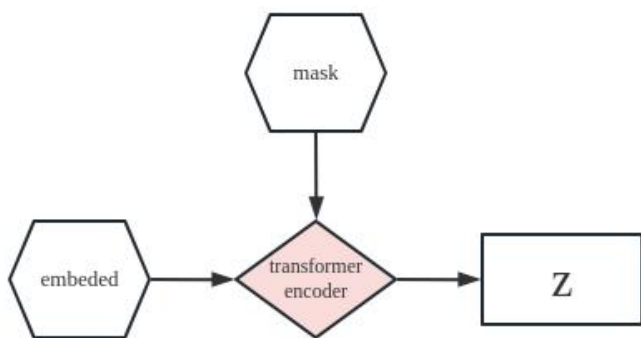
- 用前文的信息预测后文的信息，所以用上了Future Mask。mask将后文遮挡



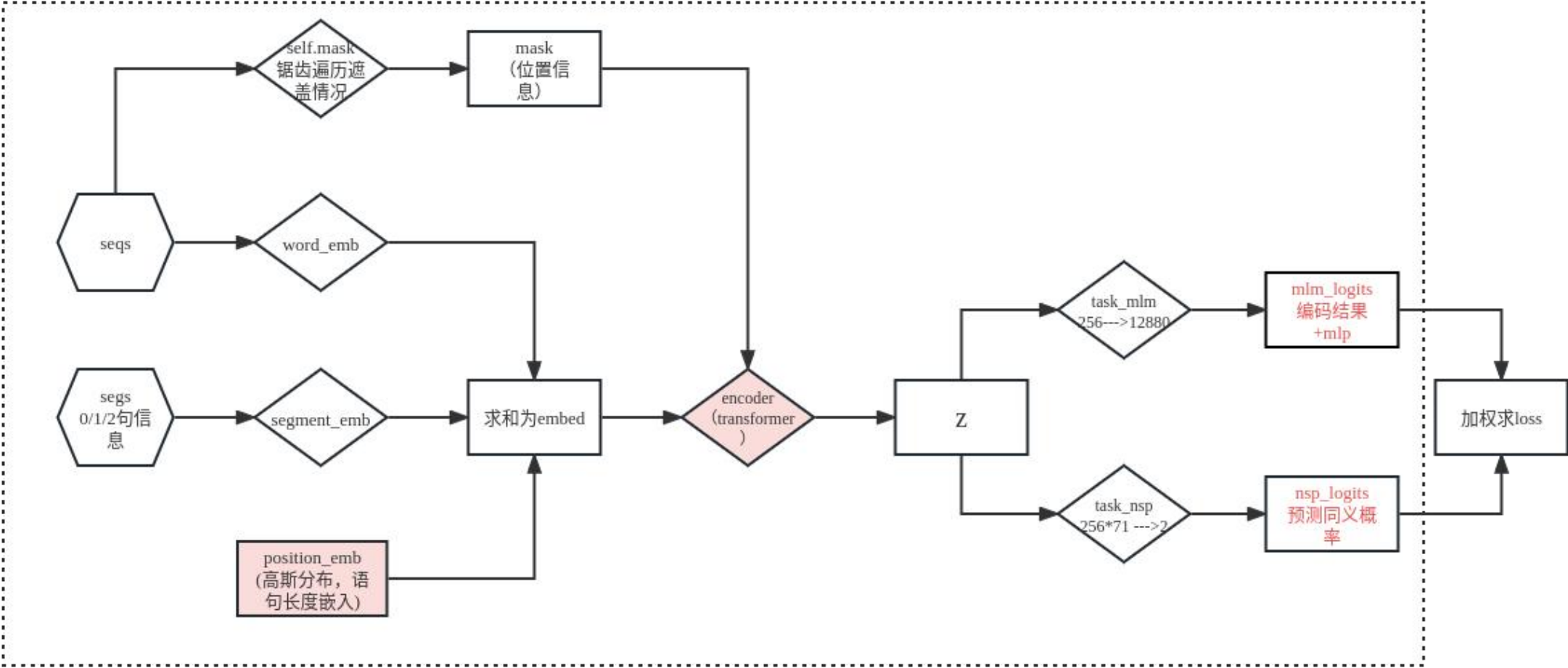
encoder

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) = Z$$

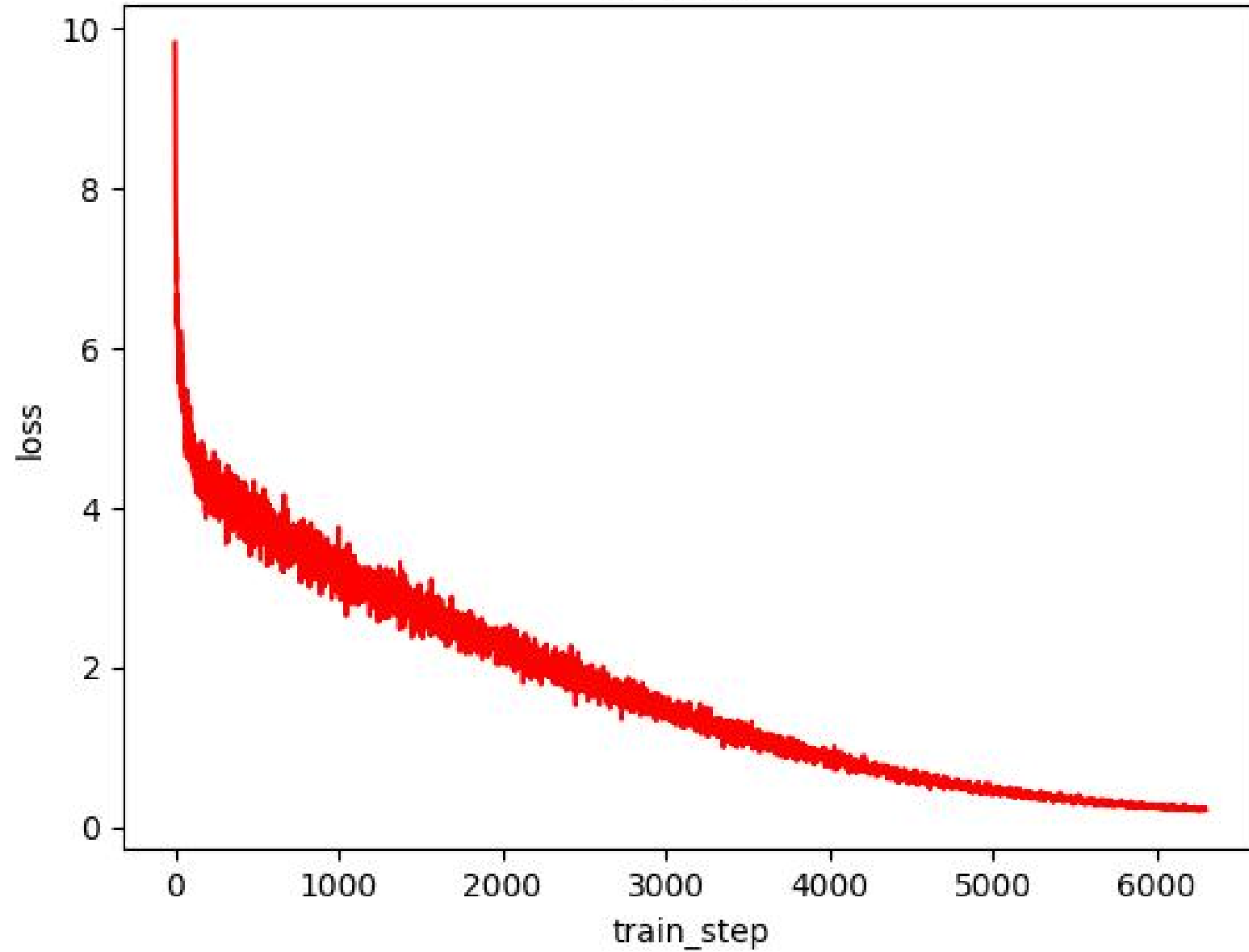
Diagram illustrating the matrix multiplication and softmax operation for the encoder. The input matrices are Q (purple) and K^T (orange), resulting in a score matrix (blue) which is then passed through a softmax function to produce the output matrix Z (pink).



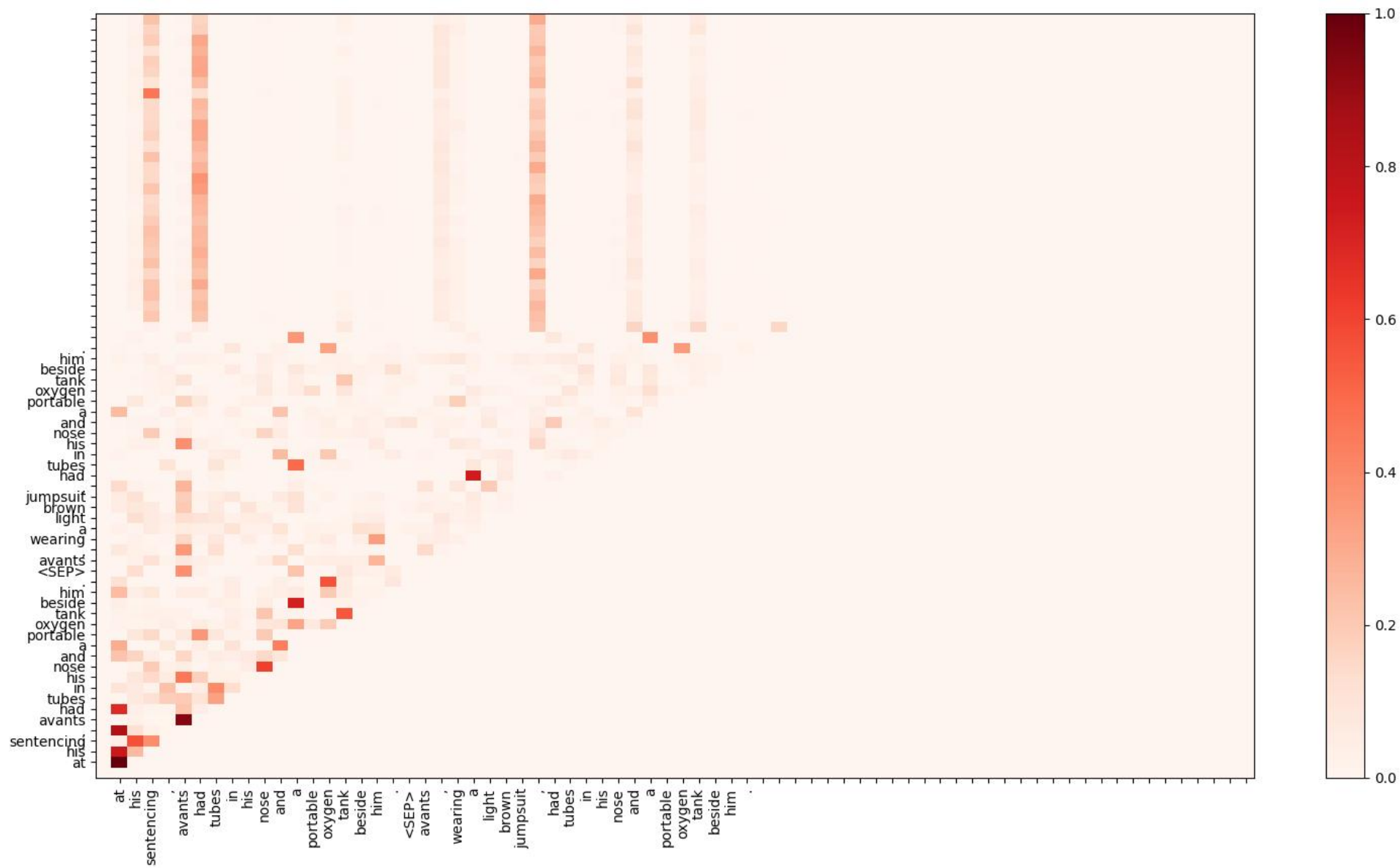
seqs	语句索引	
segs	前后句信息, 0前句, 1后句, 2空符	
xlen	两句话长度	
nsp_labels	标签 array([0]) 0or1	



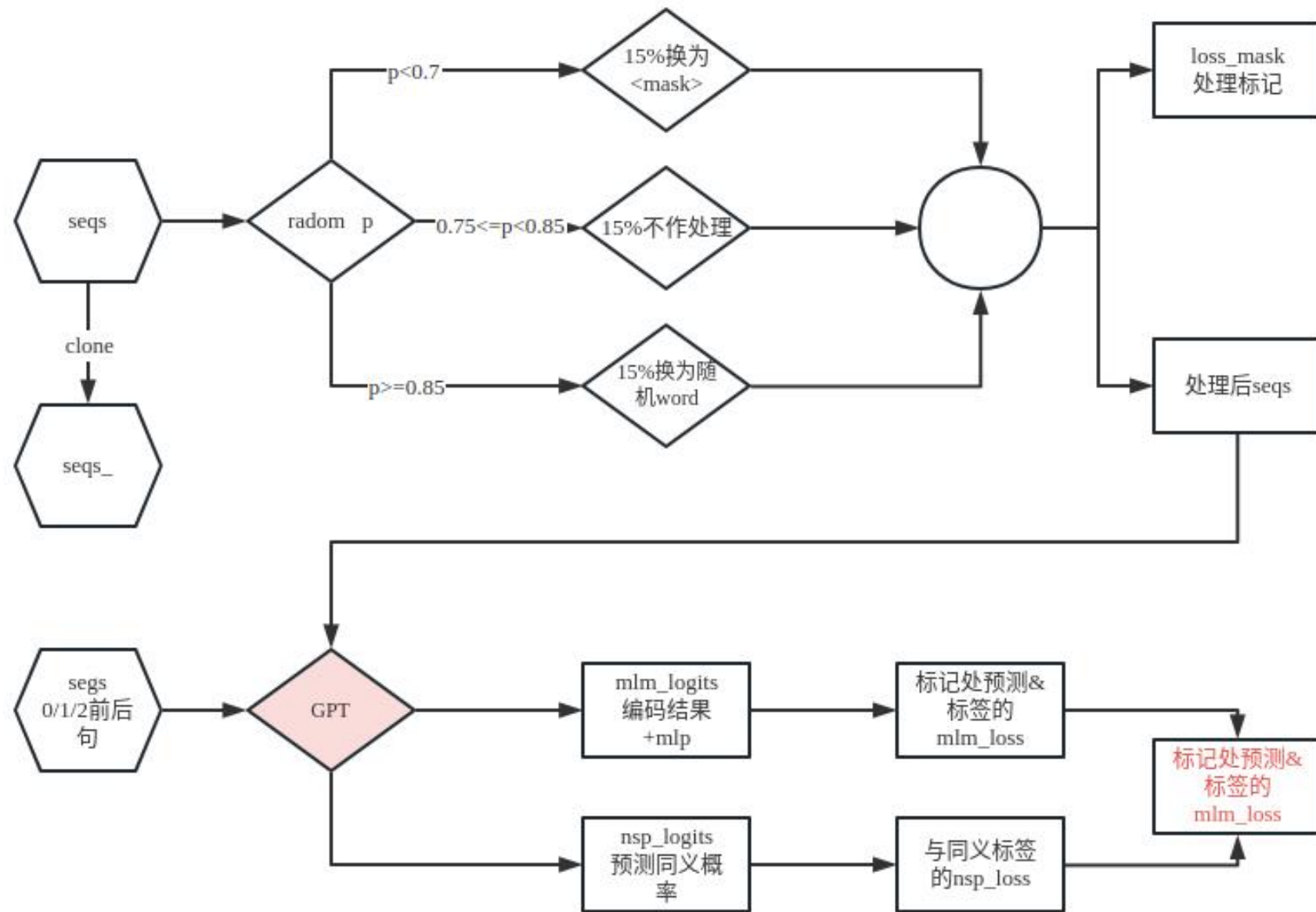
loss_line

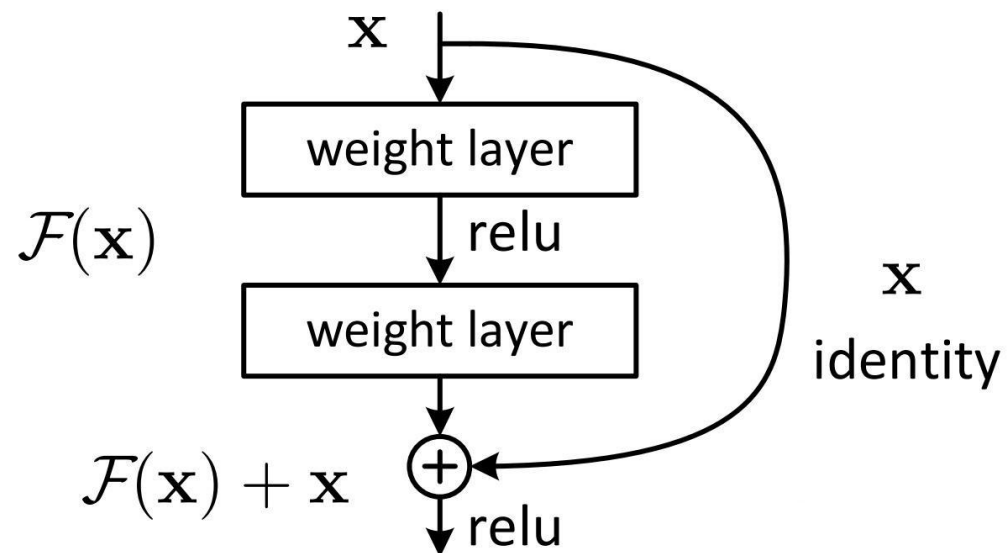






batch	list 1	tensor(32,72)	32样本72字符在词汇总表索引	seqs
	list 2	tensor(32,72)	前后句信息, 0前句, 1后句, 2空符	segs
	list 3	tensor(32,2)	一行两句话各自长度	xlen
	list 4	tensor(32,1)	标签 array([0]) 0or1	nsp_labels





- $F(x)$ 和 x 的 shape 不一致时
- pad 或者 乘以一个权重

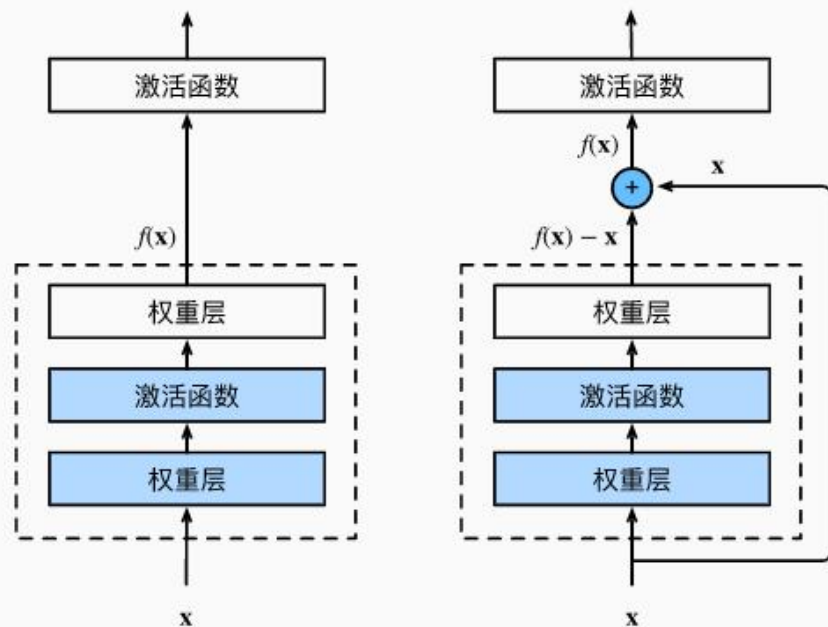
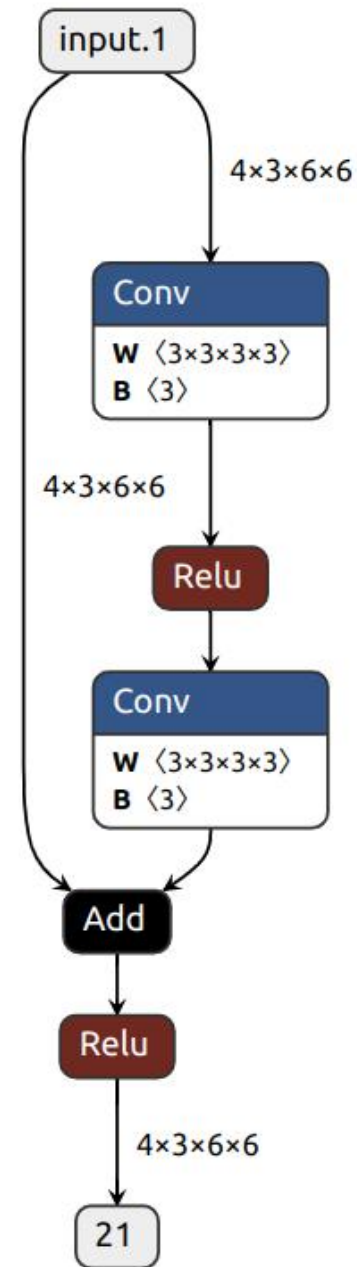
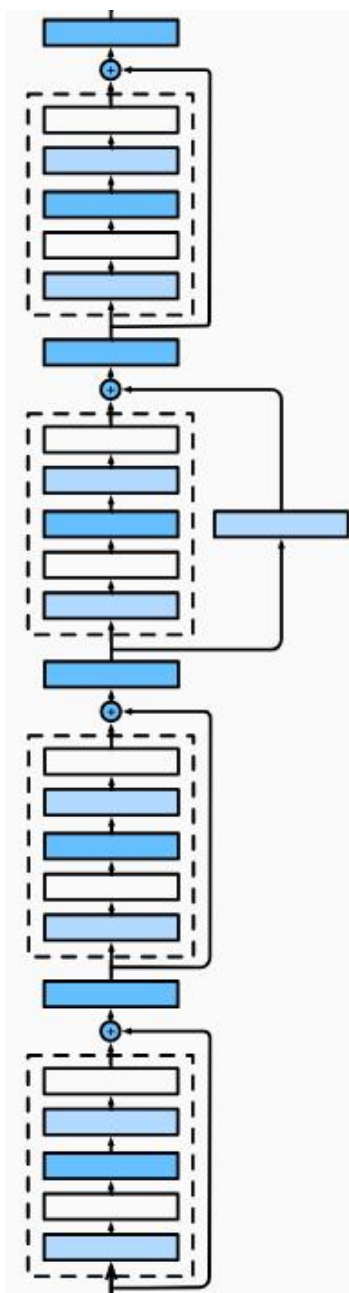


图7.6.2 一个正常块（左图）和一个残差块（右图）。





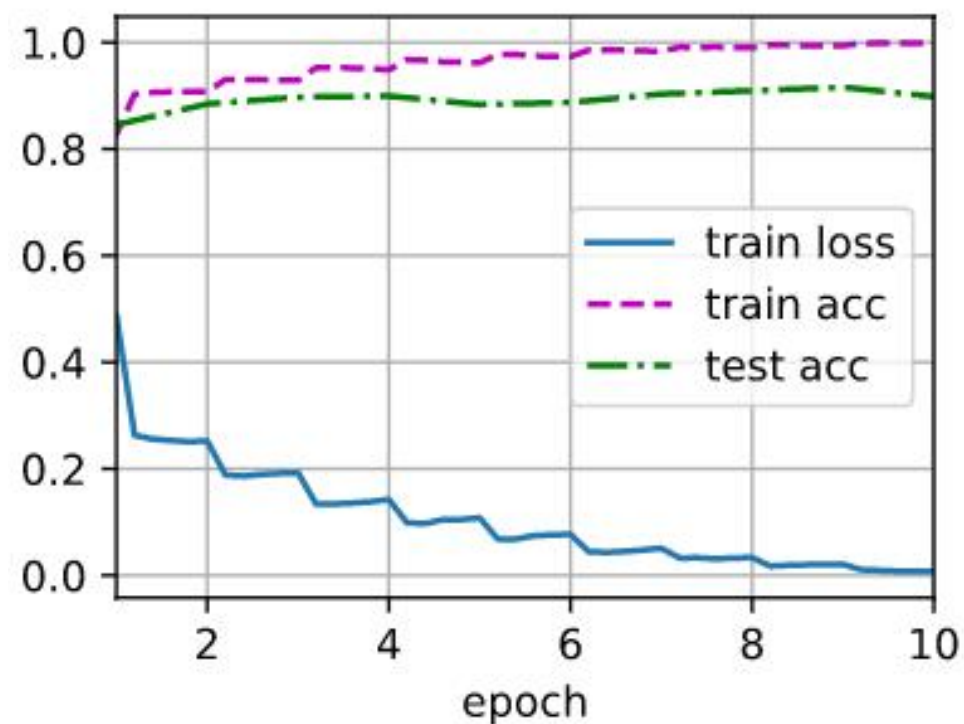
Sequential output shape: torch.Size([1, 64, 56, 56])
 Sequential output shape: torch.Size([1, 64, 56, 56])
 Sequential output shape: torch.Size([1, 128, 28, 28])
 Sequential output shape: torch.Size([1, 256, 14, 14])
 Sequential output shape: torch.Size([1, 512, 7, 7])
 AdaptiveAvgPool2d output shape: torch.Size([1, 512, 1, 1])
 Flatten output shape: torch.Size([1, 512])
 Linear output shape: torch.Size([1, 10])

loss 0.112, train acc 0.960, test acc 0.902
 1613.8 examples/sec on cuda:0

设置残差块训练 5 回合结果

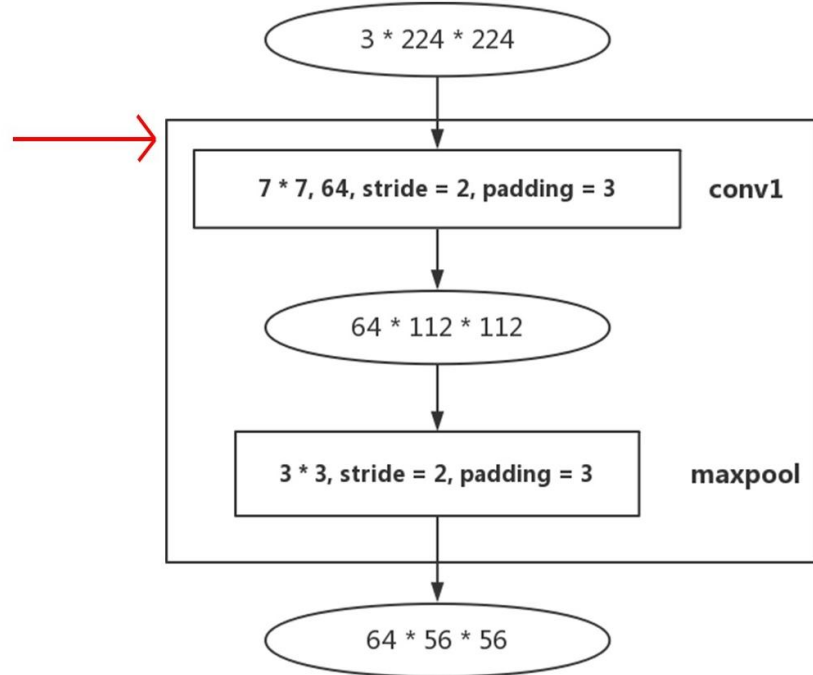
loss 0.144, train acc 0.948, test acc 0.861
 1751.9 examples/sec on cuda:0

未设置残差块训练 5 回合结果

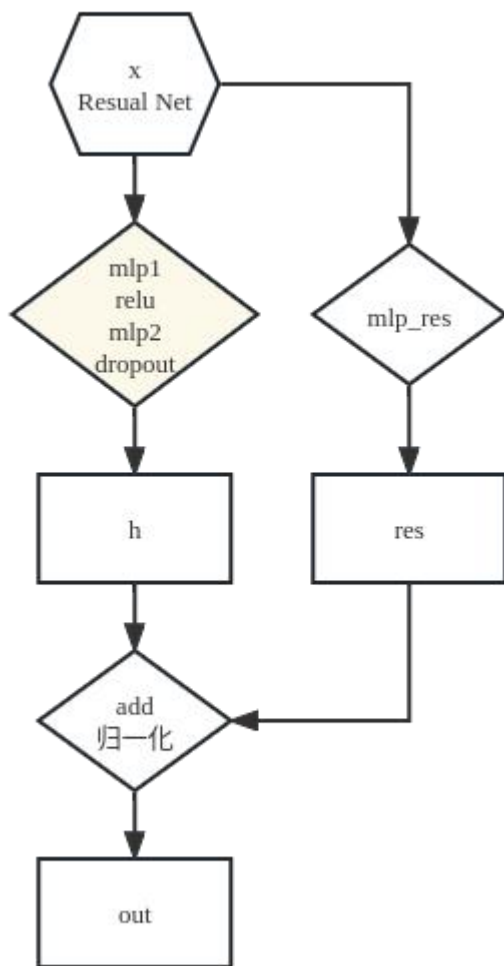


layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

layer name	output size	18-layer
conv1	112×112	7×7, 64, stride 2 3×3 max pool, stride 2
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$
	1×1	average pool, 1000-d fc, softmax

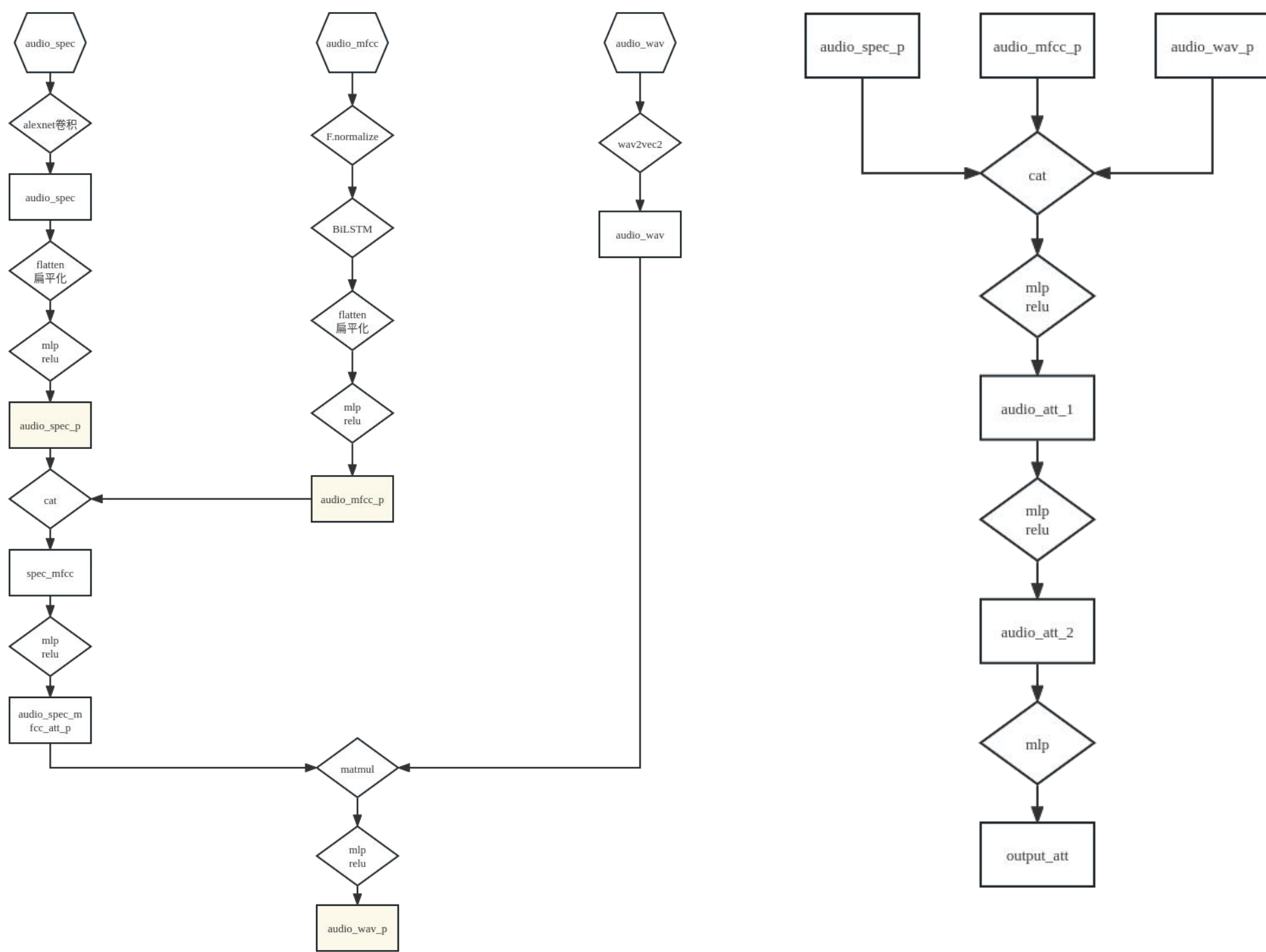


- 数字18 50 101 等分别代表的是卷积层+全连接层的层数
- 50-layer $(1+3*3+4*3+6*3+3*3+1) = 50$ 层



group name	output size	block type = $B(3,3)$
conv1	32×32	$[3 \times 3, 16]$
conv2	32×32	$\begin{bmatrix} 3 \times 3, 16 \times k \\ 3 \times 3, 16 \times k \end{bmatrix} \times N$
conv3	16×16	$\begin{bmatrix} 3 \times 3, 32 \times k \\ 3 \times 3, 32 \times k \end{bmatrix} \times N$
conv4	8×8	$\begin{bmatrix} 3 \times 3, 64 \times k \\ 3 \times 3, 64 \times k \end{bmatrix} \times N$
avg-pool	1×1	$[8 \times 8]$

- WideResNet (WRN) 希望使用一种较浅的，并在每个单层上更宽的（维度）模型来提升模型性能。
- 对于卷积层来说，宽度是指输出维度（通道）



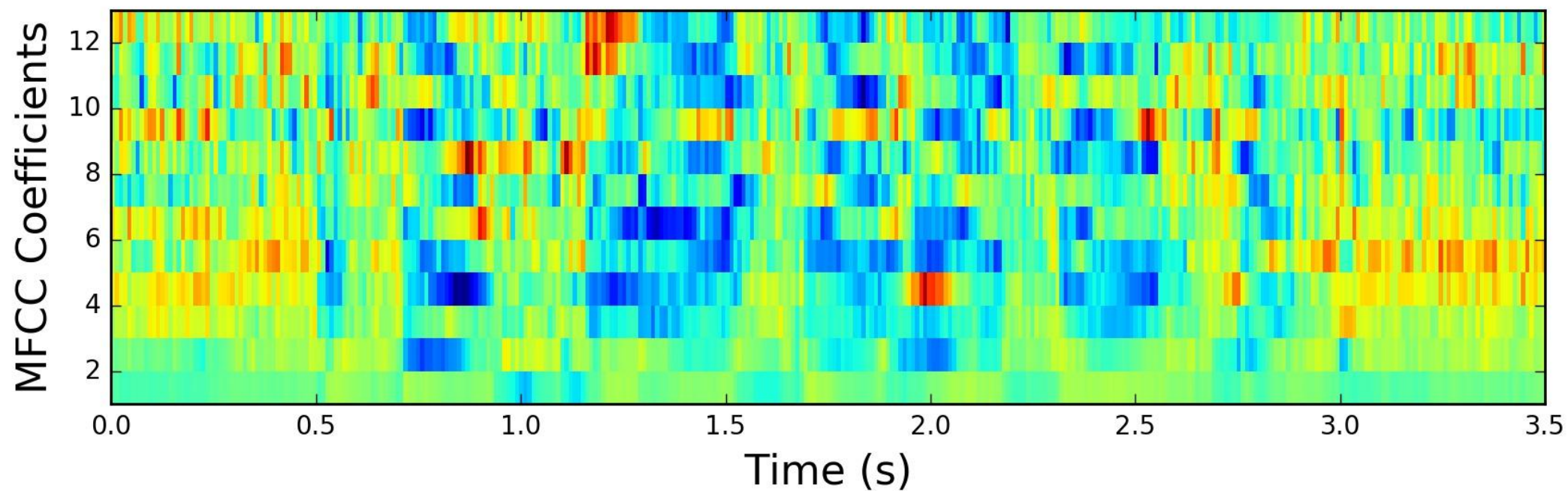
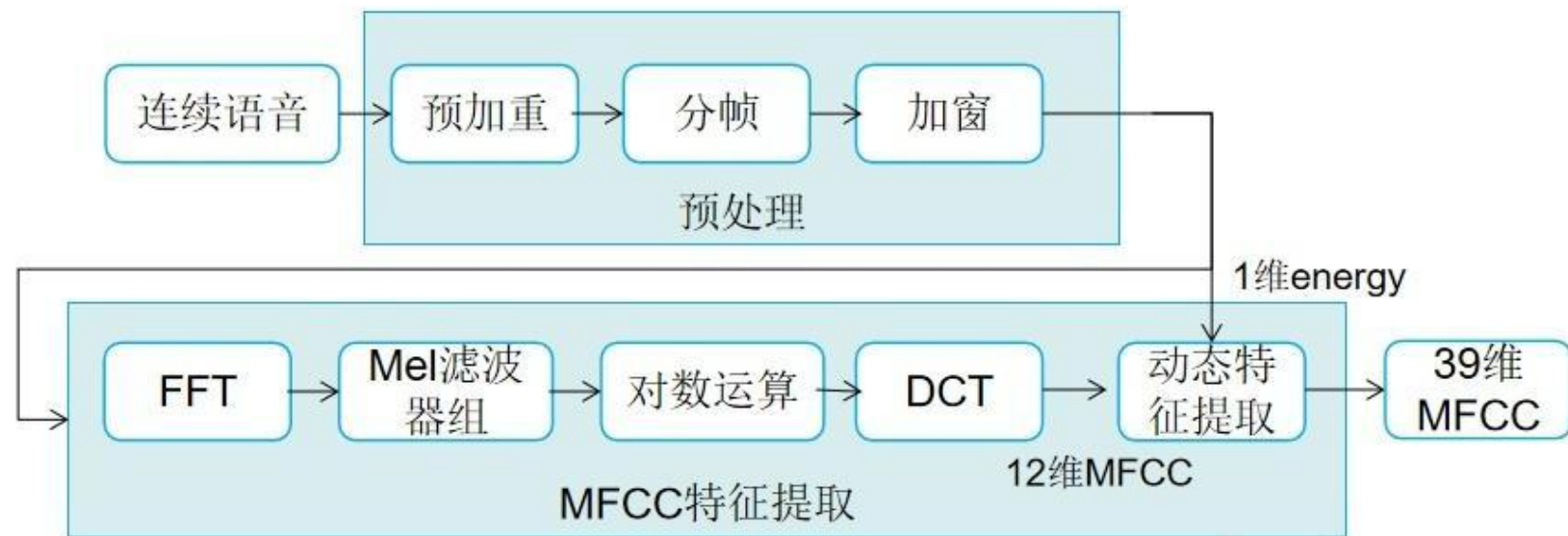
- Weather 收集自2010-2013四年间美国1600个地点的每小时气候数据。包括预测目标湿球温度以及11个气候指标。数据集/验证集/测试集分别为28/10/10个月。
- ETT(Electricity Transformer Temperature) 收集自中国两个县的电力变压器温度数据集，有2小时、1小时、15分钟三种粒度，每个数据点包括油温以及6个电力负载指标。数据集/验证集/测试集分别为12/4/4个月。
- Traffic2是加利福尼亚州交通部每小时数据的集合，描述了旧金山湾地区高速公路上不同传感器测量的道路占用率。

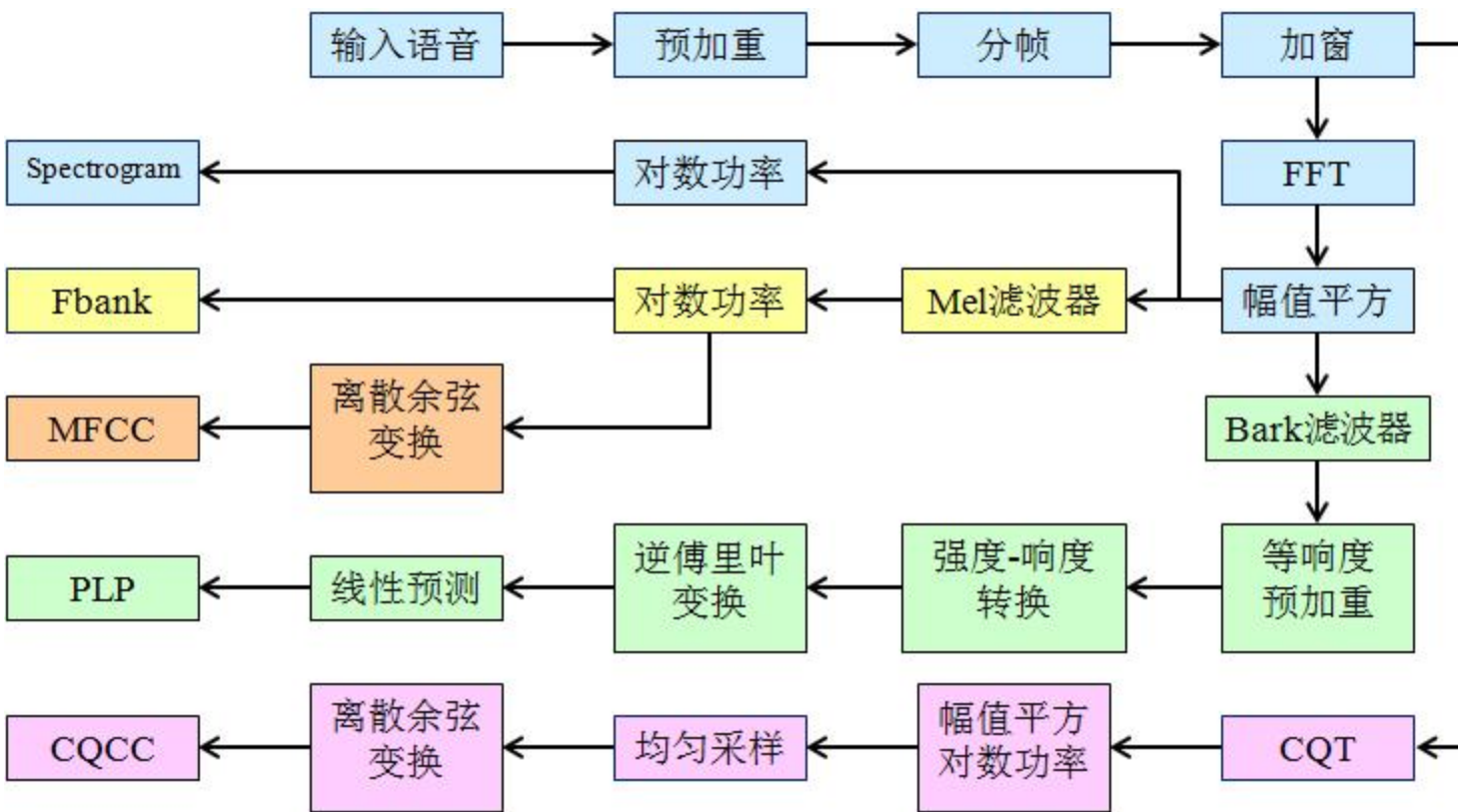
MSE (均方误差)
MAE(平均绝对误差)

Models		TiDE		PatchTST/64		DLinear		FEDformer		Autoformer		Informer		Pyraformer		LogTrans	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	0.166	0.222	0.149	0.198	0.176	0.237	0.238	0.314	0.249	0.329	0.354	0.405	0.896	0.556	0.458	0.490
	192	0.209	0.263	0.194	0.241	0.220	0.282	0.275	0.329	0.325	0.370	0.419	0.434	0.622	0.624	0.658	0.589
	336	0.254	0.301	0.245	0.282	0.265	0.319	0.339	0.377	0.351	0.391	0.583	0.543	0.739	0.753	0.797	0.652
	720	0.313	0.340	0.314	0.334	0.323	0.362	0.389	0.409	0.415	0.426	0.916	0.705	1.004	0.934	0.869	0.675
Traffic	96	0.336	0.253	0.360	0.249	0.410	0.282	0.576	0.359	0.597	0.371	0.733	0.410	2.085	0.468	0.684	0.384
	192	0.346	0.257	0.379	0.256	0.423	0.287	0.610	0.380	0.607	0.382	0.777	0.435	0.867	0.467	0.685	0.390
	336	0.355	0.260	0.392	0.264	0.436	0.296	0.608	0.375	0.623	0.387	0.776	0.434	0.869	0.469	0.734	0.408
	720	0.386	0.273	0.432	0.286	0.466	0.315	0.621	0.375	0.639	0.395	0.827	0.466	0.881	0.473	0.717	0.396
Electricity	96	0.132	0.229	0.129	0.222	0.140	0.237	0.186	0.302	0.196	0.313	0.304	0.393	0.386	0.449	0.258	0.357
	192	0.147	0.243	0.147	0.240	0.153	0.249	0.197	0.311	0.211	0.324	0.327	0.417	0.386	0.443	0.266	0.368
	336	0.161	0.261	0.163	0.259	0.169	0.267	0.213	0.328	0.214	0.327	0.333	0.422	0.378	0.443	0.280	0.380
	720	0.196	0.294	0.197	0.290	0.203	0.301	0.233	0.344	0.236	0.342	0.351	0.427	0.376	0.445	0.283	0.376
ETTh1	96	0.375	0.398	0.379	0.401	0.375	0.399	0.376	0.415	0.435	0.446	0.941	0.769	0.664	0.612	0.878	0.740
	192	0.412	0.422	0.413	0.429	0.412	0.420	0.423	0.446	0.456	0.457	1.007	0.786	0.790	0.681	1.037	0.824
	336	0.435	0.433	0.435	0.436	0.439	0.443	0.444	0.462	0.486	0.487	1.038	0.784	0.891	0.738	1.238	0.932
	720	0.454	0.465	0.446	0.464	0.472	0.490	0.469	0.492	0.515	0.517	1.144	0.857	0.963	0.782	1.135	0.852
ETTh2	96	0.270	0.336	0.274	0.337	0.289	0.353	0.332	0.374	0.332	0.368	1.549	0.952	0.645	0.597	2.116	1.197
	192	0.332	0.380	0.338	0.376	0.383	0.418	0.407	0.446	0.426	0.434	3.792	1.542	0.788	0.683	4.315	1.635
	336	0.360	0.407	0.363	0.397	0.448	0.465	0.400	0.447	0.477	0.479	4.215	1.642	0.907	0.747	1.124	1.604
	720	0.419	0.451	0.393	0.430	0.605	0.551	0.412	0.469	0.453	0.490	3.656	1.619	0.963	0.783	3.188	1.540
ETThm1	96	0.306	0.349	0.293	0.346	0.299	0.343	0.326	0.390	0.510	0.492	0.626	0.560	0.543	0.510	0.600	0.546
	192	0.335	0.366	0.333	0.370	0.335	0.365	0.365	0.415	0.514	0.495	0.725	0.619	0.557	0.537	0.837	0.700
	336	0.364	0.384	0.369	0.392	0.369	0.386	0.392	0.425	0.510	0.492	1.005	0.741	0.754	0.655	1.124	0.832
	720	0.413	0.413	0.416	0.420	0.425	0.421	0.446	0.458	0.527	0.493	1.133	0.845	0.908	0.724	1.153	0.820
ETThm2	96	0.161	0.251	0.166	0.256	0.167	0.260	0.180	0.271	0.205	0.293	0.355	0.462	0.435	0.507	0.768	0.642
	192	0.215	0.289	0.223	0.296	0.224	0.303	0.252	0.318	0.278	0.336	0.595	0.586	0.730	0.673	0.989	0.757
	336	0.267	0.326	0.274	0.329	0.281	0.342	0.324	0.364	0.343	0.379	1.270	0.871	1.201	0.845	1.334	0.872
	720	0.352	0.383	0.362	0.385	0.397	0.421	0.410	0.420	0.414	0.419	3.001	1.267	3.625	1.451	3.048	1.328

Models		Autoformer		Informer[48]		LogTrans[26]		Reformer[23]		LSTNet[25]		LSTM[17]		TCN[4]	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETT*	96	0.255	0.339	0.365	0.453	0.768	0.642	0.658	0.619	3.142	1.365	2.041	1.073	3.041	1.330
	192	0.281	0.340	0.533	0.563	0.989	0.757	1.078	0.827	3.154	1.369	2.249	1.112	3.072	1.339
	336	0.339	0.372	1.363	0.887	1.334	0.872	1.549	0.972	3.160	1.369	2.568	1.238	3.105	1.348
	720	0.422	0.419	3.379	1.388	3.048	1.328	2.631	1.242	3.171	1.368	2.720	1.287	3.135	1.354
Electricity	96	0.201	0.317	0.274	0.368	0.258	0.357	0.312	0.402	0.680	0.645	0.375	0.437	0.985	0.813
	192	0.222	0.334	0.296	0.386	0.266	0.368	0.348	0.433	0.725	0.676	0.442	0.473	0.996	0.821
	336	0.231	0.338	0.300	0.394	0.280	0.380	0.350	0.433	0.828	0.727	0.439	0.473	1.000	0.824
	720	0.254	0.361	0.373	0.439	0.283	0.376	0.340	0.420	0.957	0.811	0.980	0.814	1.438	0.784
Exchange	96	0.197	0.323	0.847	0.752	0.968	0.812	1.065	0.829	1.551	1.058	1.453	1.049	3.004	1.432
	192	0.300	0.369	1.204	0.895	1.040	0.851	1.188	0.906	1.477	1.028	1.846	1.179	3.048	1.444
	336	0.509	0.524	1.672	1.036	1.659	1.081	1.357	0.976	1.507	1.031	2.136	1.231	3.113	1.459
	720	1.447	0.941	2.478	1.310	1.941	1.127	1.510	1.016	2.285	1.243	2.984	1.427	3.150	1.458
Traffic	96	0.613	0.388	0.719	0.391	0.684	0.384	0.732	0.423	1.107	0.685	0.843	0.453	1.438	0.784
	192	0.616	0.382	0.696	0.379	0.685	0.390	0.733	0.420	1.157	0.706	0.847	0.453	1.463	0.794
	336	0.622	0.337	0.777	0.420	0.733	0.408	0.742	0.420	1.216	0.730	0.853	0.455	1.479	0.799
	720	0.660	0.408	0.864	0.472	0.717	0.396	0.755	0.423	1.481	0.805	1.500	0.805	1.499	0.804
Weather	96	0.266	0.336	0.300	0.384	0.458	0.490	0.689	0.596	0.594	0.587	0.369	0.406	0.615	0.589
	192	0.307	0.367	0.598	0.544	0.658	0.589	0.752	0.638	0.560	0.565	0.416	0.435	0.629	0.600
	336	0.359	0.395	0.578	0.523	0.797	0.652	0.639	0.596	0.597	0.587	0.455	0.454	0.639	0.608
	720	0.419	0.428	1.059	0.741	0.869	0.675	1.130	0.792	0.618	0.599	0.535	0.520	0.639	0.610
ILI	24	3.483	1.287	5.764	1.677	4.480	1.444	4.400	1.382	6.026	1.770	5.914	1.734	6.624	1.830
	36	3.103	1.148	4.755	1.467	4.799	1.467	4.783	1.448	5.340	1.668	6.631	1.845	6.858	1.879
	48	2.669	1.085	4.763	1.469	4.800	1.468	4.832	1.465	6.080	1.787	6.736	1.857	6.968	1.892
	60	2.770	1.125	5.264	1.564	5.278	1.560	4.882	1.483	5.548	1.720	6.870	1.879	7.127	1.918

* ETT means the ETTm2. See Appendix A for the **full benchmark** of ETTh1, ETTh2, ETTm1.



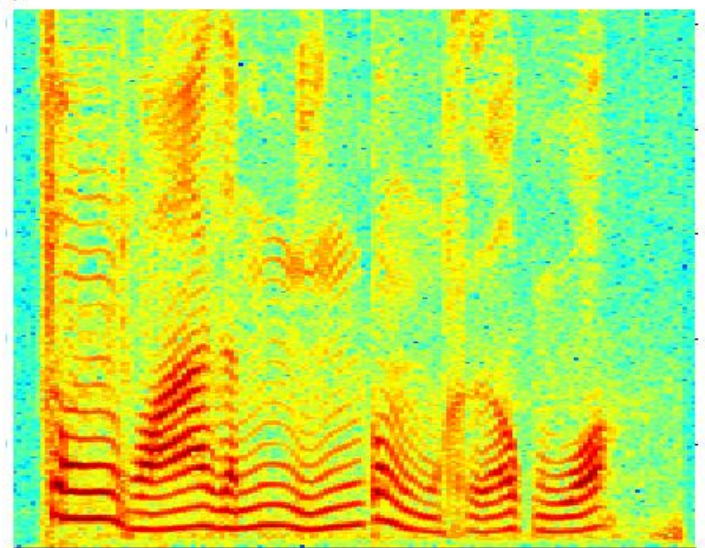


语谱图(spectrogram)

- 声谱图的横坐标是时间
- 纵坐标是频率
- 坐标点值为语音数据能量

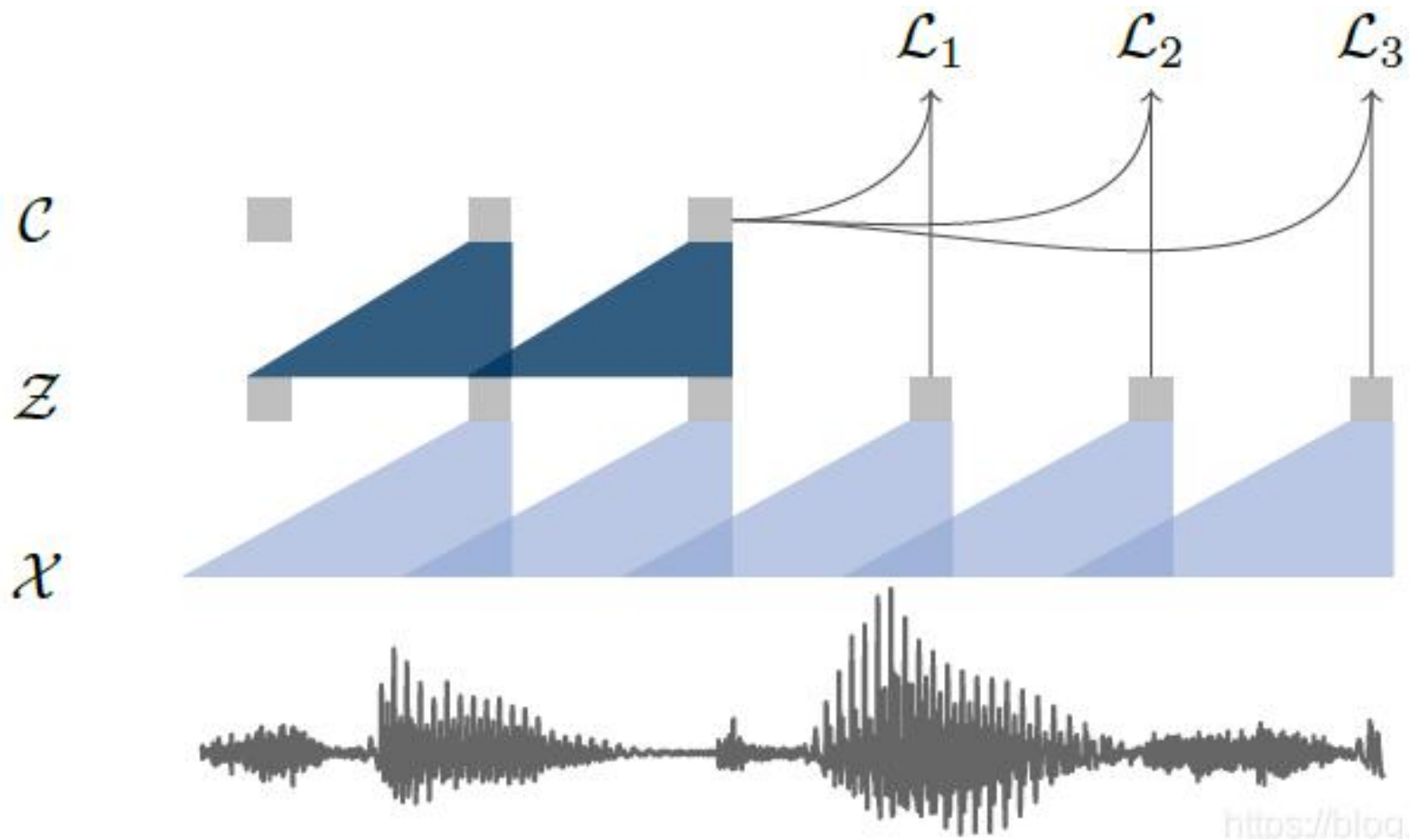
由于是采用二维平面表达三维信息，所以能量值的大小是通过颜色来表示的，颜色深，表示该点的语音能量越强。

频率



时间

将原始音频 x 编码为潜在空间 z 的 encoder network（5层卷积），和将 z 转换为contextualized representation的 context network（9层卷积），最终特征维度为512x帧数。目标是在特征层面使用当前帧预测未来帧。



MFCC

针对数据集1M进行训练&测试

github原代码

1e-05 0.3476 1.3293 61.94% 65.89% 60.07% 66.39%

MFCC+BiLSTM

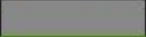
1e-05 1.0440 0.9828 60.50% 60.32% 60.62% 62.47%

		训练集loss	测试集loss		测试集最优		训练集最优
github原代码	1e-05	0.3476	1.3293	61.94%	65.89%	60.07%	66.39%
MFCC+BiLSTM	1e-05	1.0440	0.9828	60.50%	60.32%	60.62%	62.47%
MFCC+DLinear	1e-05	1.0739	1.0352	55.83%	56.91%	57.79%	58.28%
MFCC+TiDE	1e-05	1.1036	1.1508	45.27%	46.21%	49.40%	49.74%
Spec+Alexnet	1e-05	0.5312	1.0506	63.55%	64.81%	62.15%	66.82%
Spec+BiLstm	1e-05	0.2206	1.6786	55.48%	61.40%	56.50%	62.50%
wav2vec	1e-05	0.2117	1.7233	70.74%	71.99%	73.74%	74.20%

```
(lstm_spec): LSTM(384, 512, num_layers=2, batch_first=True, dropout=0.5, bidirectional=True)
(spec_linear): Linear(in_features=786432, out_features=128, bias=True)
(spec_linear_out): Linear(in_features=128, out_features=4, bias=True)
)
Number of trainable parameters: 285281765
```

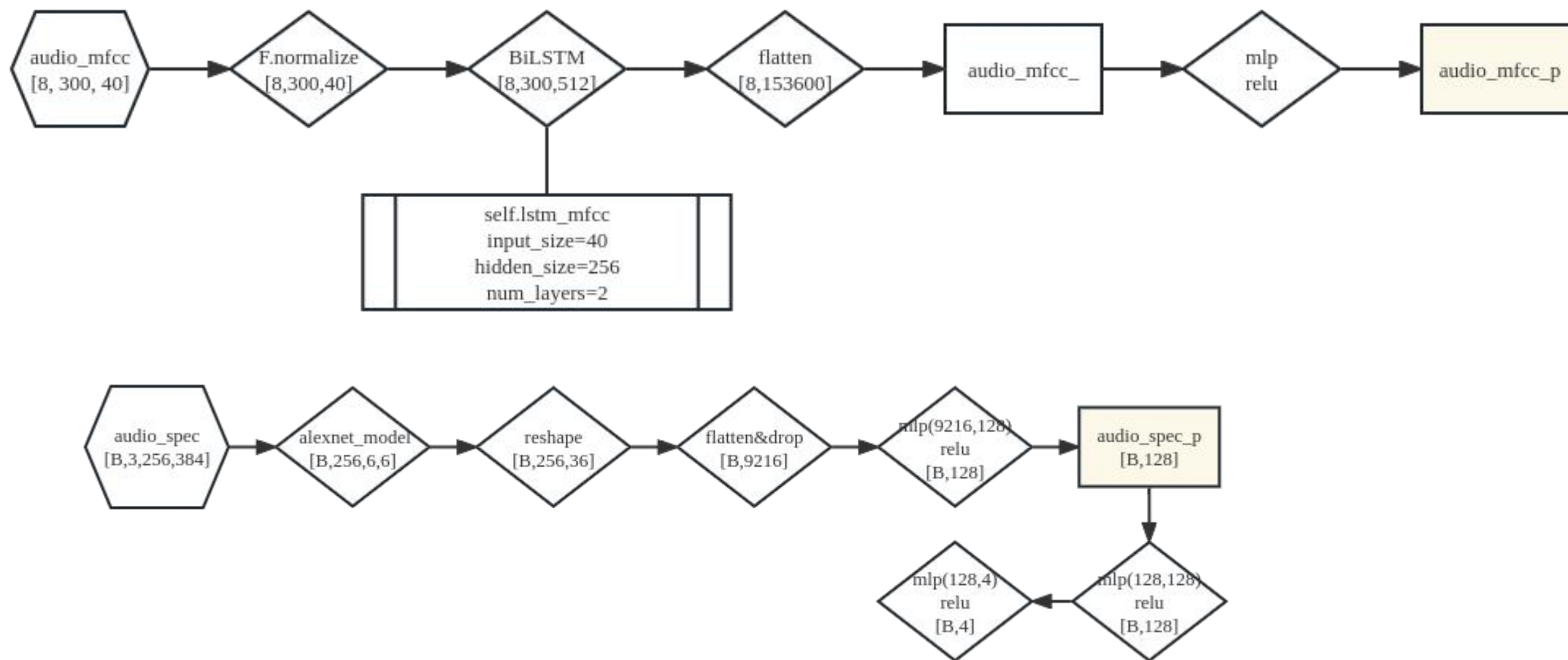
```
(spec_linear_out): Linear(in_features=128, out_features=4, bias=True)
(out_res): Linear(in_features=2048, out_features=4, bias=True)
)
Number of trainable parameters: 10332176
```

Start Training!!!

13% |  | 79/614 [00:16<01:47, 4.96it/s]

/home/windsky/Data/point_cloud/CA-github4_spec3_bilstm2

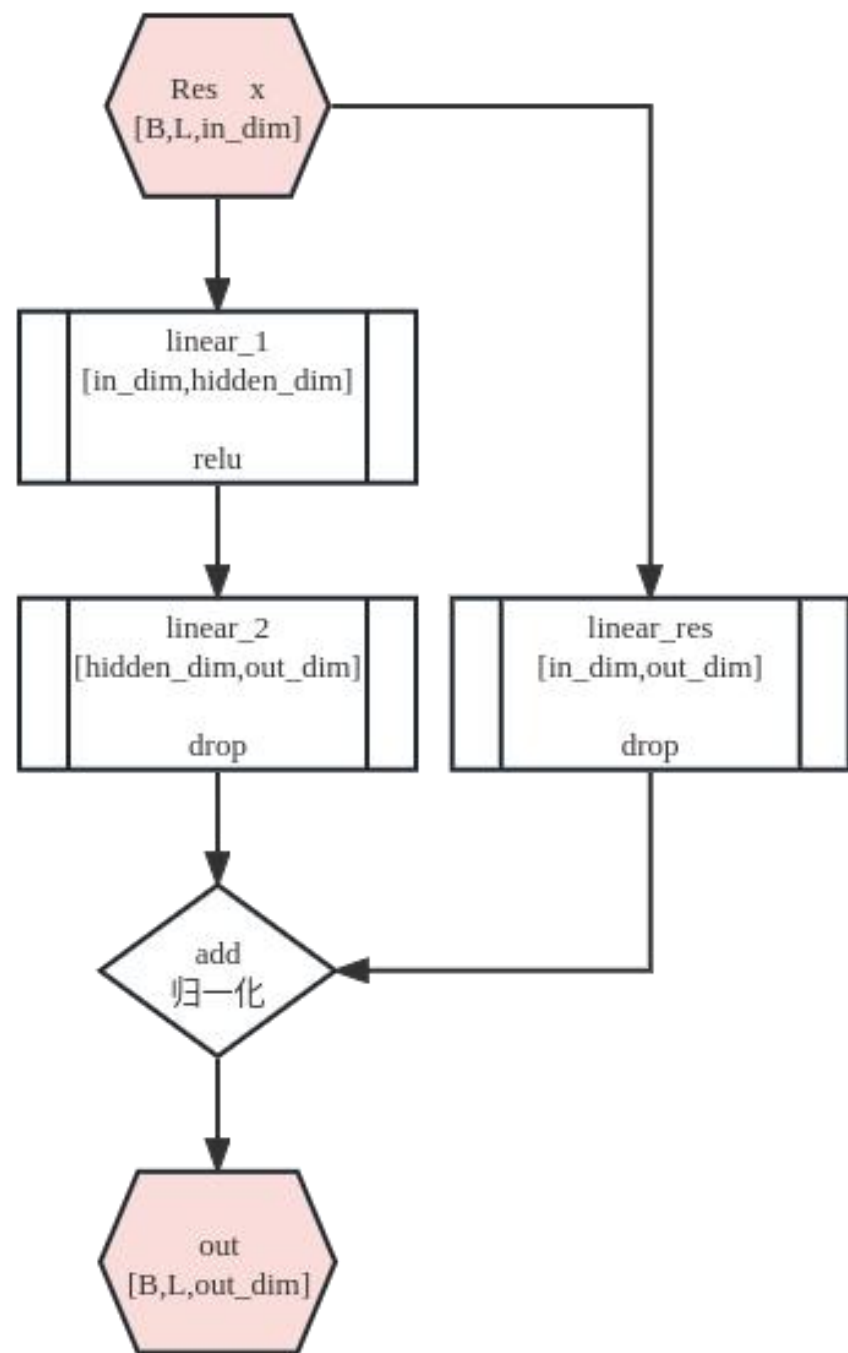
mfcc&spec

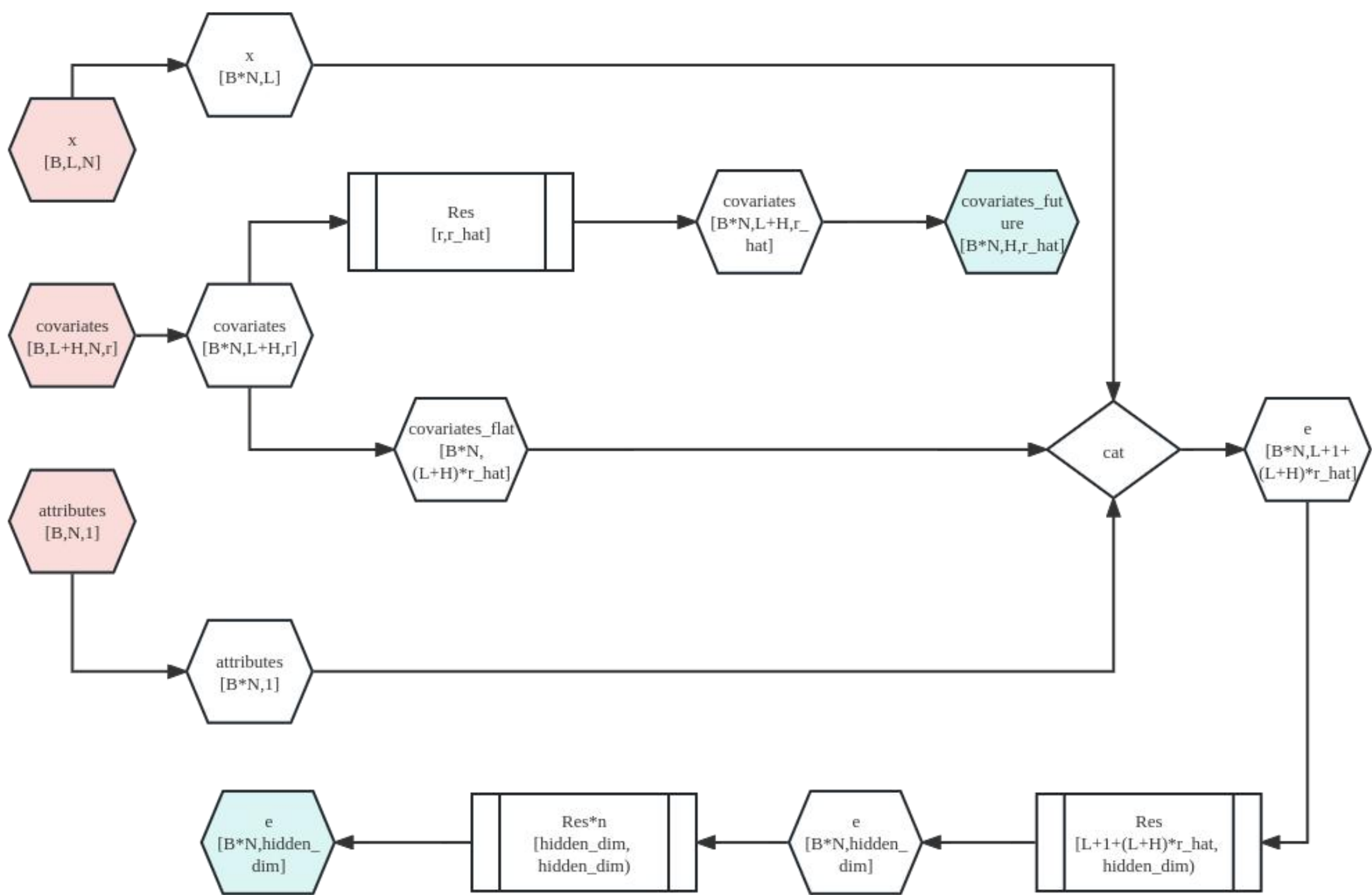


作者	数据库	模态	特征	分类器	使用数据	验证方法	UA /%	WA /%
Cai et al. ^[64]	IEMOCAP	Speech	log power-spectrum	CNN LSTM MAML	Angry, Happy, Sad, Neutral	5-fold cross validation	70.32	76.64
Liu et al. ^[65]	IEMOCAP	Speech	MFCCs	CNN Bi-LSTM	Angry, Happy +(excited), Sad, Neutral	5-fold cross validation	78.30	79.52
Peng et al. ^[66]	RECOLA SEWA	Speech	MMCG	LSTM	Arousal, Valence	5-fold cross validation		
Zhu et al. ^[67]	EmoDB CREMA-D	Speech	Log-mel spectro- gram	ACRNN	Angry, Happy +(excited), Sad, Neutral, Disgust, Fear, Boredom	10-fold cross validation	77.20 73.40	77.90 79.60
Latif et al. ^[68]	IEMOCAP MSP- IMPROV	Speech	Emobase2010	Mixup GAN	Angry, Happy +(excited), Sad, Neutral			46.60
Liu et al. ^[69]	IEMOCAP	Speech Text	Mel-spectrogram MFCCs Glove	CNN Bi-LSTM Attention	Angry, Happy, Sad, Neutral	5-fold cross validation	70.08	72.39
Chen et al. ^[70]	IEMOCAP	Speech Text	Log-mel spectrogram	CNN Bi-LSTM Attention ALBERT	Angry, Happy +(excited), Sad, Neutral	5-fold cross validation	72.05	71.06
Li et al. ^[71]	IEMOCAP RAVDESS	Speech Text	Filterbank GloVe	Bi-LSTM	Angry, Happy, Sad, Neutral	5-fold cross validation	73.50 70.80	72.70 72.00
Kumar et al. ^[72]	IEMOCAP	Speech Text	Mel-spectrogram MFCC Chroma	GRU Attention BERT	Angry, Happy +(excited), Sad, Neutral	5-fold cross validation	75.00	71.70
Wang et al. ^[34]	IEMOCAP	Speech Text	MFCCs One-hot embedding	Transformer Attention	Angry, Happy, Sad, Neutral	5-fold cross validation	77.10	76.80
Hou ^[73]	IEMOCAP MELD	Speech Text	LMBFs BERT	GRU Attention	Angry, Happy +(excited), Sad, Neutral	5-fold cross validation	77.60	75.60 64.90
Lian et al. ^[74]	IEMOCAP	Speech Text	IS13-ComparE ELMo	GRU Multi-Head- Attention	Angry, Happy +(excited), Sad, Neutral	session 5 validation		82.68
Pan et al. ^[75]	IEMOCAP	Speech Visual Text	IS13-ComparE Word2vec	LSTM Attention	Angry, Happy +(excited), Sad, Neutral, Frustrated			73.94
Khare et al. ^[76]	CMU- MOSEI	Speech Visial Text	LFBE GloVe	Transformer Multi-Head- Attention	Angry, Happy, Sad, Disgust, Surprise, Fear			66.40

Table 2. Ablation study on the proposed model.

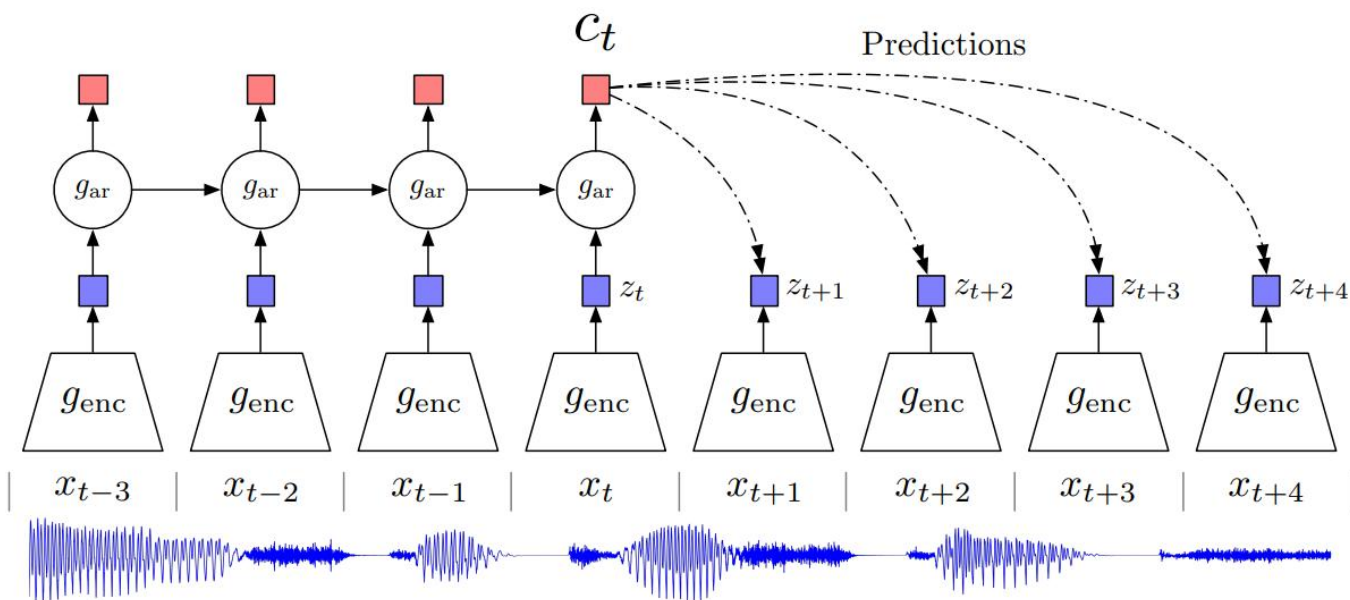
Model	WA	UA
MFCC	57.60	58.09
Spectrogram	62.13	62.25
W2E	64.03	65.67
MFCC+W2E (w/o co-att)	64.62	65.93
Spectrogram+W2E (w/o co-att)	66.20	67.22
MFCC+Spectrogram+W2E (w/o co-att)	67.22	67.81
W2E (w/ co-att)	67.55	68.65
MFCC+W2E (w/ co-att)	69.11	70.30
Spectrogram+W2E (w/ co-att)	70.05	71.30
MFCC+Spectrogram+W2E (w/ co-att)	71.64	72.70





Contrastive Predictive
Coding
(对比预测编码)

- target (future) x : 预测目标
- context (present) c : 已知上下文



网络的目标是最大化 x 和 c 之间的互信息

$$I(x; c) = \sum_{x, c} p(x, c) \log \frac{p(x|c)}{p(x)}$$

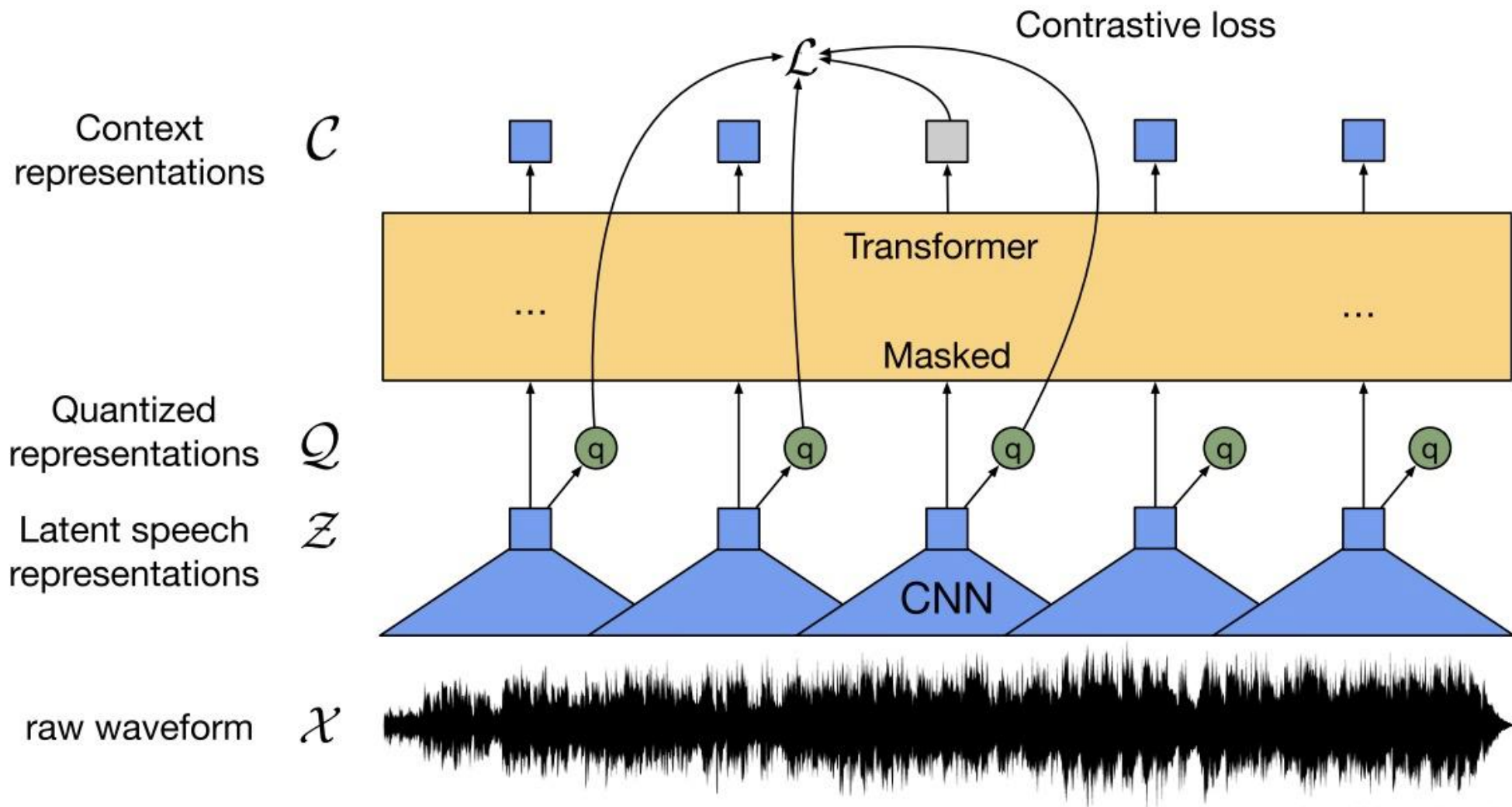
给定 c , 找到最契合的 x , 而不是随机采样

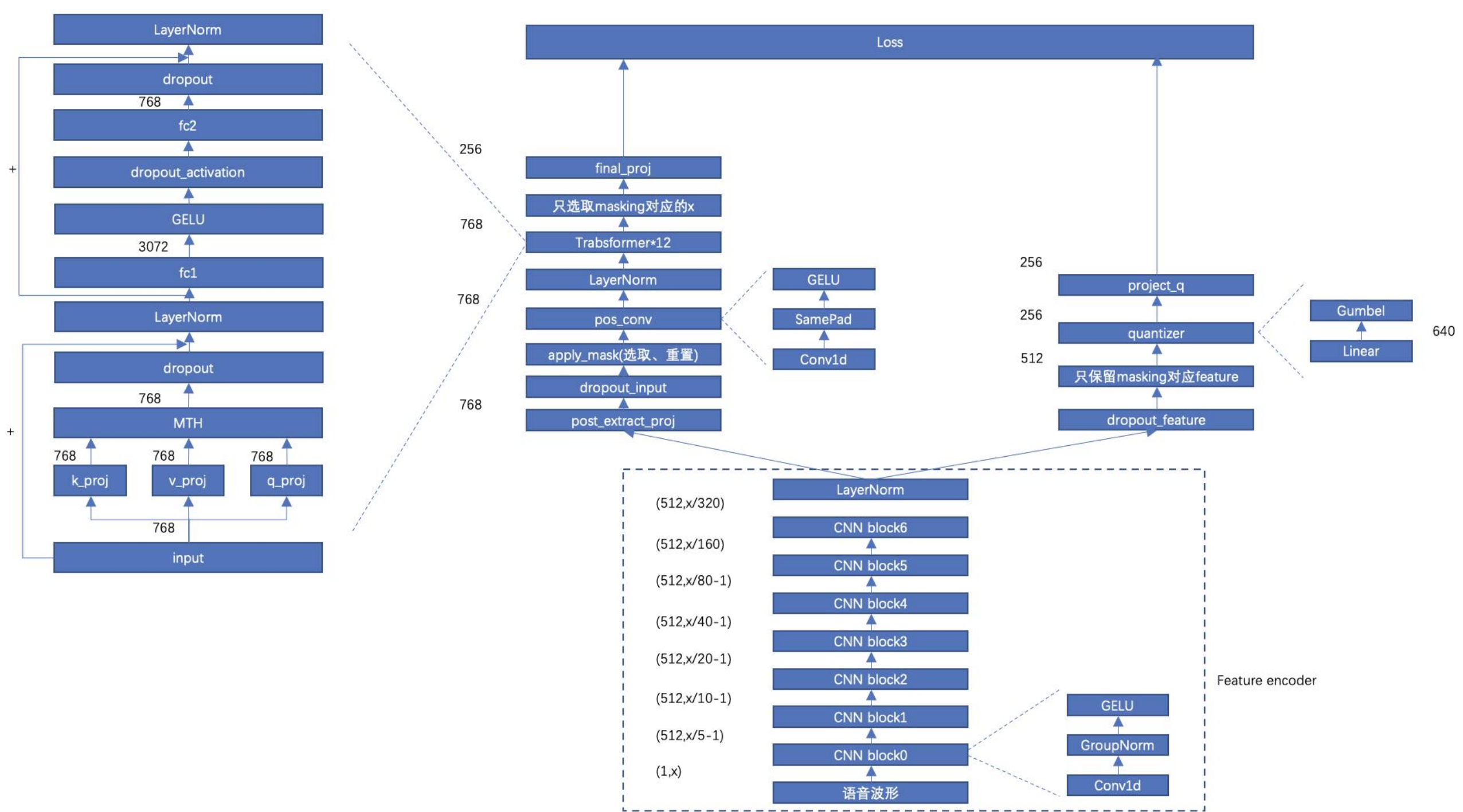
信息熵 $H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i)$

$$\begin{aligned} I(x, c) &= \sum_{x, c} p(x, c) \log \frac{p(x|c)}{p(x)} \\ &= \sum_{x, c} p(x, c) \log \frac{p(x|c)p(c)}{p(x)p(c)} \\ &= \sum_{x, c} p(x, c) \log p(x|c) - \sum_{x, c} p(x, c) \log p(x) \\ &= \sum_c p(c) \sum_x p(x|c) \log p(x|c) - \sum_x \sum_c p(x, c) \log p(x) \\ &= H(x) - H(x|c) \end{aligned}$$

- 样本: $X = \{x_1, \dots, x_N\}$
- x_{t+k} 为正样本 (positive sample), 其余 $N-1$ 个是负样本。
- 在用 c_t 预测 x_{t+k} , 尽可能预测到正样本

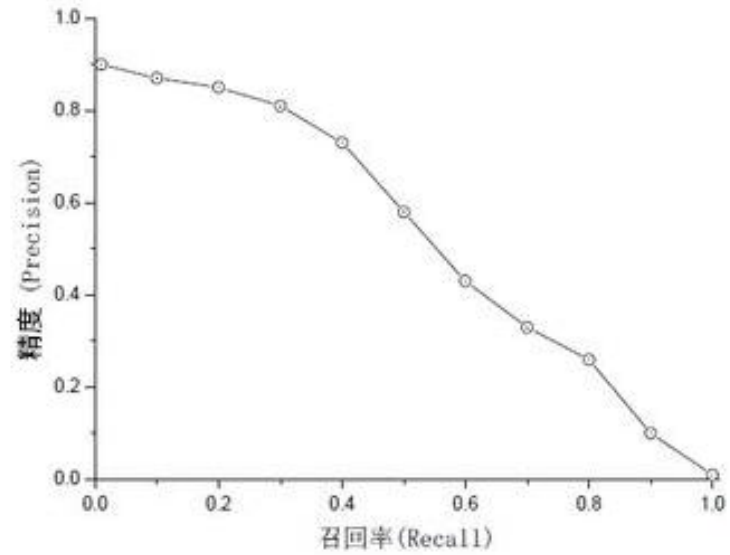
$$\mathcal{L}_N = - \mathbb{E}_X \left[\log \frac{f_k(x_{t+k}, c_t)}{\sum_{x_j \in X} f_k(x_j, c_t)} \right]$$





准确/召回 率
评价指标 roc auc

真阳性	true positive	正确地检测到阳性结果
假阳性	false positive	错误地检测到阳性结果
真阴性	true negative	正确地检测到阴性结果
假阴性	false negative	错误地检测到阴性结果



$$AP = \int_0^1 p(r)dr$$

准确率 (Accuracy)	全部预测中，正确预测结果占比
精度 (Precision)	全部阳性预测中，正确预测结果占比
召回率 (Recall)	全部阳性事件中，正确预测结果占的比例
误报率	全部阳性预测中，错误预测结果占比

$$a = \frac{TP + TN}{TP + TN + FP + FN}$$

$$p = \frac{TP}{TP + FP}$$

$$r = \frac{TP}{TP + FN} = \frac{TP}{P}$$

$$q = \frac{FP}{TP + FP} = 1 - p$$



• FP越小越好

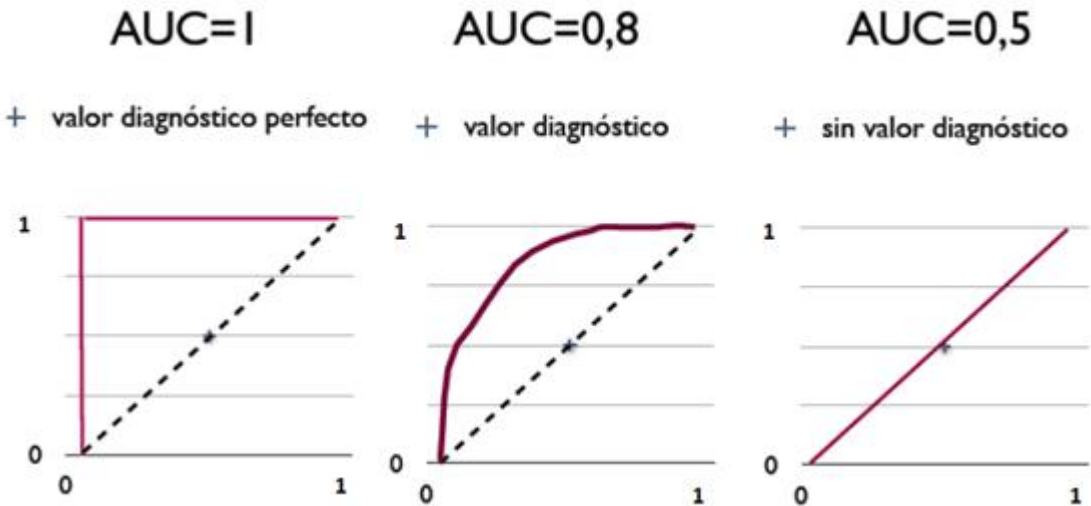
• FN越小越好



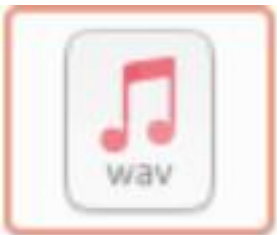
受试者工作特征曲线
(ROC曲线)

$$AP = \int_0^1 p(r)dr$$

AUC (Area under Curve): ROC 曲线下的面积，介于 0.1 和 1 之间，作为数值可以直观的评价分类器的好坏，值越大越好。



AUC = 1	完美分类器，采用这个预测模型时，存在至少一个阈值能得出完美预测。。绝大多数预测的场合，不存在完美分类器。
0.5 < AUC < 1	优于随机猜测。这个分类器（模型）妥善设定阈值的话，能有预测价值。
AUC = 0.5	跟随机猜测一样（例：丢铜板），模型没有预测价值。
AUC < 0.5	比随机猜测还差；但只要总是反预测而行，就优于随机猜测。



`sample_width`(eg:8,16,24,32):每个样本的位数(分辨率)

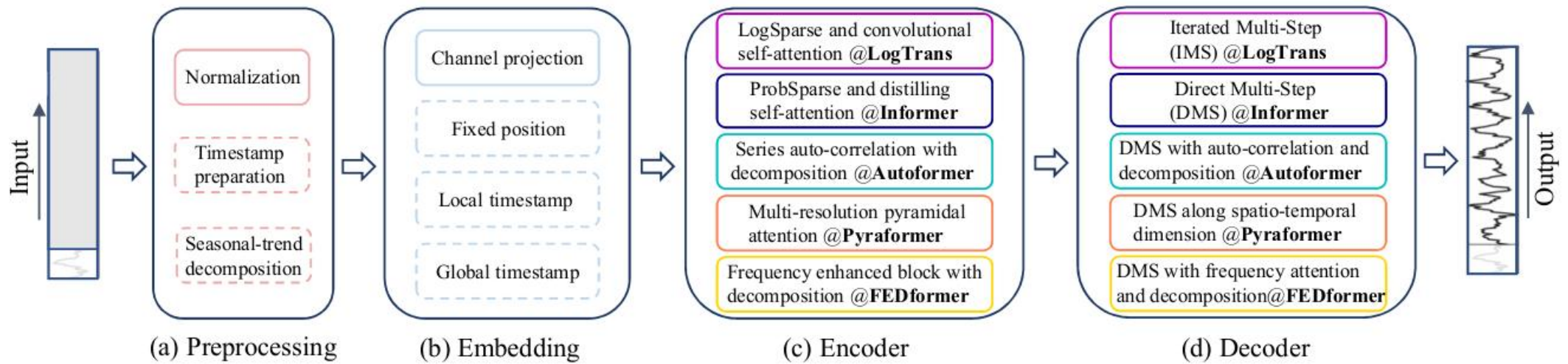
`sample_rate` (采样率):音频中每秒钟采样的样本数。

`num_frames` (帧数):音频信号中的采样帧的总数。

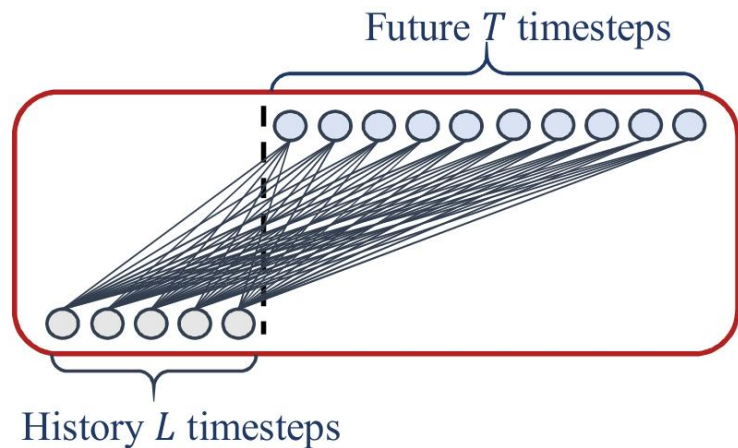
`num_channels` (声道数):音频信号中的独立声道数量

model	算法	DLinear	[Autoformer, Informer, Transformer]
假阳性	false positive		错误地检测到阳性结果
真阴性	true negative		正确地检测到阴性结果
假阴性	false negative		错误地检测到阴性结果

2023.6.3



D_linear

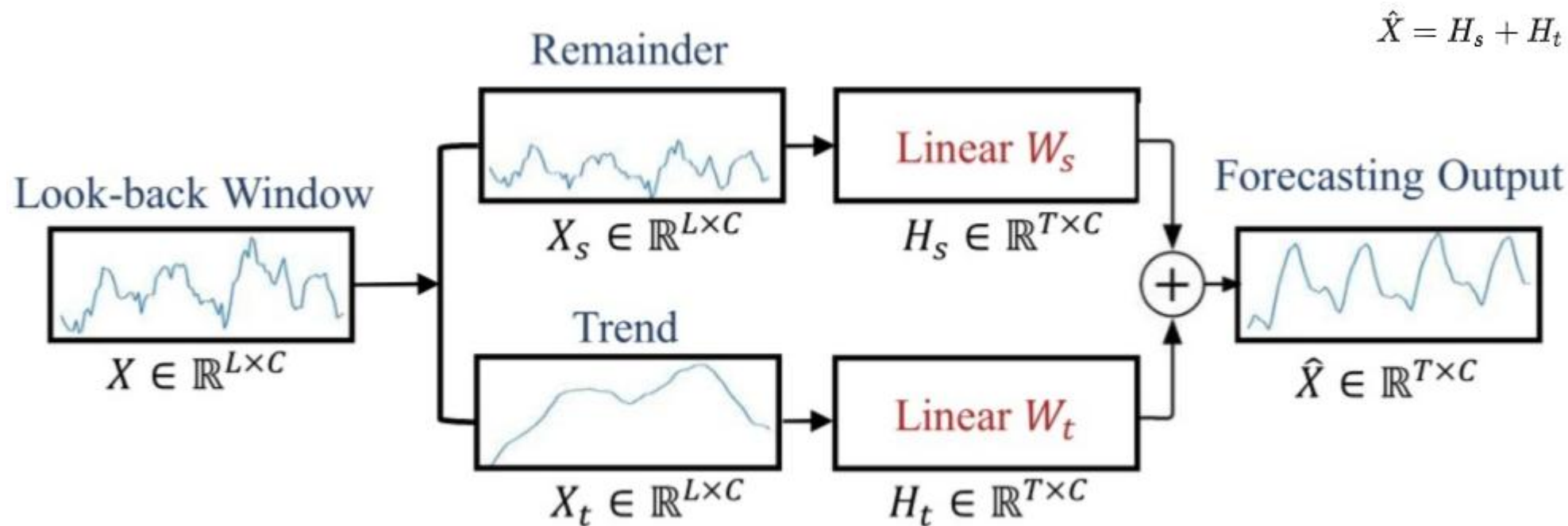


- 历史时间序列数据
- 趋势(Trend)数据
- 剩余(Remainder)数据

$$H_s = W_s X_s \in \mathbb{R}^{T \times C}, W_s \in \mathbb{R}^{T \times L}$$

$$H_t = W_t X_t \in \mathbb{R}^{T \times C}, W_t \in \mathbb{R}^{T \times L}$$

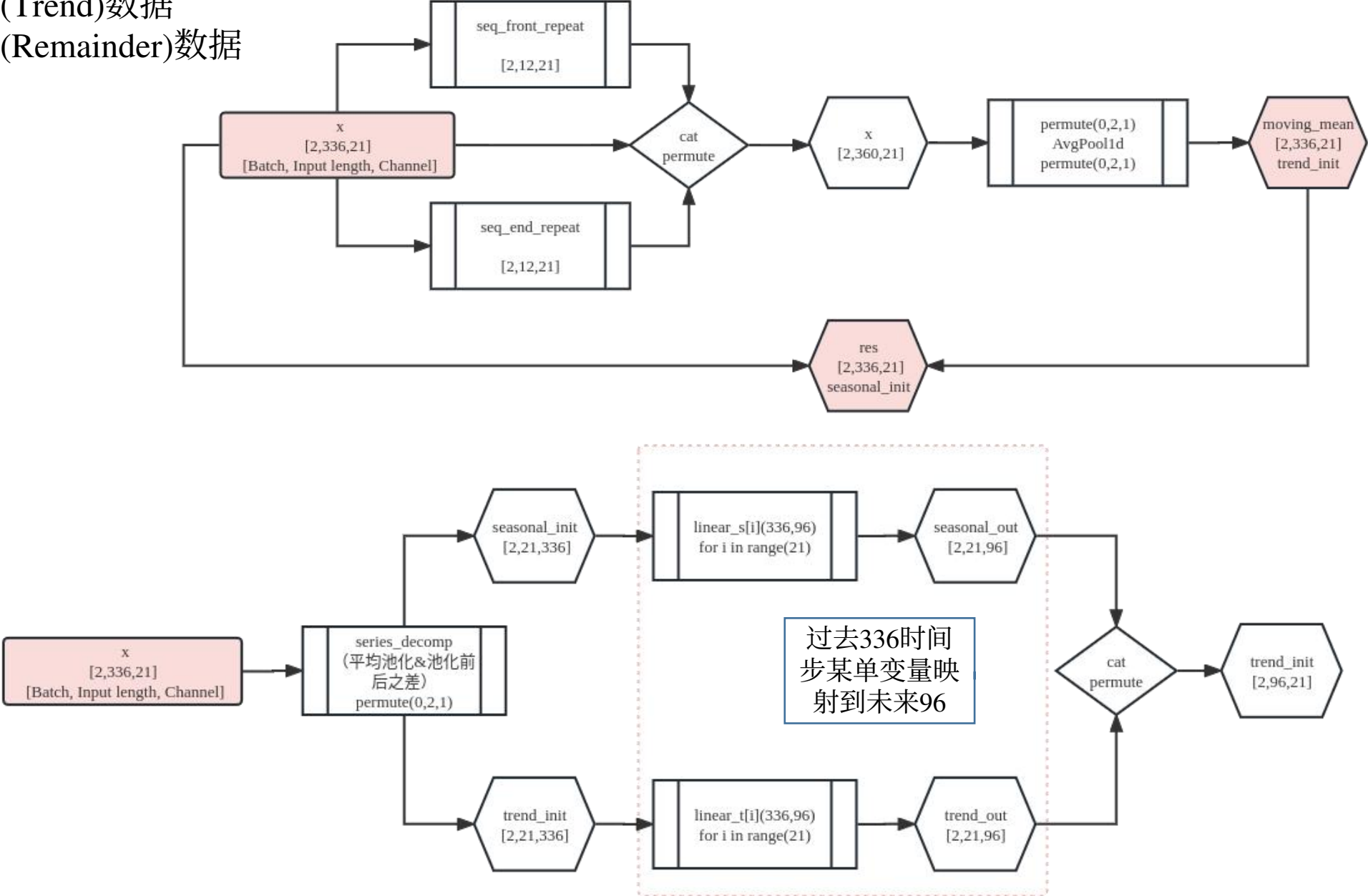
Figure 2. Illustration of the basic linear model.



$$\hat{X} = H_s + H_t$$

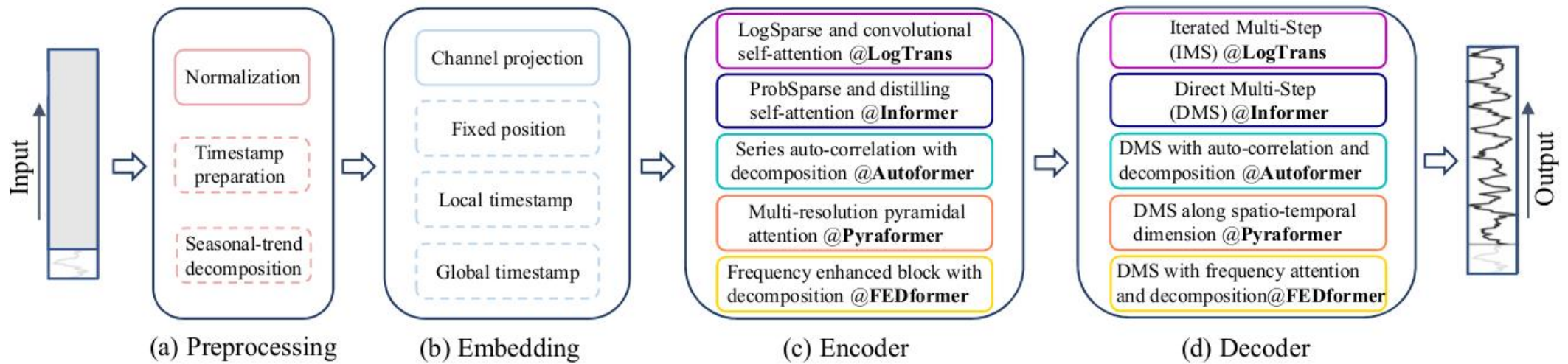
(a) The whole structure of *DLinear*

- 历史时间序列数据
- 趋势(Trend)数据
- 剩余(Remainder)数据

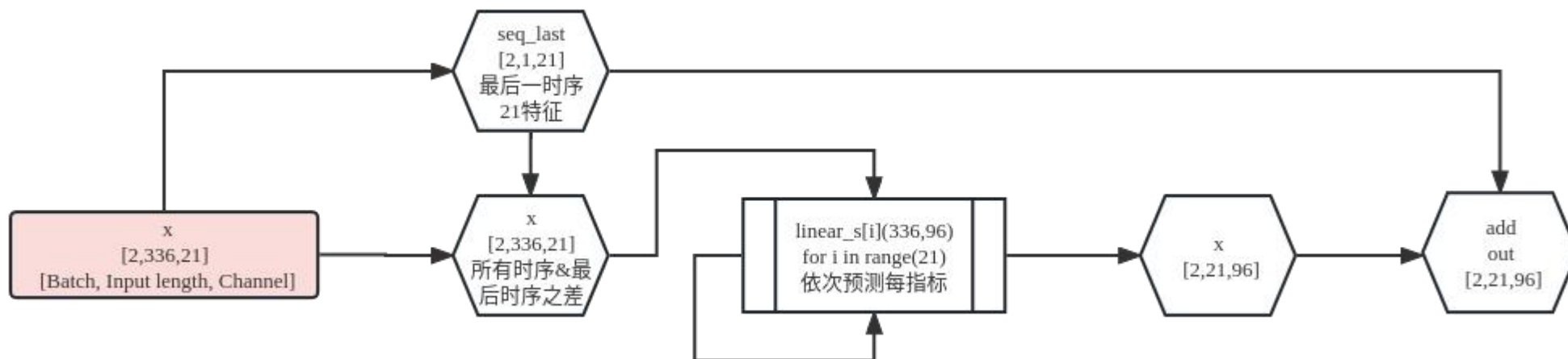


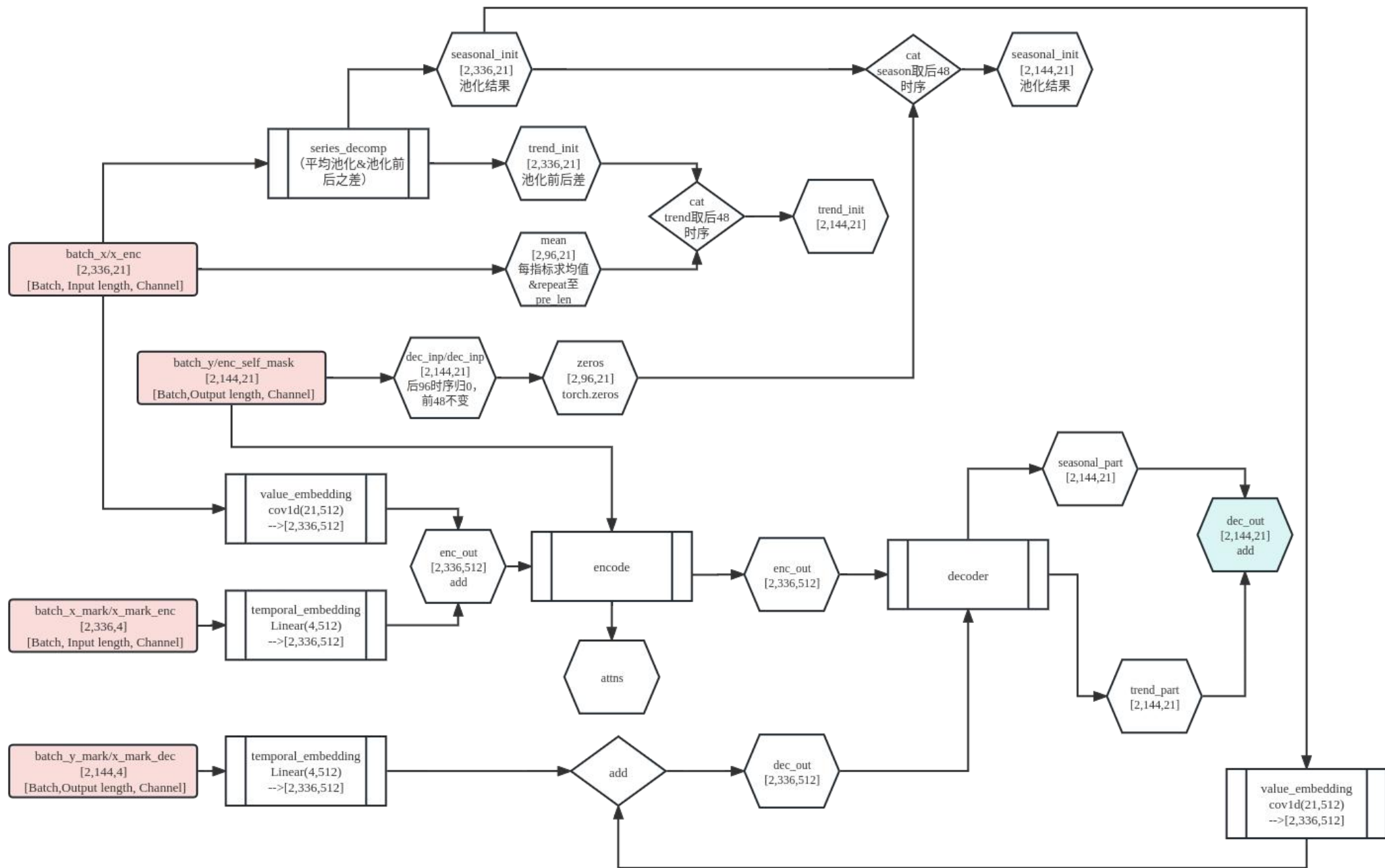
Methods	WA	UA
Speech-only		
LSTM+Attn (our implementation)	63.4	57.4
LSTM+Attn (Mirsamadi <i>et al.</i> , 2017)	63.5	58.8
CNN+LSTM (Satt <i>et al.</i> , 2018)	68	59.4
TDNN+LSTM (Sarma <i>et al.</i> , 2018)	70.1	60.7
Text-only (ASR text)		
LSTM+Attn	60.3	54.8
Multimodal (ASR text)		
Concat (our implementation)	68.1	66.0
Concat (Yoon <i>et al.</i> , 2018)	69.1	-
Proposed	70.4	69.5

2023.6.6



- NLinear将当前输入减去上一时刻的值，然后经过一个线性层，然后在做最终的预测前将减去的部分加回来。对数据的加减是对输入序列的一种简单标准化(normalizaiton)





2023.6.9

name	shape
audio_spec	[batch, 3, 256, 384]
audio_mfcc	[batch, 300, 40]
audio_wav	[batch, 48000]

D_linear

####spec+dlinear####

```
Ser_Model(  
  (Linear_Seasonal): ModuleList(  
    (0-127): 128 x Linear(in_features=768, out_features=128, bias=True)  
  )  
  (Linear_Trend): ModuleList(  
    (0-127): 128 x Linear(in_features=768, out_features=128, bias=True)  
  )  
  (post_spec_dropout): Dropout(p=0.1, inplace=False)  
  (decompstion): series_decomp(  
    (moving_avg): moving_avg(  
      (avg): AvgPool1d(kernel_size=(25,), stride=(1,), padding=(0,))  
    )  
  )  
  (dlinear1): Linear(in_features=49152, out_features=128, bias=True)  
  (dlinear2): Linear(in_features=128, out_features=4, bias=True)  
)
```

####spec+dlinear####

```
Ser_Model(  
  (decompstion): series_decomp(  
    (moving_avg): moving_avg(  
      (avg): AvgPool1d(kernel_size=(25,), stride=(1,), padding=(0,))  
    )  
  )  
  (Linear_Seasonal): ModuleList(  
    (0-39): 40 x Linear(in_features=300, out_features=16, bias=True)  
  )  
  (Linear_Trend): ModuleList(  
    (0-39): 40 x Linear(in_features=300, out_features=16, bias=True)  
  )  
  (dlinear1): Linear(in_features=640, out_features=128, bias=True)  
  (dlinear2): Linear(in_features=128, out_features=4, bias=True)  
  (dropout): Dropout(p=0.01, inplace=False)  
)
```

训练集loss	测试集loss	训练准确率	测试准确率
0.8837	1.1109	58.34%	56.91%

训练集loss	测试集loss	训练准确率	测试准确率
0.8097	1.1729	58.28%	57.09%

MFCC—Nlinear			
训练集loss	测试集loss	训练准确率	测试准确率
1.1280	1.1193	55.56%	54.94%

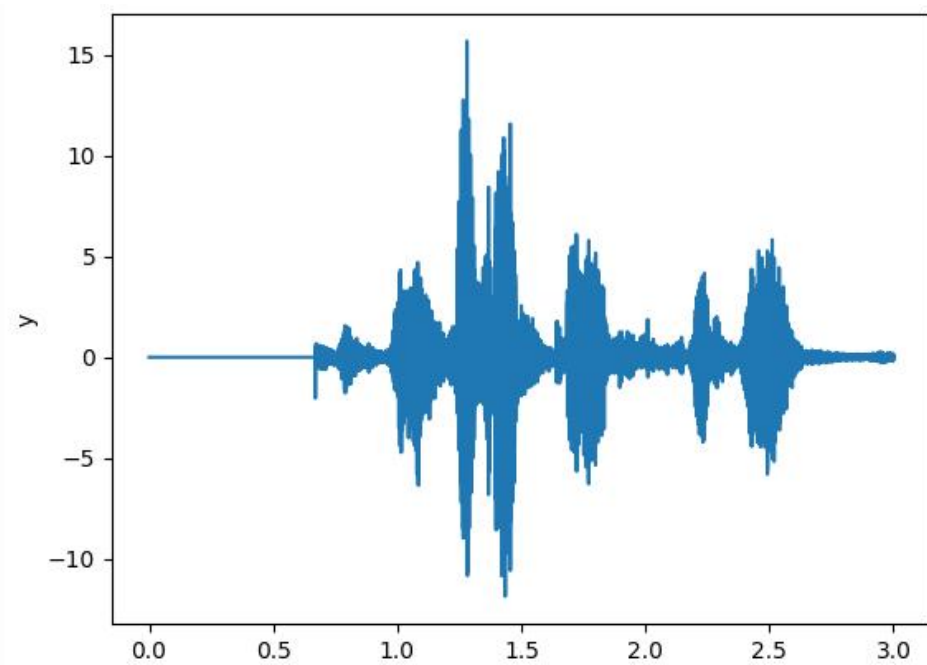
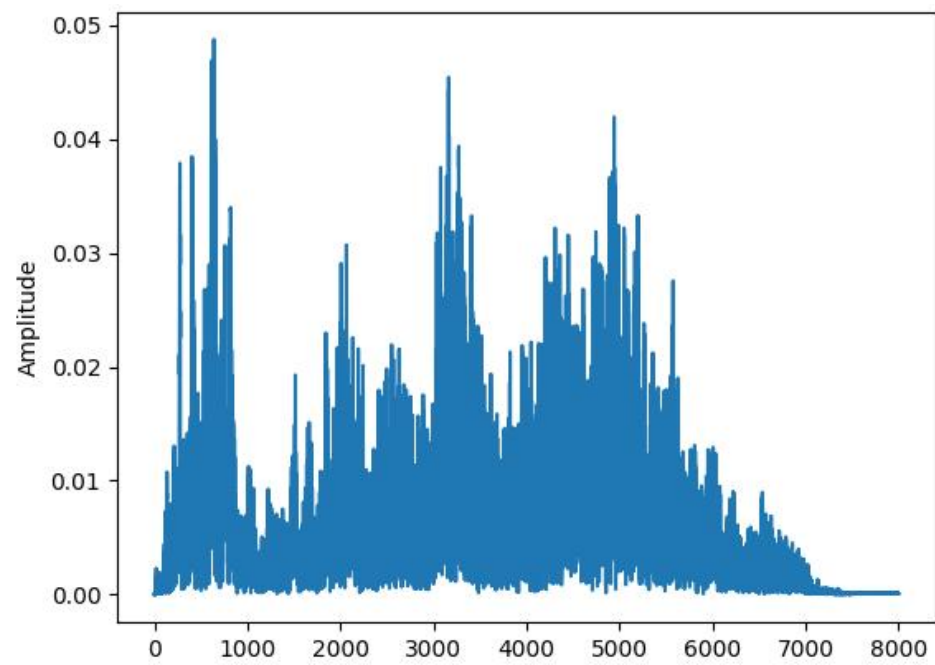
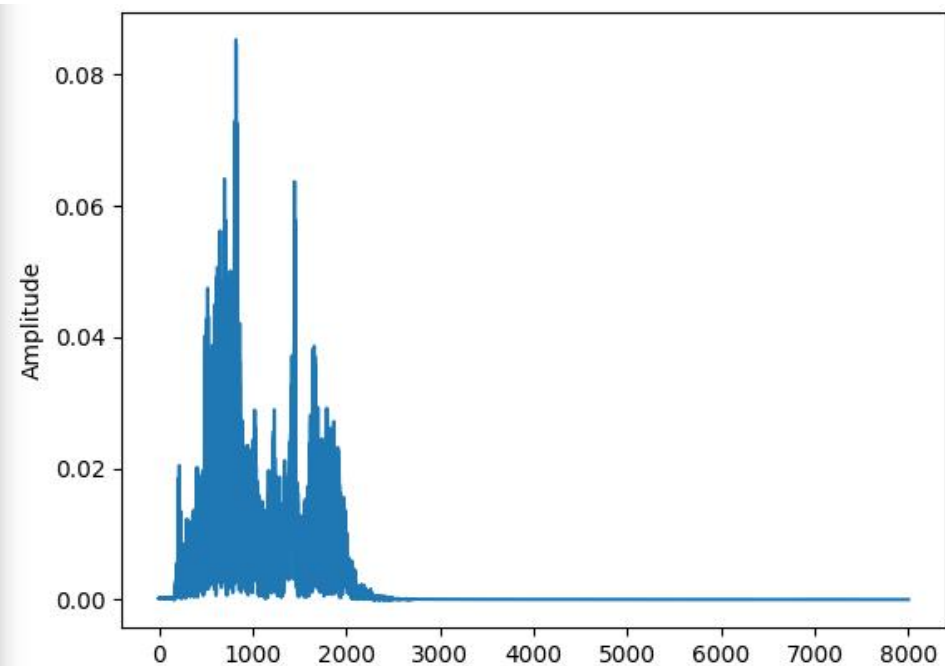
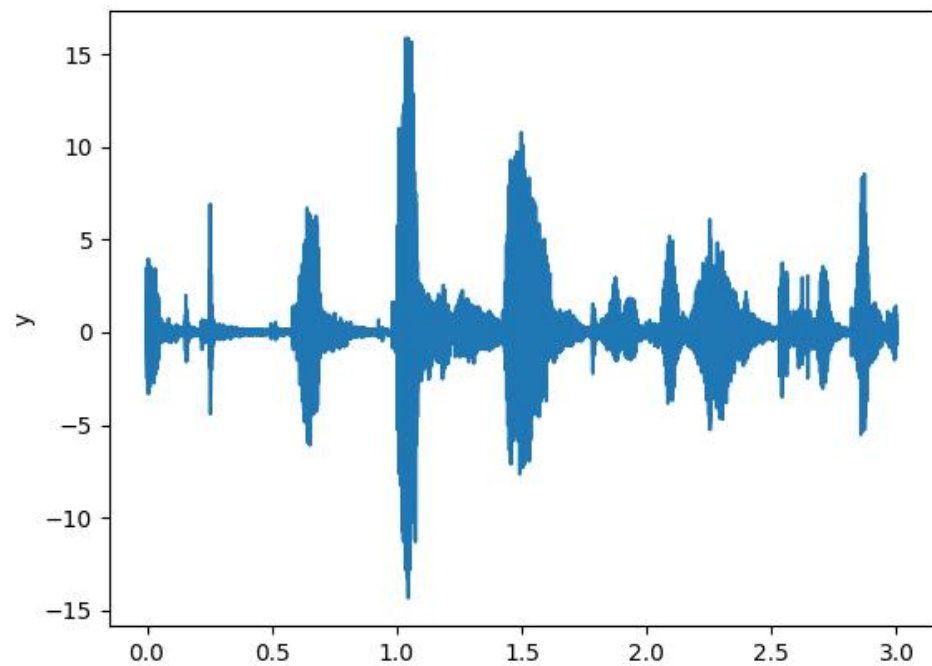
2023.6.10

spec_Dlinear
wav2vec
cat

```
CA-github5_wav2vac1测试开始时间 2023-06-10 17:51:31
2023-06-10 17:51:31 v_id,t_id 为 1M1M
Number of trainable parameters: {140510340}
0 1e-05 1.3535 1.3068 39.50% 39.50% 32.95% 32.95%
1 1e-05 1.1907 1.1163 56.19% 56.19% 60.56% 60.56%
2 1e-05 0.9828 1.0227 59.07% 59.07% 64.88% 64.88%
3 1e-05 0.8792 0.9472 67.15% 67.15% 69.82% 69.82%
```

```
CA-github5_wav2vac1测试开始时间 2023-06-10 16:24:24
2023-06-10 16:24:24 v_id,t_id 为 1M1M
Number of trainable parameters: {140510340}
0 1e-05 1.3535 1.3068 39.50% 39.50% 32.95% 32.95%
1 1e-05 1.1907 1.1163 56.19% 56.19% 60.56% 60.56%
2 1e-05 0.9828 1.0227 59.07% 59.07% 64.88% 64.88%
3 1e-05 0.8792 0.9472 67.15% 67.15% 69.82% 69.82%
```

由于dropout设置较少，两次训练雷同

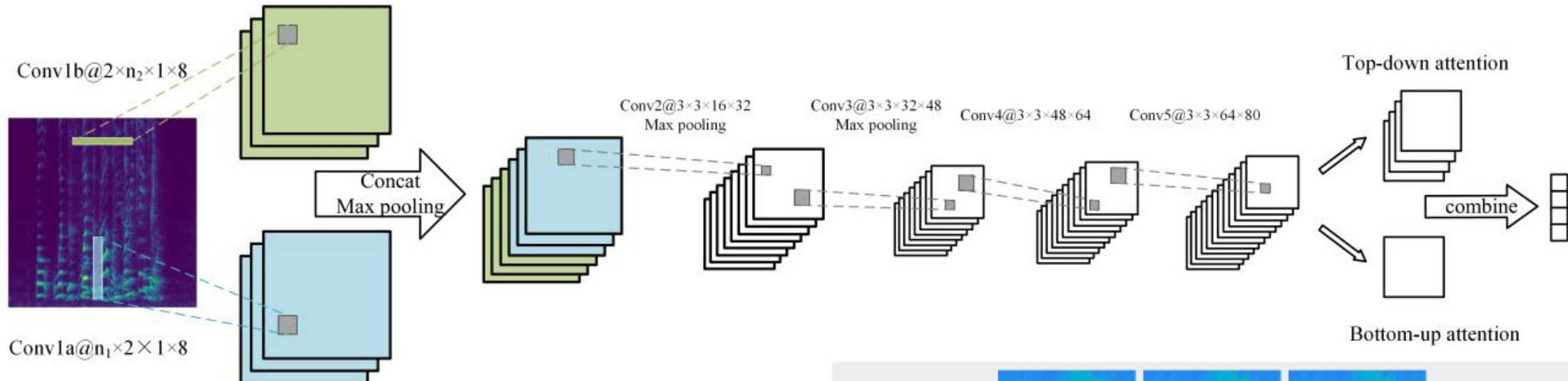


$$J^2(w, b) = J(w, b) + \frac{\lambda}{2m} \Omega(w)$$

式中， $\frac{\lambda}{2m}$ 是一个常数， m 为样本个数， λ 是一个超参数，用于控制正则化程度。

- 目的：防止模型过拟合
- 原理：在损失函数上加上某些规则（限制），缩小解空间，从而减少求出过拟合解的可能性
 - 0范数，向量中非零元素的个数。
 - 1范数，为绝对值之和。
 - 2范数，就是通常意义上的模。
 - $+\infty$ 范数，即求向量元素绝对值中的最值

2023/06/20



Input: spectrogram 200x300 (4000Hz x 3 sec)

Convolution 1: 16 filters of 12x16 (240Hz x 160msec)

Max-pooling 2:1 $\rightarrow$ 100x150

Convolution 2: 24 filters of 8x12 (320Hz x 240 msec)

Max-pooling 2:1 $\rightarrow$ 50x75

Convolution 3: 32 filters of 5x7 (400Hz x 280 msec)

Max-pooling 2:1 $\rightarrow$ 25x37

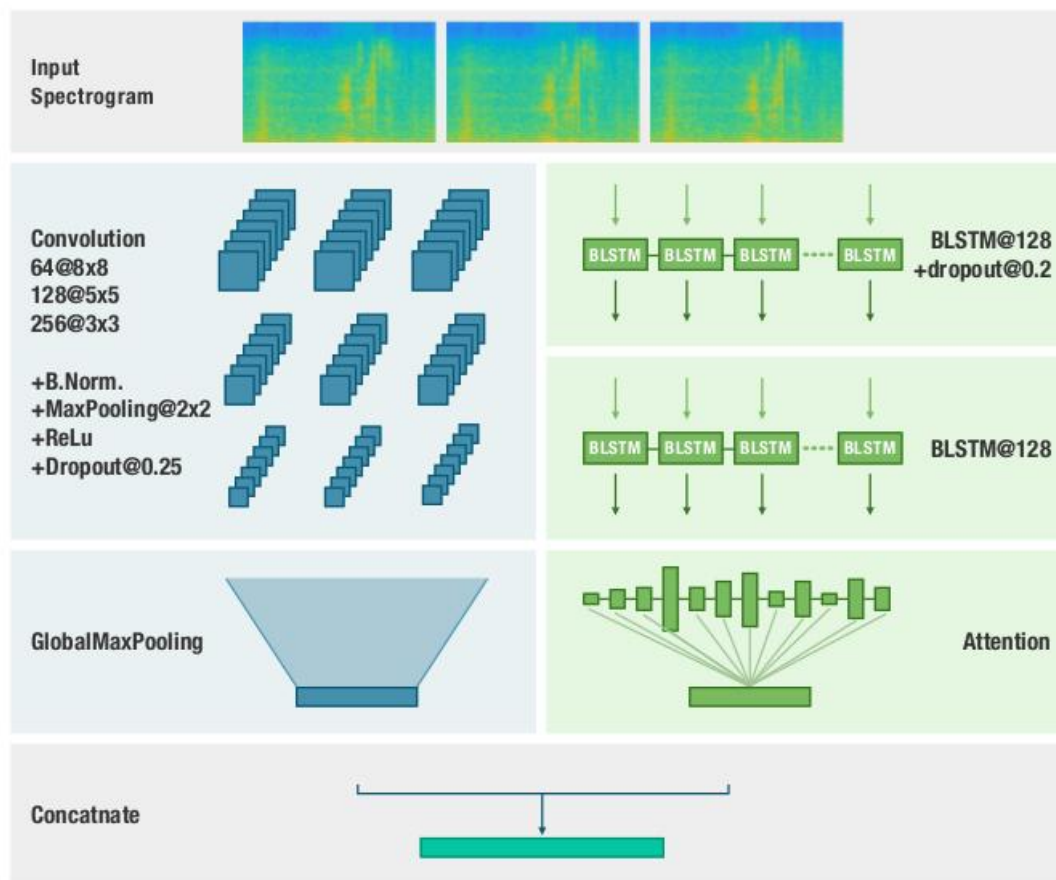
LSTM: bi-directional, 128x2

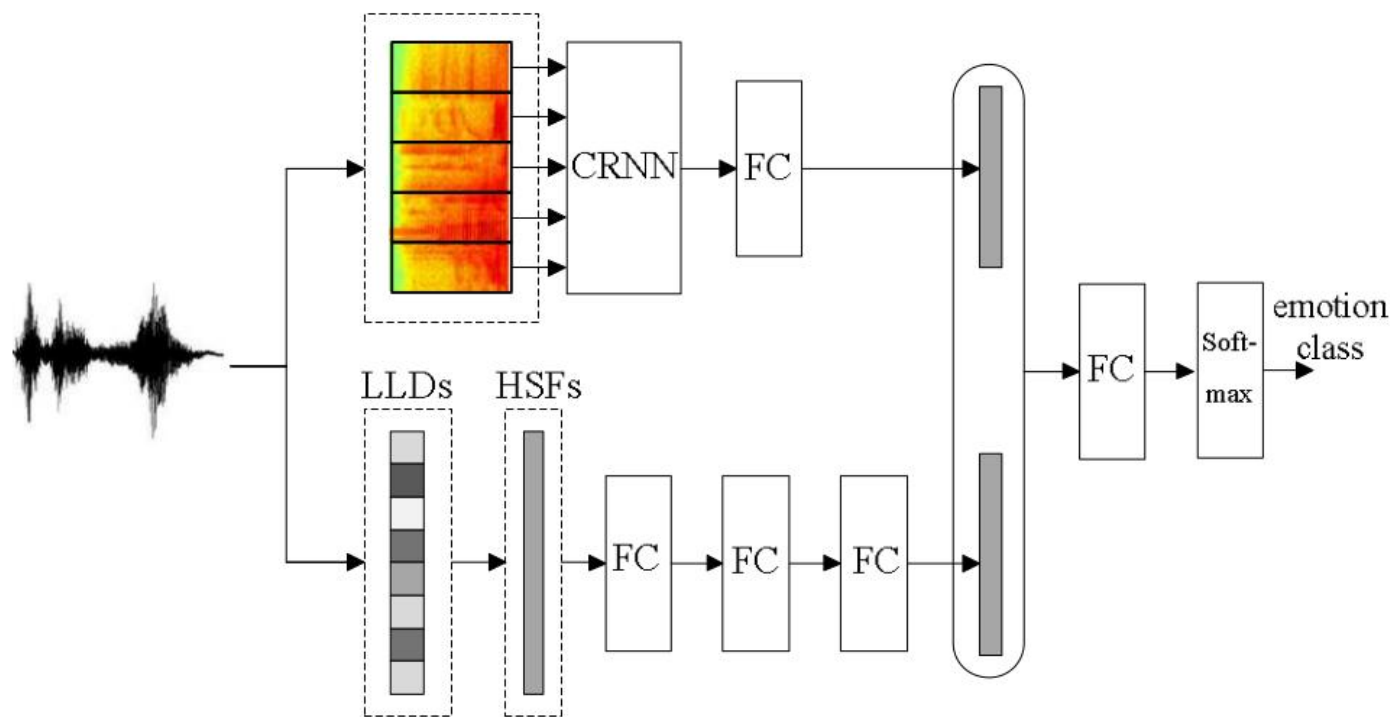
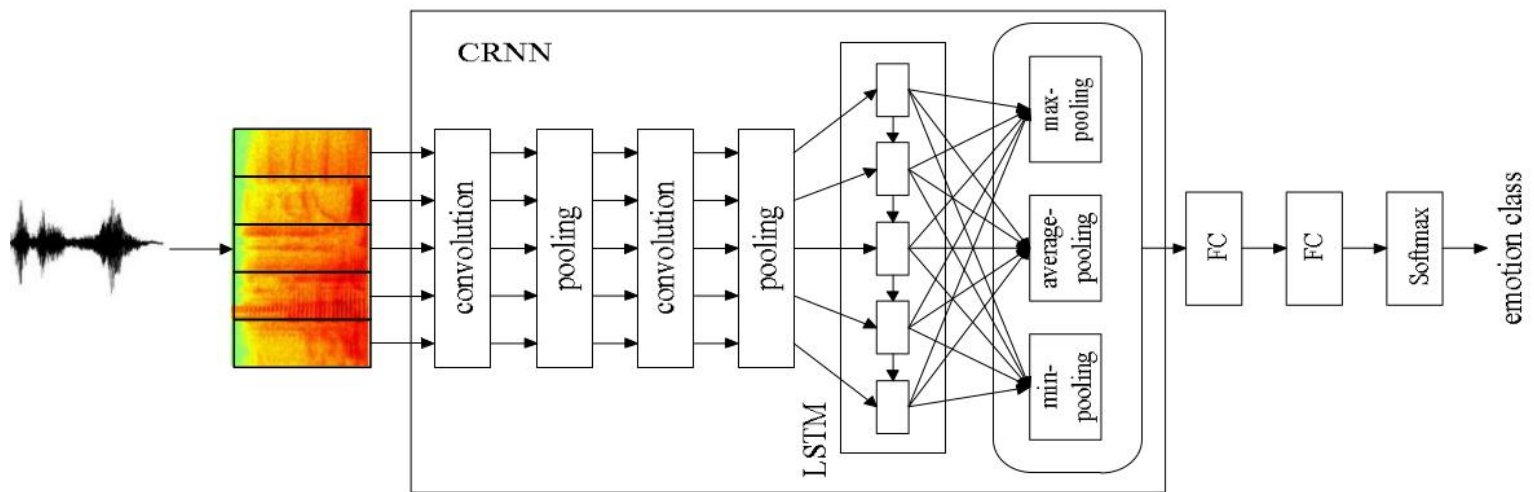
Dense layer: length 64

Dropout: length 64

Soft-max: length 4

Output: 4 posterior probabilities

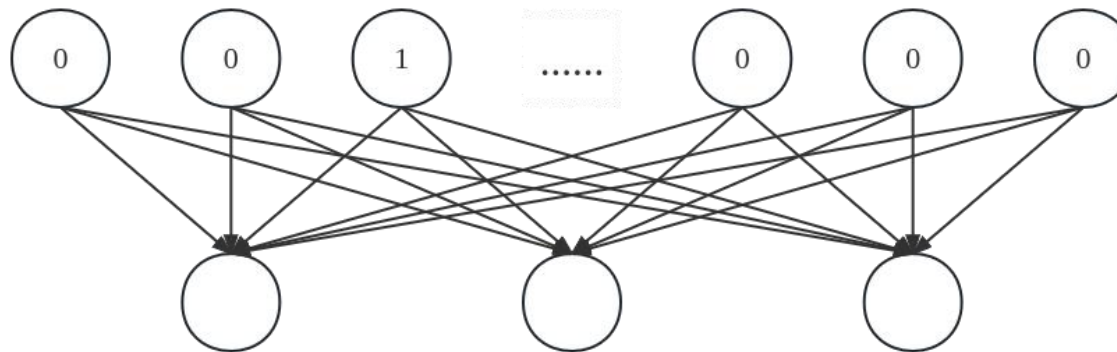
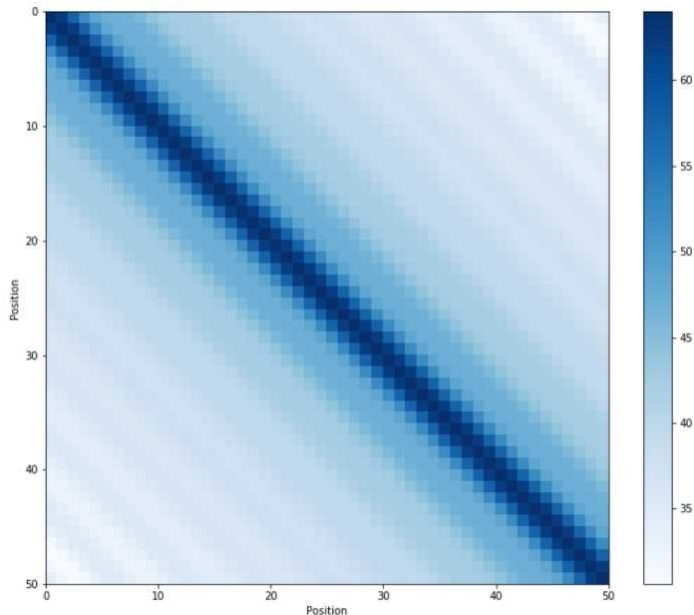




2023/06/26

Bert

- BERT 的创建者:当使用超过 512 个令牌的文档时, 性能会显着下降。



$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases}$$

$$\omega_k = \frac{1}{10000^{2k/d}}$$

- t: 句中该词序号
- d: 词向量维度 (512)
- i: 嵌入的维度序号

Clipping (截断法)
Pooling (池化法)
划窗法
RNN (循环法)

1	原代码	3任务B64-cu		3任务/4-b16		3任务/4-b16	
2 1M	74.33%	74.01%	72.17%	73.25%	73.79%	74.37%	71.81% 74.30%
3 1F	68.37%	71.54%	68.75%	71.93%	68.56%	70.82%	68.56% 71.75%
4 2M	74.91%	78.37%	74.72%	78.64%	75.65%	80.31%	74.91% 76.83%
5 2F	78.17%	78.87%	75.26%	76.33%	75.68%	74.59%	78.59% 78.02%
6 3M	72.97%	73.14%	68.04%	68.59%	69.00%	69.89%	68.52% 68.91%
7 3F	71.84%	71.77%			70.50%	70.10%	69.54% 68.68%
8 4M	65.81%	68.06%			67.00%	68.42%	66.00% 68.24%
9 4F	72.73%	68.48%			75.00%	70.11%	73.67% 73.31%
10 5M	69.43%	70.19%			70.51%	69.60%	70.35% 70.51%
11 5F	71.86%	74.08%			73.05%	74.38%	73.73% 74.45%
12	72.04(t)	72.85(t)			71.87	72.26	71.57 72.95
13	71.64(p)	72.70(p)					

Table 1. Performance comparison with 5-fold leave-one-session-out [12,7,13] and 10-fold leave-one-speaker-out [14,15,16,17,18,5] cross-validation strategy on IEMOCAP.

Model	WA	UA
CNN-ELM+STC attention[12]	61.32	60.43
Audio ₂₅ [7]	60.64±1.96	61.32±2.26
IS09 - classification [13]	68.1	63.8
Ours	69.80	71.05
RNN(prop.)-ELM[14]	62.85	63.89
3D ACRNN[15]	-	64.74±5.44
BLSTM-CTC-CA[16]	69.0	67.0
CNN GRU-SeqCap[17]	72.73	59.71
CNN_TF_Att.pooling[18]	71.75	68.06
HNSD[5]	70.5	72.5
Ours	71.64	72.70

3.1

IE

rec

sci

wc

ca

ter

In

mo

ou

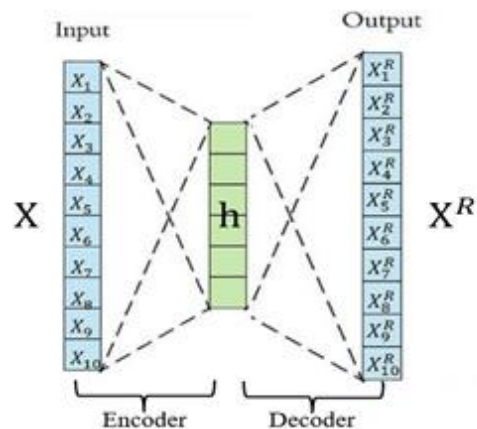
str

we

un

3.2

AutoEncoder 栈式自编码(SAE)



层数深，低层网络梯度弥散

```
AE(  
  (encoder): Sequential(  
    (0): Linear(in_features=784, out_features=256, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=256, out_features=20, bias=True)  
    (3): ReLU()  
  )  
  (decoder): Sequential(  
    (0): Linear(in_features=20, out_features=256, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=256, out_features=784, bias=True)  
    (3): Sigmoid()  
  )  
)
```

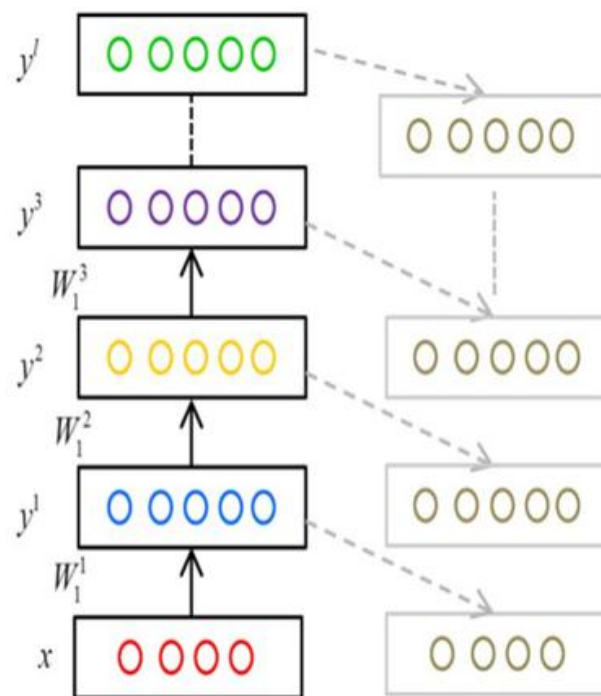
优化目标函数

$$\text{MinimizeLoss} = \text{dist}(X, X^R)$$

栈式自编码(SAE)

- 参数预训练，每一层进行参数初始化
- 对每层网络的进行逐层无监督训练
- 无监督训练完毕后，用有标签的数据对整个网络的参数继续进行梯度下降调整。

无监督预训练
有监督微调



变分自编码器 (VAE)

优点：泛化性强，无监督，不需数据标注
缺点：针对异常识别场景，训练数据需要为正常数据。

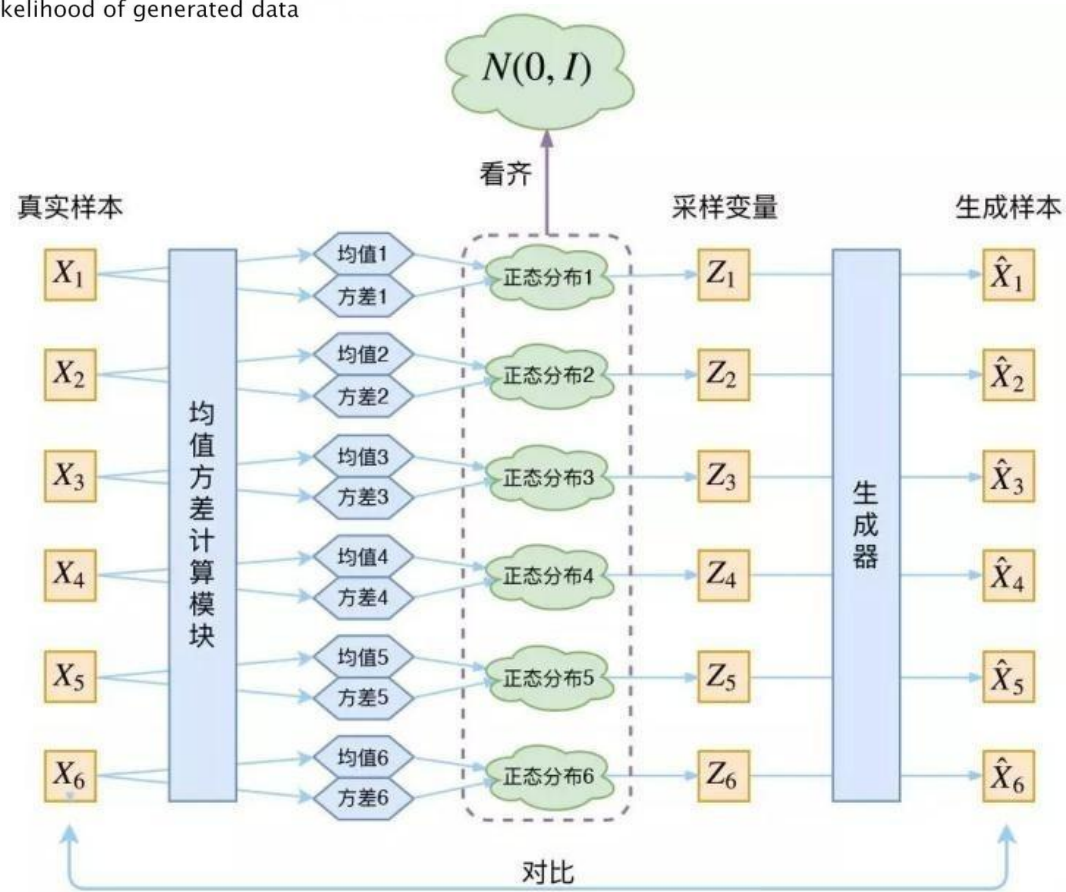
VAE

Hidden Variables

$$p(z|x) = \frac{p(x|z)p(z)}{p(x)}$$

Likelihood of generated data

$P(z)$ 就是隐层变量 Z 的分布
又称先验分布



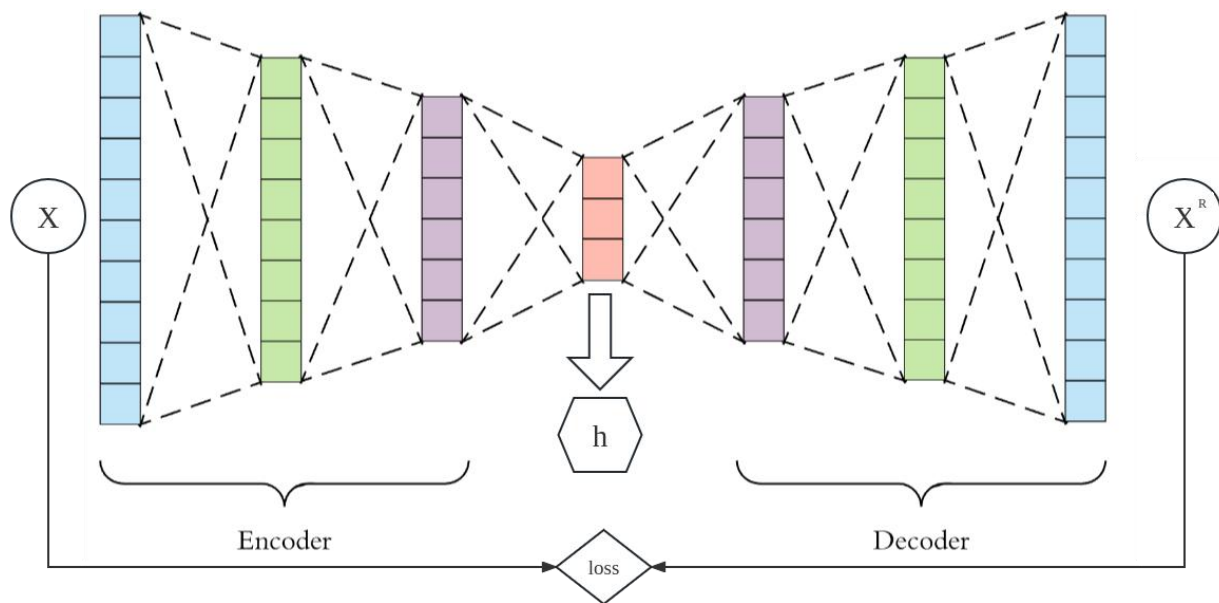
2023/06/26

- AutoEncoder 自编码器
- Variational AutoEncoder 变分自编码器

AutoEncoder 自编码器

自编码是一种非线性降维方式

- 目标：使隐藏层h包含高维输入X尽可能多的重要信息。



自编码器特点

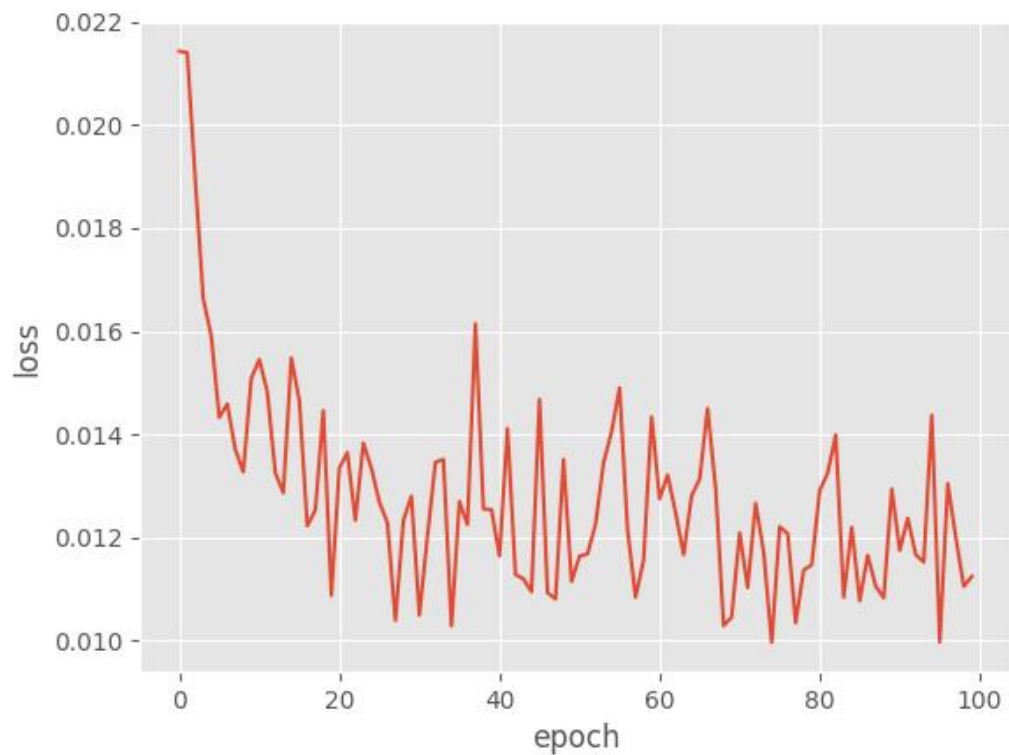
- 前馈神经网络：输出层的数据以及准确率
- 自编码器：中间隐层的结果。

层数深时低层网络梯度弥散时改进方案

- 参数预训练
- 逐层无监督训练
- 用有标签的数据调整

AutoEncoder 自编码器

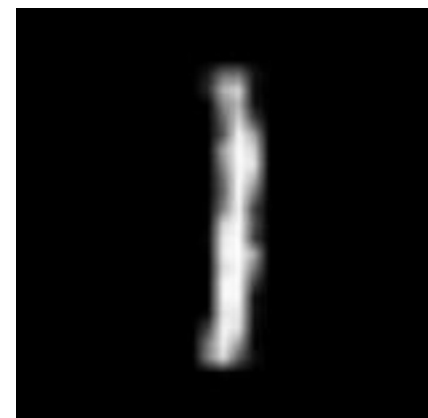
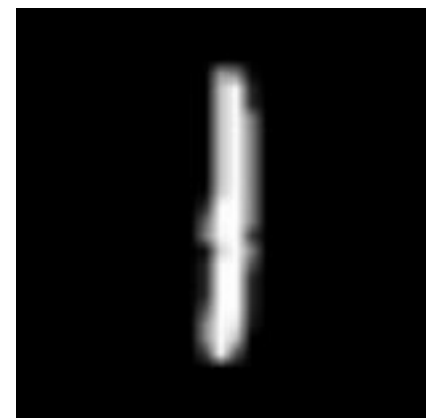
- 实验：手写字母编码与重建
- 数据集：Minst
- 优化目标：原图和重建图均方误差



epoch 0



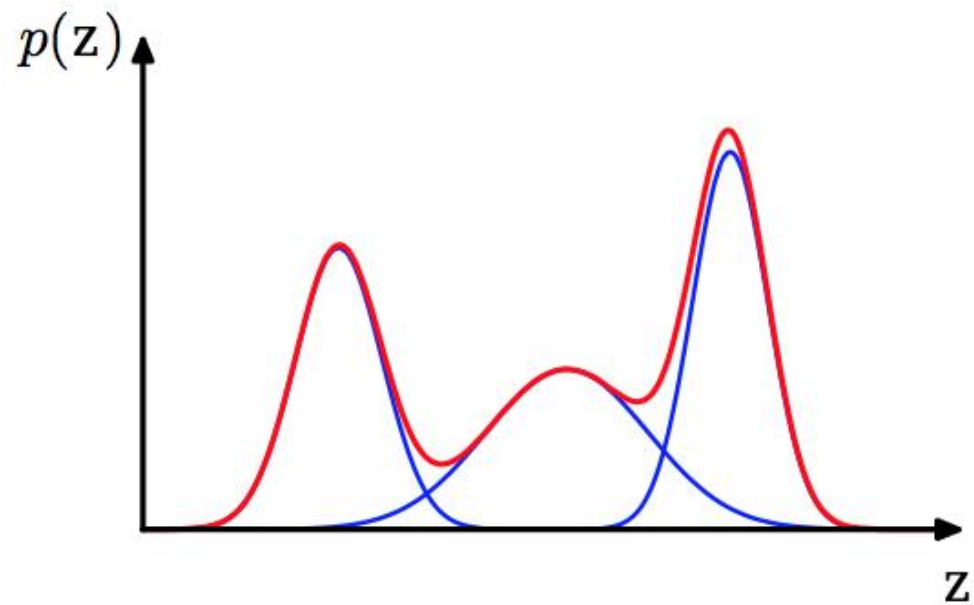
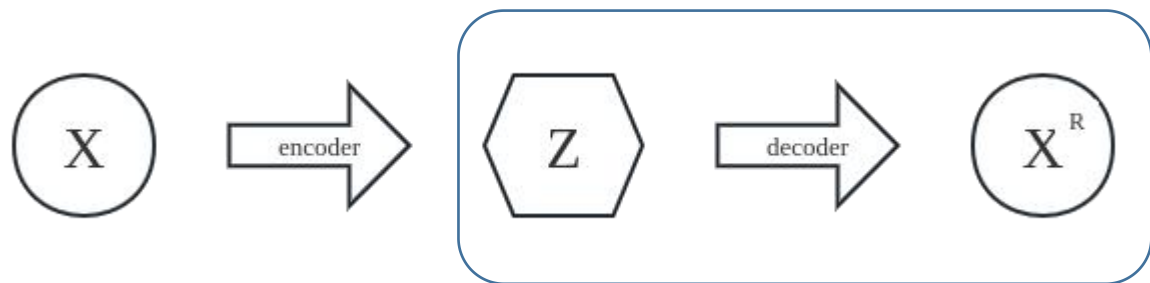
epoch90



变分自编码器 VAE

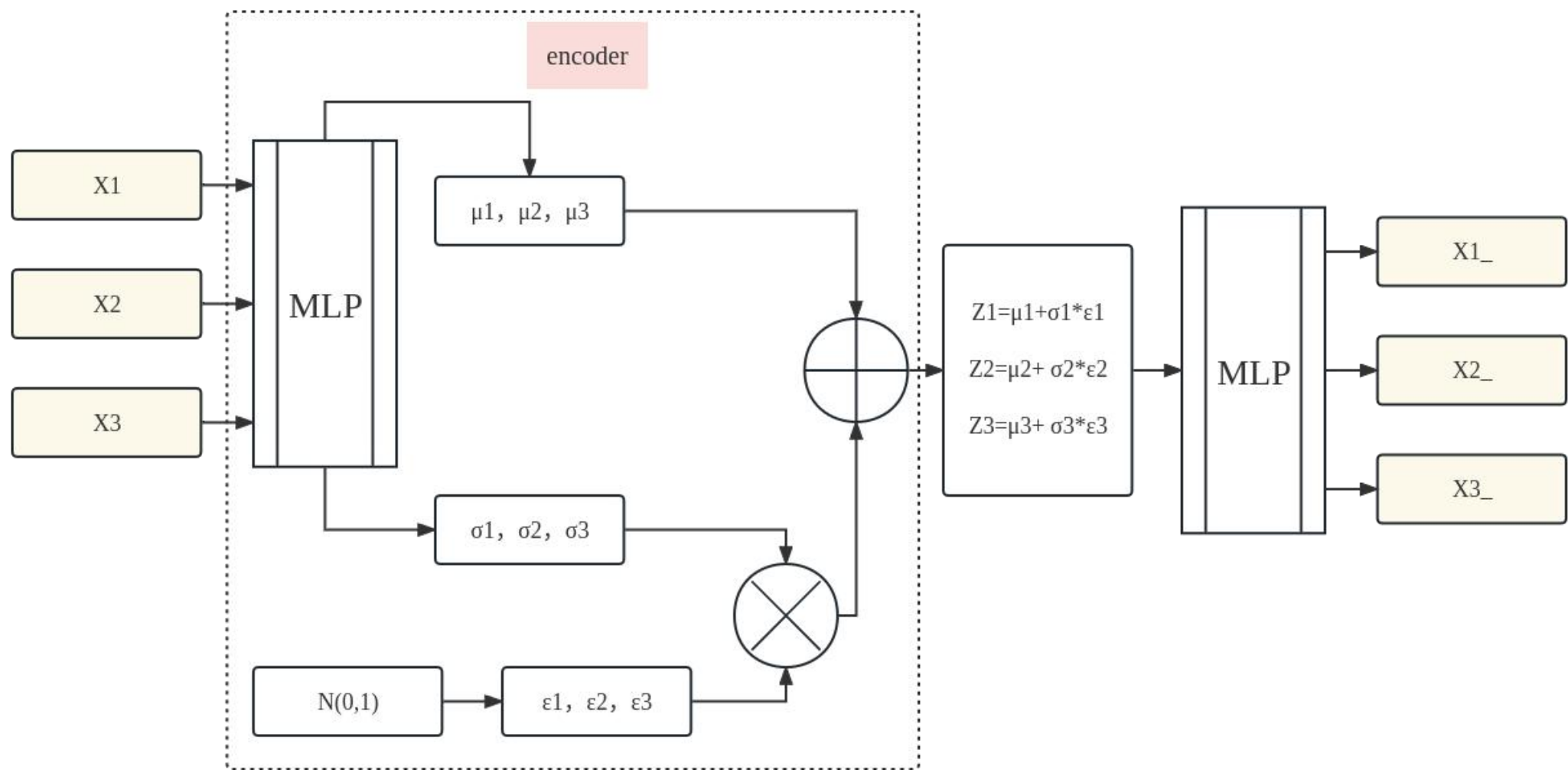
主要目标：构建从隐变量 Z 生成目标数据 X 的模型

假设：样本服从某些常见的分布(eg:正态)



$p(Z)$: 先验分布, 假设为正态分布

$p(X^R|Z)$: 后验分布, 由 Z 来生成 X 的模型



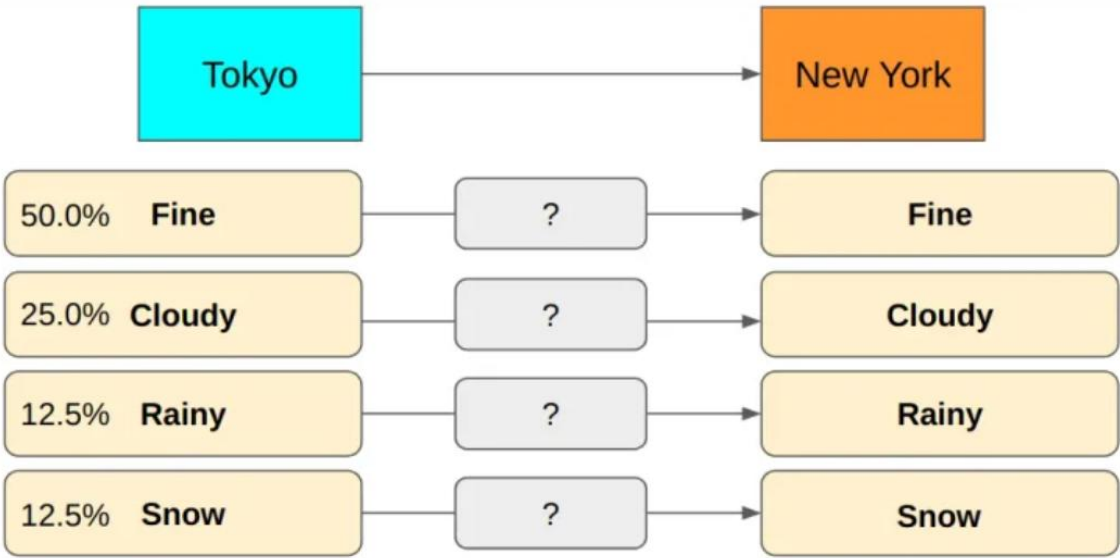
2023/07/02

Huffman compression
哈夫曼编码

0	1	1	0	
---	---	---	---	-------

字符编码两原理：

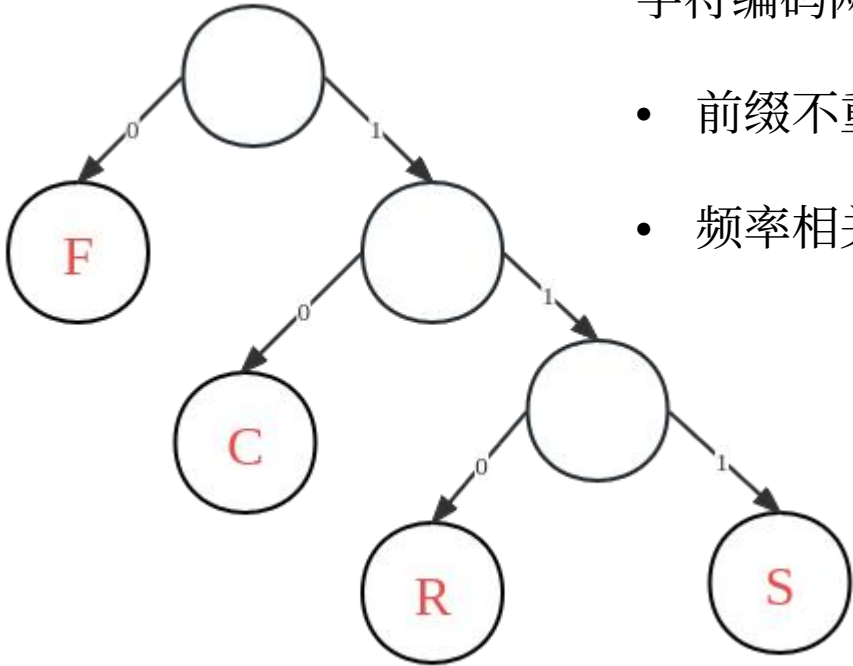
- 前缀不重复（prefix-free）
- 频率相关



编码方式	F(50.0%)	C(25.0%)	R(12.5%)	S(12.5%)	编码长度期望
方式1	0	1	10	11	error
方式2	10	110	0	111	$2*0.5+3*0.25+1*0.125+3*0.125=2.25$
方式3	0	10	110	111	$1*0.5+2*0.25+3*0.125+3*0.125=1.75$

Huffman compression
哈夫曼编码

编码方式	F(50.0%)	C(25.0%)	R(12.5%)	S(12.5%)	编码长度期望
方式1	0	1	10	11	error
方式2	10	110	0	111	$2*0.5+3*0.25+1*0.125+3*0.125=2.25$
方式3	0	10	110	111	$1*0.5+2*0.25+3*0.125+3*0.125=1.75$



字符编码两原理:

- 前缀不重复 (prefix-free)
- 频率相关

长度

- F: $-\log_2 0.5=1$
- C: $-\log_2 0.25=2$
- R: $-\log_2 0.125=3$
- S: $-\log_2 0.125=3$

信息熵就是平均最小编码长度

$$H(X) = - \sum_{i=1}^b P_i \log_2(P_i)$$

熵
信息熵
交叉熵

熵

- 服从某一特定概率分布事件的理论最小平均编码长度
- 熵在信息学中就是信息熵

$$H(X) = - \sum_{i=1}^b P_i \log_2(P_i)$$

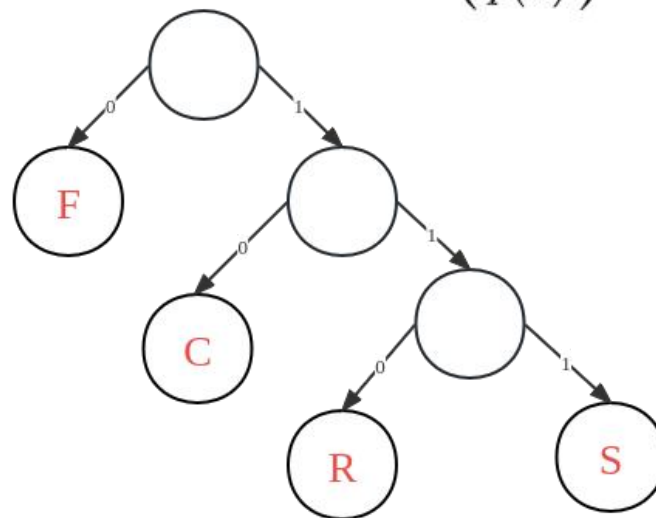
$$H(x) = - \int P(x) \log_2 P(x) dx$$

F(50.0%)	C(25.0%)	R(12.5%)	S(12.5%)
----------	----------	----------	----------

把来自一个分布q的消息使用另一个分布p的最佳代码传达的平均消息长度) 称为交叉熵。

one-hot	F	C	R	S
实际值q	1	0	0	0
预测值p	1/2	1/4	1/8	1/8

$$H_p(q) = \sum_x q(x) \log_2 \left(\frac{1}{p(x)} \right) = - \sum_x q(x) \log_2 p(x)$$



用p表示q所需最佳编码平均长度

$$H_p(q) = - \{ 1 * \log_2 0.5 + 0 * \log_2 0.25 + 0 * \log_2 0.125 + 0 * \log_2 0.125 \} = 1$$

$$H_q(q) = - \{ 1 * \log_2 1 + 0 * \log_2 0.25 + 0 * \log_2 0.125 + 0 * \log_2 0.125 \} = 0$$

$$H_q(p) = \infty$$

KL散度

one-hot	F	C	R	S
实际值p	1	0	0	0
预测值p	1/2	1/4	1/8	1/8

实际值p最小编码长度

$H(p) = -1 \cdot \log_2 1 + \dots = 0$
用预测值q编码实际值p期望长度
 $H_q(p) = 1$

KL散度：熵和交叉熵之间的差

$$D_q(p) = H_q(p) - H(p)$$

KL散度：预测值为了表示实际值
编码长度期望变长了多少

设 $I = \int_{-\infty}^{+\infty} e^{-x^2} dx$, 那么

$$I^2 = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} e^{-(x+y)^2} dx dy$$

作极坐标变换 $r^2 = x^2 + y^2$, $dx dy = r dr d\theta$:

$$I^2 = \int_0^{2\pi} \int_0^\infty e^{-r^2} r dr d\theta = \frac{1}{2} \int_0^{2\pi} d\theta \int_0^\infty e^{-r^2} dr^2 = \pi$$

所以

$$I = \int_{-\infty}^{+\infty} e^{-x^2} dx = \sqrt{\pi}$$

实际值p信息熵

$$H(p) = -\int p \log_2 p dx$$

预测值q信息熵

$$H(q) = -\int q \log_2 q dx$$

预测值q编码实际值p期望长度

$$H(p||q) = -\int p \log_2 q dx$$

KL散度(q编码p - p编码p)

$$KL(p||q) = H(p||q) - H(p)$$

$$= -\int p \log_2 q / p dx$$

p,q均为高斯分布

$$p \sim N(\mu_1, \delta_1), \quad q \sim N(\mu_2, \delta_2)$$

$$\int \exp(-x^2) dx = \pi^{1/2}$$

$$\int x^2 \exp(-x^2) dx = \pi^{1/2} / 2$$

$$H(p(x)) = \int p(x) \log_2 p(x) dx$$

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$KL(p, q) = -\int p(x) \log q(x) dx + \int p(x) \log p(x) dx$$

$$= \frac{1}{2} \log(2\pi\sigma_2^2) + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} (1 + \log 2\pi\sigma_1^2)$$

$$= \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

$$q \sim N(0, 1) \quad p(\mu, \delta)$$

$$KL(p, q) = -\log \delta + \delta / 2 + \mu^2 / 2 - 1 / 2$$

```
# D_KL(Q(z|X) || P(z)); calculate in closed form as both dist. are Gaussian
# here we assume that \Sigma is a diagonal matrix, so as to simplify the computation
KLD = 0.5 * torch.sum( torch.exp(log_var) + torch.pow(mu, 2) - 1. - log_var)
# 0.5*( e^data + mu^2 - 1 - data)
# 0.5*data + 0.5*e^data + 0.5*mu^2 - 0.5
```

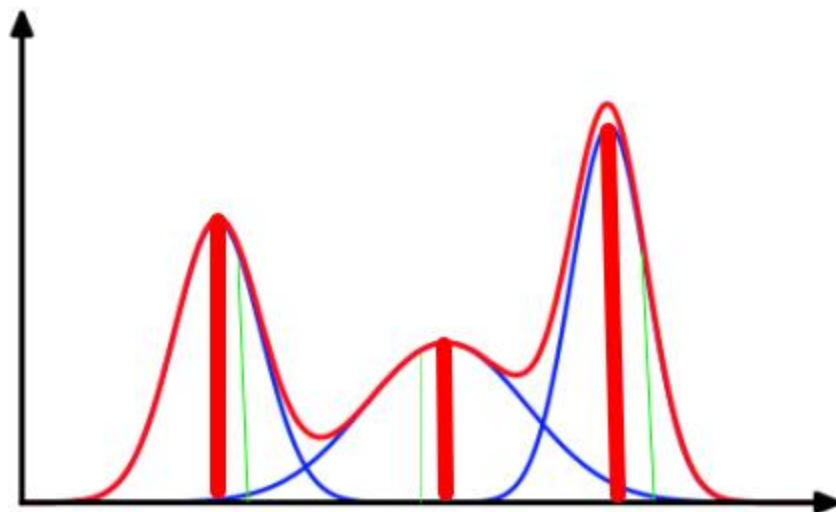
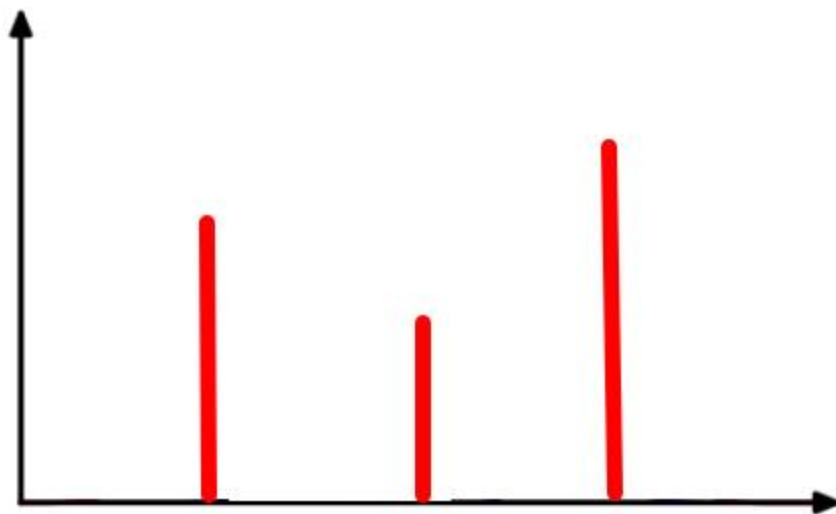
变分自编码器
VAE



目标1：构建从隐变量 Z 生成目标数据 X 的模型

$$\text{Loss1} = \text{BCE}(X, X^R)$$

X 和 X^R 的二值交叉熵



变分自编码器 VAE

主要目标：构建从隐变量 Z 生成目标数据 X 的模型

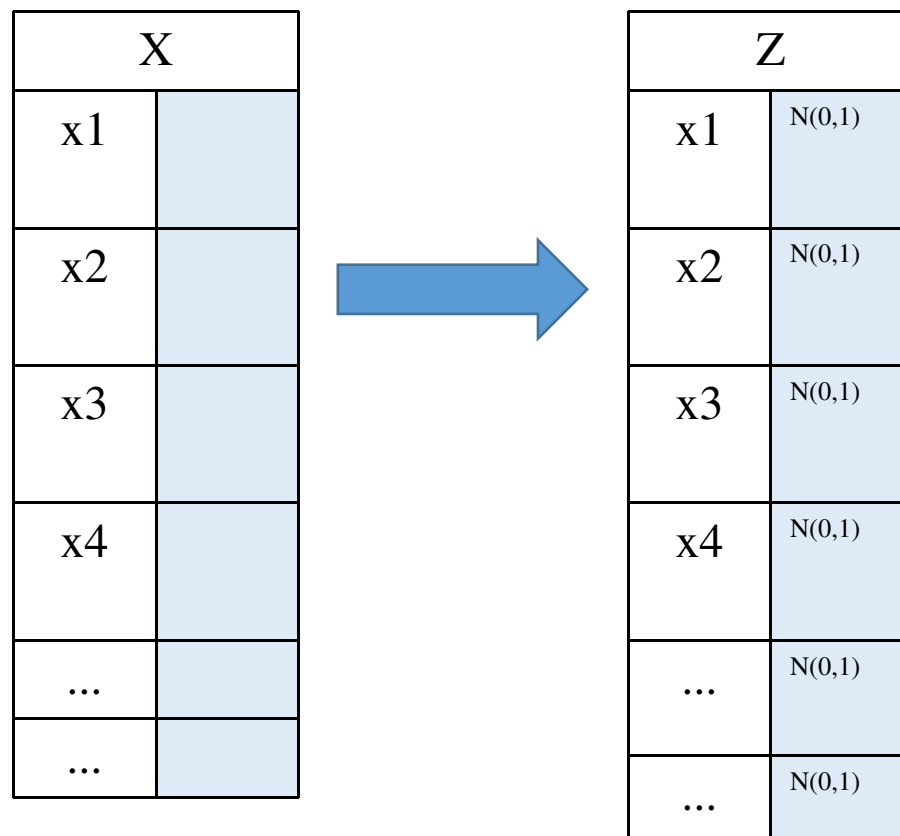
假设：样本服从某些常见的分布(eg:正态)

任意模型均可映射为高维标准正态分布



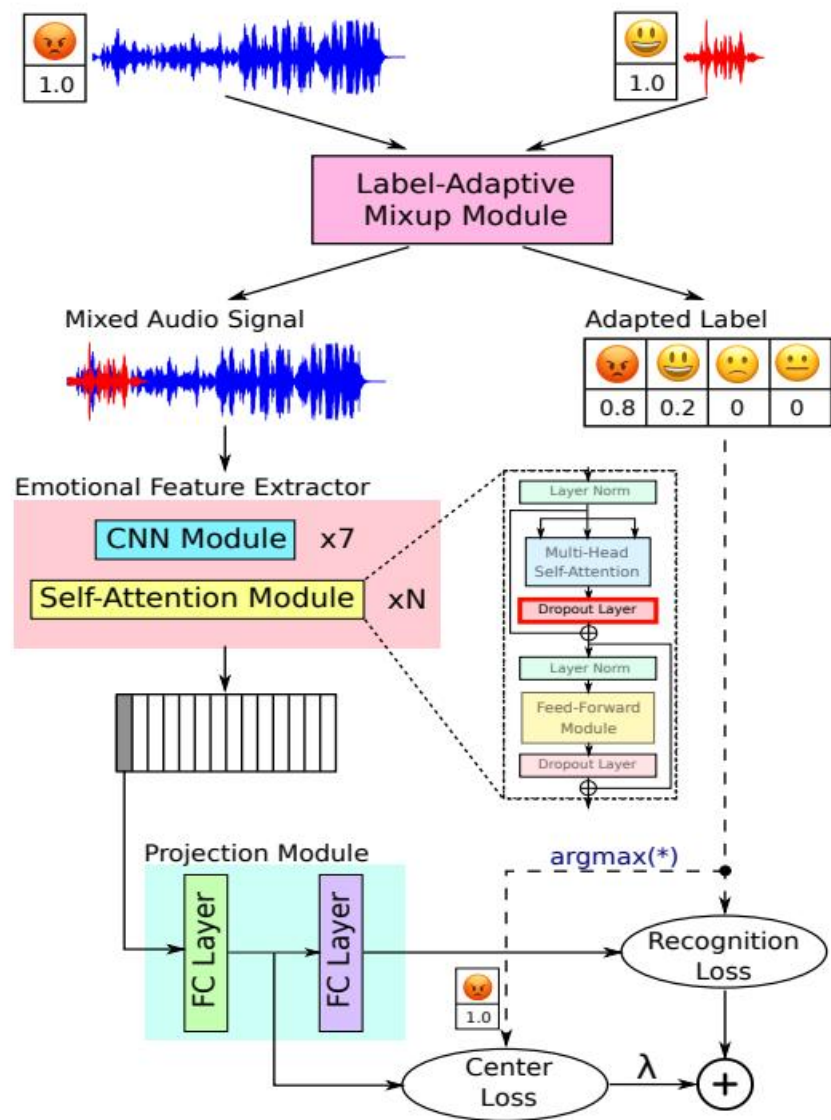
$p(Z)$: 先验分布, 假设为标准正态分布

$p(X|Z)$:后验分布,由 Z 来生成 X 的模型



2023/07/10

论文关键点
标签自适应混合方法
Label-Adaptive Mixup



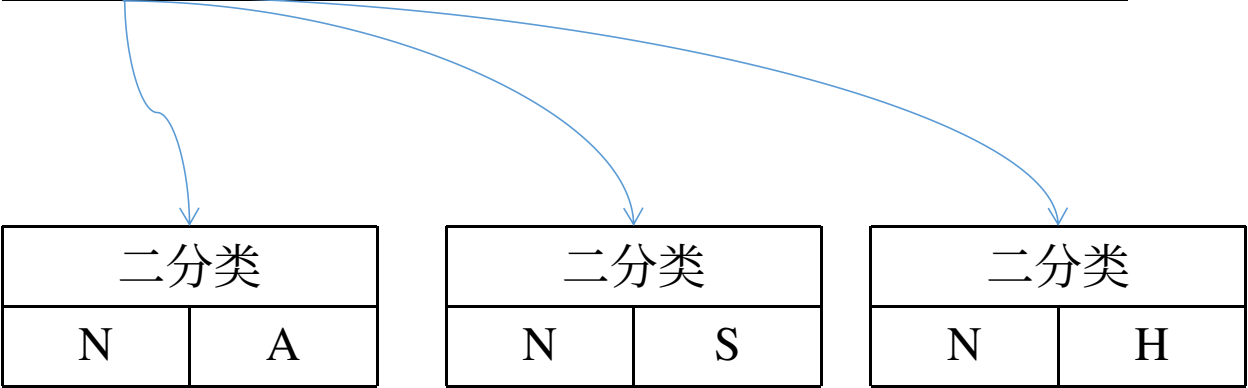
- 修改hu-bert
提出标签自适应混合方法
中心损失+识别损失

Method	Year	WA (%)	UA(%)
Human Performance [4]	2017	69.00	70.00
TDNN-LSTM-attn <i>et al.</i> [6]	2018	70.10	60.70
LSTM <i>et al.</i> [19]	2019	56.99	53.07
IS09-classification <i>et al.</i> [7]	2019	64.33	64.79
CNN-GRU-SeqCap <i>et al.</i> [20]	2019	72.73	59.71
HGFM <i>et al.</i> [21]	2020	66.60	70.50
ACNN <i>et al.</i> [5]	2020	67.28	67.94
ASR-SER <i>et al.</i> [22]	2020	68.60	69.70
Lightweight model <i>et al.</i> [1]	2020	70.39	71.72
SSL&CMKT fusion <i>et al.</i> [23]	2021	61.16	62.50
Audio _{25,250} +BERT <i>et al.</i> [2]	2021	69.44	70.90
Selective MTL <i>et al.</i> [24]	2022	56.87	59.47
MFCC+Spectrogram+W2E <i>et al.</i> [18]	2022	69.80	71.05
CNN-SeqCap <i>et al.</i> [3]	2022	70.54	56.94
Proposed	2023	75.37	76.04

加噪声(现实意义的)
多次分类

Input: spectrogram 200x300 (4000Hz x 3 sec)
Convolution 1: 16 filters of 12x16 (240Hz x 160msec)
Max-pooling 2:1 → 100x150
Convolution 2: 24 filters of 8x12 (320Hz x 240 msec)
Max-pooling 2:1 → 50x75
Convolution 3: 32 filters of 5x7 (400Hz x 280 msec)
Max-pooling 2:1 → 25x37
LSTM: bi-directional, 128x2
Dense layer: length 64
Dropout: length 64
Soft-max: length 4
Output: 4 posterior probabilities

四分类			
N	A	S	H



Network		Overall accuracy	Class accuracy
1.	5 convolution layers 10Hz grid resolution	66.1%	56.6%
2.	5 convolution layers 20 Hz grid resolution	63.6%	53.4%
3.	3 convolution layers and LSTM 10 Hz grid resolution	68.8%	59.4%
4.	3 convolution layers and LSTM 20 Hz grid resolution	65.8%	56.6%
5.	Two-step predictor (refer to explanation below) 10 Hz grid resolution	67.3%	62.0%

流形学习 (manifold learning) 假设数据在高维空间的分布位于某一更低维的流形